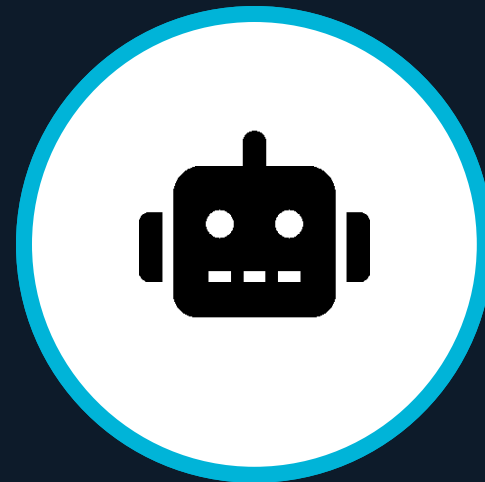


NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 1

Wprowadzenie do percepcji i nawigacji robota humanoidalnego

dr inż. Mateusz Pomianek



Percepcja

Mapowanie

Sensory

Architektura

Plan wykładu

Sekcja 1

Problem nawigacji humanoida, moduły systemu, ramy odniesienia, miary jakości

Sekcja 2

Sensory i percepcja: IMU, LiDAR, kamery, filtracja i synchronizacja danych

Sekcja 3

Reprezentacja środowiska: mapy zajętości, cost map, mapowanie lokalne/globalne

Kody Python

Struktury modułów, filtracja LiDAR, detekcja przeszkód, budowa grid map

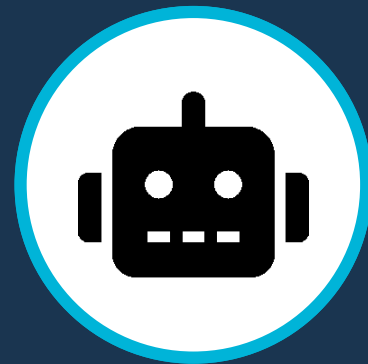
Najpierw omówimy sam problem nawigacji humanoida, główne moduły systemu, układy odniesienia oraz miary jakości. Następnie przejdziemy do sensorów i percepcji, czyli IMU, LiDAR-u, kamer, filtracji i synchronizacji danych. W trzeciej części zajmiemy się reprezentacją środowiska: mapami zajętości, cost mapami oraz mapowaniem lokalnym i globalnym. Po drodze będziemy też analizować krótkie przykłady w Pythonie.

01

SEKCJA

Problem nawigacji humanoida

*Definicja, architektura systemu i miary jakości
nawigacji dwunożnej*



Nawigacja humanoida a platforma kołowa

Robot kołowy (AGV / różnicowy)

- Stabilna baza – zawsze min. 2 punkty kontaktu
- Prosta kinematyka – odometria kół wystarczy
- Brak ryzyka wywrotki przy ruchu do przodu
- Typowe środowisko: płaska posadzka
- Dojrzałe algorytmy SLAM i planowania

Robot humanoidalny

- Dynamiczna równowaga: stopa uniesiona w połowie cyklu
- Złożona kinematyka – 30+ stopni swobody
- Każdy krok to potencjalna utrata stabilności
- Środowisko: schody, progi, nierówności, drzwi
- Konieczność planowania kroków i trajektorii CoM

Proszę zwrócić uwagę, że humanoid nawigacyjnie bardzo różni się od platformy kołowej. Robot kołowy ma stabilną bazę, prostą kinematykę i zwykle porusza się po płaskiej posadzce, więc wiele klasycznych metod działa tam bardzo dobrze. Humanoid natomiast funkcjonuje w warunkach dynamicznej równowagi, ma złożoną kinematykę, ryzyko utraty stabilności przy każdym kroku i musi radzić sobie ze schodami, progami, nierównościami oraz drzwiami. Dlatego w jego przypadku sama trajektoria nie wystarcza, trzeba planować także kroki i trajektorię środka masy.

Główne moduły systemu nawigacji

Percepcja

IMU, enkodery, kamery,
LiDAR, czujniki sił

Lokalizacja

Estymacja stanu, fuzja
sensoryczna, SLAM

Mapowanie

Occupancy grid, height map,
mapa semantyczna

Planowanie globalne

Dijkstra, A*, graph-based
path planning

Planowanie lokalne

DWA, potential fields, MPC

Sterowanie chodem

ZMP, LIPM, footstep
planning, WBC

Typowy system nawigacji humanoida składa się z sześciu głównych modułów. Najpierw mamy percepcję, która zbiera dane z IMU, enkoderów, kamer, LiDAR-u i czujników sił. Potem lokalizacja, czyli estymacja stanu, fuzja sensoryczna i często SLAM. Dalej mapowanie, na przykład occupancy grid, height map albo mapa semantyczna. Na tej podstawie działa planowanie globalne, później planowanie lokalne, a na końcu sterowanie chodem, które zamienia decyzje planistyczne na rzeczywiste kroki, trajektorie CoM i komendy całego ciała.

Przepływ informacji w systemie nawigacji

1. **Sensory** → **Percepcja**: surowe dane z IMU, enkoderów, LiDAR, kamer
2. **Percepcja** → **Estymacja stanu**: przefiltrowane obserwacje do fuzji sensorycznej
3. **Estymacja stanu** → **Planista globalny**: aktualna pozycja i mapa środowiska
4. **Planista globalny** → **Planista lokalny**: globalna ścieżka i mapa kosztów
5. **Planista lokalny** → **Generator kroków**: lokalna trajektoria do wykonania
6. **Generator kroków** → **Kontroler chodu**: sekwencja ustawień stóp i trajektoria CoM
7. **Kontroler chodu** → **Aktuatory**: komendy momentów i pozycji stawów
8. **Aktuatory** → **Sensory**: zamknięta pętla sprzężenia zwrotnego

Przepływ informacji w takim systemie jest sekwencyjny, ale zamknięty pętlą sprzężenia zwrotnego. Sensory dostarczają surowe dane do percepcji, percepcja przekazuje przefiltrowane obserwacje do estymacji stanu, a estymator podaje aktualną pozycję i mapę do planisty globalnego. Następnie planista globalny wyznacza ścieżkę, planista lokalny dopasowuje ją do bieżącej sytuacji, generator kroków tworzy sekwencję ustawień stóp, kontroler chodu zamienia to na trajektorie i momenty, a aktuatory wykonują ruch. Potem sensory znowu mierzą efekt tej akcji i pętla się zamyka.

Ramy odniesienia i układy współrzędnych

- **world** – bezwzględna ramka inercjalna, stała w przestrzeni
- **map** – ramka mapy globalnej; związana z world przez transformację SLAM
- **odom** – ramka odometrii; ciągła, ale kumuluje dryft w długim horyzoncie
- **base** – środek ciężkości lub pelvis robota; baza kinematyki odwrotnej
- **foot_L / foot_R** – ramki stóp; kluczowe podczas fazy podparcia
- **camera** – ramka optyczna kamery; kalibracja extrinsic względem base
- **lidar** – ramka skanera; wymaga precyzyjnej kalibracji kąta i przesunięcia
- **Drzewo transformacji TF (ROS):** spójna hierarchia wszystkich układów

W nawigacji musimy bardzo precyzyjnie rozróżniać układy współrzędnych. Ramka world jest inercjalna i stała w przestrzeni. Ramka map opisuje mapę globalną. Ramka odom jest ciągła, ale z czasem akumuluje dryft. Ramka base jest związana z miednicą albo środkiem ciężkości robota. Do tego dochodzą ramki stóp, kamery i LiDAR-u. Całość musi być spięta spójnym drzewem transformacji TF, bo bez tego nawet bardzo dobre algorytmy zaczną produkować błędne wyniki.

Ruch całego ciała a nawigacja bazowa

Nawigacja bazowa (CoM / pelvis)

- Trajektoria środka masy w przestrzeni 3D
- Pozycja miednicy i orientacja tułowia
- Globalny kierunek i prędkość przemieszania
- Wejście do globalnego i lokalnego planisty

Ruch kończyn (whole-body)

- Sekwencja ustawień stóp i czas kontaktu
- Trajektorie końcówek roboczych rąk
- Ruchy kompensacyjne tułowia dla równowagi
- Wyjście kontrolera chodu do aktuatorów

Warto odróżnić nawigację bazową od ruchu całego ciała. W nawigacji bazowej interesuje nas przede wszystkim trajektoria środka masy, pozycja miednicy, orientacja tułowia, globalny kierunek ruchu i prędkość przemieszczania. Natomiast ruch kończyn obejmuje sekwencję ustawień stóp, trajektorie rąk oraz ruchy kompensacyjne tułowia niezbędne do utrzymania równowagi. Innymi słowy: planista myśli o tym, dokąd robot ma iść, a kontroler całego ciała myśli o tym, jak dokładnie ma to wykonać.

Miary jakości nawigacji

Dokładność lokalizacji

RMSE pozycji [m],
błąd orientacji [°]

Czas dotarcia do celu

Czas od startu do osiągnięcia
punktu docelowego [s]

Zużycie energii

Sumaryczny pobór mocy
aktuatorów [Wh/m]

Liczba korekt kroku

Ponowne planowanie stóp,
wskaźnik reaktywności

Liczba kolizji

Kontakt z przeszkodą /
upadek w trakcie misji

Zapas stabilności

Marginesy ZMP i Capture
Point względem granicy

Jakość nawigacji mierzymy wieloma wskaźnikami jednocześnie. Interesuje nas dokładność lokalizacji, czyli na przykład RMSE pozycji i błąd orientacji. Interesuje nas czas dotarcia do celu, zużycie energii, liczba korekt kroku, liczba kolizji oraz zapas stabilności, wyrażony choćby przez marginesy ZMP i Capture Point. Dobra nawigacja humanoida nie jest tylko szybka — ona musi być jednocześnie dokładna, bezpieczna i stabilna.

Środowiska pracy humanoida

- **Korytarze biurowe** – wąskie przejścia, dynamiczne przeszkody (ludzie), równe podłoże
- **Schody i rampy** – zmienne wysokości, konieczność planowania kroków 3D
- **Drzwi i progi** – precyzyjny ruch chwytnych, klamki, siłowniki, progi podłogowe
- **Nierówne podłoże** – gruz, kostka brukowa, miękkie nawierzchnie, pochyłości terenu
- **Otoczenie ludzkie** – przewidywanie ruchu, strefa bezpieczeństwa, protokoły interakcji
- **Środowiska zmieniające się dynamicznie** – przesuwane meble, otwierane drzwi, ruchome obiekty

Humanoid pracuje w środowiskach znacznie trudniejszych niż klasyczny robot mobilny. W korytarzach biurowych ma wąskie przejścia i ludzi jako przeszkody dynamiczne. Na schodach i rampach musi uwzględniać wysokość terenu. Przy drzwiach i progach potrzebuje precyzji ruchu oraz często manipulacji. Na nierównym podłożu dochodzi problem kontaktu stopy z gruntem. W otoczeniu ludzkim pojawia się kwestia bezpieczeństwa i przewidywania ruchu człowieka, a w środowiskach dynamicznych mapa może się zmieniać praktycznie cały czas.

Studium przypadku: robot w budynku biurowym

- **Misja:** przemieszczenie z punktu A (wejście) do punktu B (sala konf.) w budynku 3-piętrowym
- **Etap 1 – Lokalizacja startowa:** inicjalizacja SLAM, wczytanie mapy budynku
- **Etap 2 – Planowanie globalne:** wyznaczenie trasy A^* z uwzględnieniem wind lub schodów
- **Etap 3 – Nawigacja korytarzem:** DWA z detekcją ludzi i dynamicznym omijaniem
- **Etap 4 – Przejście przez drzwi:** detekcja klamki, planowanie kroków przy progu
- **Etap 5 – Weryfikacja celu:** potwierdzenie lokalizacji względem mapy semantycznej
- **Kluczowe wyzwania:** synchronizacja sensorów, obsługa wind, interakcja z ludźmi

Rozważmy prosty przypadek: robot ma przejść od wejścia do sali konferencyjnej w trzypiętrowym biurze. Najpierw musi ustalić pozycję startową i zainicjalizować SLAM. Potem planuje globalną trasę z uwzględnieniem schodów albo windy. Następnie porusza się korytarzem, omijając ludzi i inne przeszkody dynamiczne. Gdy napotyka drzwi, musi wykryć klamkę, próg i zaplanować kroki. Na końcu weryfikuje osiągnięcie celu względem mapy semantycznej. Ten przykład pokazuje, że w praktyce nie ma jednego problemu nawigacji, jest cały łańcuch problemów połączonych w jeden system.

Struktura modułów systemu nawigacji

```
class SensorManager:
    def __init__(self):
        self.imu_data = None
        self.lidar_scan = None
        self.depth_frame = None
    def update(self, imu, lidar, depth):
        self.imu_data, self.lidar_scan, self.depth_frame = imu, lidar, depth

class StateEstimator:
    def estimate(self, sensors: SensorManager) -> dict:
        return {'pos': [0.0, 0.0, 0.0], 'yaw': 0.0, 'contact': [True, False]}

class Planner:
    def plan(self, state: dict, goal: list) -> list:
        return [] # lista kroków (footsteps)

class GaitController:
    def execute(self, footsteps: list) -> list:
        return [] # komendy stawowe [Nm]
```

Opis: Szkielet architektury oprogramowania – cztery główne klasy systemu nawigacji humanoida.

02

SEKCJA

Sensory i percepcja

Charakterystyka sensorów, propriocepcja, percepcja wizyjna, LiDAR i filtracja danych



Sensory stosowane w humanoidzie

IMU

Żyroskop + akcelerometr; orientacja i prędkość kątowna, $f = 100\text{-}1000\text{ Hz}$

Enkodery

Kąty stawów, prędkości; $f = 100\text{-}500\text{ Hz}$, wysoka rozdzielczość

Kamera RGB

Obraz kolorowy; detekcja obiektów, śledzenie, rozpoznawanie sceny

Kamera RGB-D

Obraz + głębina (structured light / ToF); $30\text{-}90\text{ Hz}$, zasięg $0,3\text{-}5\text{ m}$

LiDAR

Chmura punktów 2D/3D; dokładność $\sim 2\text{ cm}$, zasięg do 100 m

Czujniki sił

Siły i momenty w stopach (6-DoF F/T); detekcja kontaktu i fazy chodu

Humanoid korzysta z kilku klas sensorów jednocześnie. IMU daje orientację i prędkość kątowną przy bardzo dużej częstotliwości. Enkodery dostarczają kąty i prędkości stawów. Kamera RGB umożliwia detekcję obiektów i rozpoznawanie sceny. Kamera RGB-D dodaje do obrazu głębiny, co jest bardzo użyteczne przy nawigacji w pomieszczeniach. LiDAR daje precyzyjną geometrię otoczenia, a czujniki sił w stopach pozwalają wykrywać kontakt i fazę chodu. Każdy z tych sensorów widzi inny fragment rzeczywistości, dlatego pojedynczy sensor prawie nigdy nie wystarcza.

Kluczowe parametry sensorów

- **Zasięg:** od kilku cm (kamery RGB-D) do 100 m (LiDAR zewnętrzny)
- **Częstotliwość:** IMU 200-1000 Hz, kamery 30-90 Hz, LiDAR 10-20 Hz
- **Szum i dryft:** IMU dryfuje $\sim 1^\circ/\text{min}$; kamera wrażliwa na oświetlenie
- **Opóźnienie (latencja):** kamera 30-100 ms; IMU < 5 ms – krytyczne dla stabilności
- **Koszt obliczeniowy:** przetwarzanie chmur punktów i segmentacja semantyczna to $\sim 80\%$ CPU
- **Odporność środowiskowa:** LiDAR wrażliwy na kurz, kamera na silne kontrastowe oświetlenie
- **Kalibracja extrinsic:** każdy sensor wymaga precyzyjnego wyznaczenia pozycji względem base

Przy doborze sensorów patrzymy na kilka parametrów. Po pierwsze zasięg, bo kamera głębi pracuje na krótkich dystansach, a LiDAR może sięgać dziesiątek metrów. Po drugie częstotliwość, bo IMU pracuje dużo szybciej niż kamera czy LiDAR. Po trzecie szum i dryft, które są krytyczne dla estymacji. Dalej mamy latencję, koszt obliczeniowy, odporność na warunki środowiskowe i wreszcie kalibrację extrinsic, czyli dokładne położenie każdego sensora względem bazy robota. Sam zakup sensora nie rozwiązuje problemu, równie ważna jest jego poprawna integracja.

IMU i propriocepcja

- **Żyroskop:** pomiar prędkości kątowej [rad/s] – integracja daje orientację, ale dryfuje
- **Akcelerometr:** siły inercjalne [m/s²] – wyznaczenie kierunku grawitacji = kąt nachylenia
- **Enkodery stawowe:** konfiguracja ciała $q(t)$; wejście do kinematyki prostej i odwrotnej
- **Kontakt stóp** (czujniki F/T): detekcja fazy podparcia – kluczowe dla odometrii humanoida
- **Propriocepcja** to minimum informacji do zamkniętej pętli równowagi bez zewnętrznych sensorów
- **Filtr komplementarny / EKF** łączy szybki żyroskop z wolnym, ale niedryfującym akcelerometrem

Propriocepcja to ta część percepcji, która mówi robotowi o nim samym. Żyroskop mierzy prędkość kątową, akcelerometr dostarcza informacji o grawitacji i przyspieszeniach, a enkodery stawowe dają konfigurację ciała w czasie. Czujniki sił w stopach pozwalają określić, czy stopa jest w kontakcie z podłożem. Te informacje są absolutnie kluczowe dla utrzymania równowagi i odometrii humanoida. Zwykle łączymy szybki, ale dryfujący żyroskop z wolniejszym, ale długoterminowo stabilnym akcelerometrem za pomocą filtru komplementarnego albo EKF.

Percepcja wizyjna 2D i 3D

Percepcja 2D (obraz RGB)

- **Detekcja cech** (ORB, SIFT): punkty orientacyjne do lokalizacji
- **Detekcja obiektów** (YOLO, SSD): ludzie, drzwi, schody
- **Segmentacja semantyczna**: klasy terenu (podłoga, ściana, przeszkoda)
- **Śledzenie** (KLT optical flow): ruch obiektów w scenie

Percepcja 3D (RGB-D / stereo)

- **Mapa głębi**: odległość do każdego piksela
- **Rekonstrukcja chmury punktów** otoczenia
- **Detekcja przeszkód w 3D**: weryfikacja wolnego przejścia
- **Szacowanie normalnych powierzchni**: kąt nachylenia terenu (wektory normalne)

W percepcji wizyjnej rozróżniamy podejście 2D i 3D. W obrazie RGB możemy wykrywać cechy, obiekty, klasy semantyczne i śledzić ruch w scenie. W percepcji 3D, na przykład z RGB-D albo stereo, dostajemy głębię, rekonstrukcję chmury punktów, detekcję przeszkód przestrzennych i możliwość estymacji nachylenia powierzchni. Dla humanoida to istotne, bo on nie pyta tylko, czy coś stoi przed nim, ale też czy na danym fragmencie terenu można bezpiecznie postawić stopę.

LiDAR i chmury punktów

- **Zasada działania:** pomiar czasu przelotu impulsu laserowego (ToF) lub przesunięcia fazowego
- **Konfiguracje:** LiDAR 2D (skaner obrotowy), LiDAR 3D (wielowiązkowy, np. Velodyne 16-64)
- **Typowe parametry:** zasięg 5-100 m, kąt 360° , rozdzielczość kątowa $0,1-0,4^\circ$, $f = 10-20$ Hz
- **Reprezentacja:** chmura punktów $\{(x, y, z, \text{intensity})\}$ – surowy format PCL
- **Zalety w środowisku wewnętrznym:** niezależność od oświetlenia, precyzja geometryczna
- **Ograniczenia:** brak informacji o kolorze i teksturze; problemy z lustrami i przezroczystymi powierzchniami
- **Zastosowanie:** budowa mapy 2D/3D, lokalizacja ICP, detekcja przeszkód i wolnej przestrzeni

LiDAR jest jednym z najważniejszych sensorów geometrycznych. Działa na zasadzie pomiaru czasu przelotu albo przesunięcia fazowego impulsu laserowego. Może być 2D lub 3D, ma duży zasięg, pełne pole widzenia i dobrą dokładność geometryczną. Jego ogromną zaletą w pomieszczeniach jest niezależność od oświetlenia. Ograniczenia też są istotne: LiDAR nie daje koloru ani tekstury, ma problemy z lustrami i przezroczystymi powierzchniami, ale mimo to świetnie nadaje się do budowy map, lokalizacji, detekcji przeszkód i wyznaczania wolnej przestrzeni.

Detekcja przeszkód statycznych i dynamicznych

Przeszkody statyczne

- Ściany, meble, kolumny – stałe w czasie
- Budowa globalnej mapy zajętości
- Klasyfikacja: wolna przestrzeń / zajęte / nieznane
- Wystarczy pojedynczy skan do detekcji

Przeszkody dynamiczne

- Ludzie, wózki, inne roboty – ruch w czasie
- Wymagana filtracja czasowa (ostatnie N skanów)
- Predykcja trajektorii (Kalman, LSTM)
- Strefa bezpieczeństwa zależna od prędkości obiektu

Musimy odróżniać przeszkody statyczne od dynamicznych. Ściany, meble czy kolumny są względnie stałe i nadają się do budowy mapy globalnej. Wystarczy często pojedynczy skan, aby je poprawnie sklasyfikować. Natomiast ludzie, wózki czy inne roboty poruszają się w czasie, więc potrzebujemy filtracji czasowej, śledzenia i często predykcji trajektorii. W przypadku obiektów dynamicznych strefa bezpieczeństwa powinna zależeć od ich prędkości i przewidywanego ruchu.

Filtracja danych sensorycznych

- **Filtr medianowy:** usuwa impulsowe błędy (*np. refleksy lustrzane w skanerze LiDAR*)
- **Średnia ruchoma (*moving average*):** wygładzanie szumowych serii pomiarowych
- **Progowanie odległości:** odrzucenie pomiarów poza zakresem $[d_min, d_max]$ sensora
- **Odrzucanie wartości odstających (*outlier removal*):** statystyczne odchylenie od sąsiedztwa
- **RANSAC:** dopasowanie płaszczyzny podłogi przy silnym szumie i dużej liczbie punktów
- **Voxel grid downsampling:** redukcja gęstości chmury punktów przed przetwarzaniem

Surowe dane sensoryczne niemal nigdy nie nadają się do bezpośredniego użycia. Stosujemy więc filtr medianowy, średnią ruchomą, progowanie odległości, odrzucanie obserwacji odstających, RANSAC do dopasowania płaszczyzny podłogi oraz voxel grid downsampling do redukcji rozmiaru chmury punktów. Celem nie jest upiększenie danych, tylko uzyskanie takiego sygnału, na którym estymacja i planowanie będą działały stabilnie i przewidywalnie.

Filtrowanie danych z LiDAR-a

```
import numpy as np

def filter_lidar_scan(ranges: list,
                      min_dist: float = 0.15,
                      max_dist: float = 10.0,
                      kernel: int = 5) -> np.ndarray:
    """Oczyszczanie skanu LiDAR: próg + filtr medianowy."""
    arr = np.array(ranges, dtype=float)
    # 1. Zamień wartości poza zakresem sensora na NaN
    arr[(arr < min_dist) | (arr > max_dist)] = np.nan
    # 2. Filtr medianowy - usuwa impulsowe błędy refleksji
    valid_mask = ~np.isnan(arr)
    indices = np.where(valid_mask)[0]
    for idx in indices:
        neighbors = arr[max(0, idx-kernel//2) : idx+kernel//2+1]
        arr[idx] = np.nanmedian(neighbors)
    return arr
```

Opis: Oczyszczanie skanu: progowanie zasięgu i medianowe usuwanie błędnych odbić.

Detekcja przeszkody z kamery głębi

```
import numpy as np

def detect_obstacle(depth_image: np.ndarray,
                    roi_top: int = 100, roi_bottom: int = 300,
                    roi_left: int = 200, roi_right: int = 440,
                    safety_dist: float = 0.6) -> bool:
    """Sprawdz, czy w obszarze ROI jest przeszkoda bliżej niż safety_dist [m]."""
    region = depth_image[roi_top:roi_bottom, roi_left:roi_right]
    # Pomijamy błędne piksele (0 = brak pomiaru)
    valid = region[(region > 0) & (region < 8000)]
    if valid.size == 0:
        return False # Brak danych - brak kolizji
    min_depth_m = valid.min() / 1000.0 # mm -> m
    collision_alarm = min_depth_m < safety_dist
    return collision_alarm
```

Opis: ROI to centralny kadr obrazu głębi; wartości w mm konwertowane na metry.

Synchronizacja czasowa sensorów

1. **Problem:** IMU działa z $f = 200$ Hz, kamera 30 Hz, LiDAR 10 Hz – różne timestamps
2. **Każdy sensor powinien dołączać znacznik czasu** (ROS: `Header.stamp`) przy akwizycji
3. **Buforowanie wiadomości:** kolejka FIFO z timestampami dla każdego sensora
4. **Interpolacja liniowa:** wyznaczenie stanu sensora A w chwili pomiaru sensora B
5. **Synchronizator aproksymacyjny** (ROS: `message_filters`): dobór pomiarów w oknie czasowym
6. **Efekt braku synchronizacji:** błędy fuzji i duchy przeszkód w chmurze punktów

Bardzo ważna jest synchronizacja czasowa sensorów. IMU może pracować z częstotliwością 200 herców, kamera z 30, a LiDAR z 10, więc jeśli nie będziemy pilnować znaczników czasu, to zaczniemy łączyć pomiary pochodzące z różnych chwil. W praktyce każdy sensor powinien publikować timestamp przy akwizycji, dane powinny być buforowane, a brakujące chwile odtwarzane przez interpolację albo synchronizację aproksymacyjną. Bez tego w fuzji pojawiają się błędy, a w chmurach punktów mogą powstawać tak zwane duchy przeszkód.

03

SEKCJA

Reprezentacja środowiska i mapowanie

*Typy map, mapa 2D kontra 3D, koszt przejścia,
mapowanie lokalne i globalne*



Reprezentacje map środowiska

Occupancy Grid

Siatka 2D/3D komórek:
zajęte/wolne/nieznane; prostota,
wysoka pamięć

Voxel Map

Siatka 3D wokseli (np. OctoMap);
kompresja drzewem oktalnym

Point Cloud Map

Nieskompresowana chmura
punktów; wysoka wierność, duże
wymagania RAM

Topological Map

Graf węzłów i krawędzi; szybkie
planowanie, niska dokładność
geometryczna

Semantic Map

Mapa wzbogacona o etykiety klas
(podłoga, ściana, drzwi, stół)

Height Map

Mapa wysokości terenu (elevation
map); kluczowa dla planowania
kroków

Środowisko można reprezentować na różne sposoby. Occupancy grid to siatka komórek zajęte, wolne albo nieznane. Voxel map jest przestrzennym odpowiednikiem siatki 3D, często kompresowanym przez drzewo oktalne. Point cloud map daje wysoką wierność, ale jest pamięciożerna. Topological map upraszcza środowisko do grafu relacji, a semantic map dodaje etykiety znaczeniowe. Dla humanoida szczególnie ważna jest też height map, bo wysokość terenu bezpośrednio wpływa na planowanie kroków.

Mapa 2D kontra mapa 3D dla humanoida

Kiedy wystarczy mapa 2D

- Równe, płaskie podłoże (biuro, magazyn)
- Planowanie trasy i unikanie dużych przeszkód
- Mała pamięć i szybkie przetwarzanie
- Kompatybilność z klasycznym Nav Stack (ROS)
- Algorytmy A*, Dijkstra bezpośrednio na grid

Kiedy potrzebna jest mapa 3D

- Schody, rampy, progi: wymagana wysokość terenu
- Planowanie stóp z uwzględnieniem nierówności
- Wolna przestrzeń nad stopą (clearance)
- Wykrywanie nawisów i przeszkód górną
- Nawigacja po nierównym terenie zewnętrznym

Mapa 2D wystarcza wtedy, gdy środowisko jest płaskie i interesuje nas głównie wyznaczenie ścieżki oraz omijanie dużych przeszkód. Jest szybka, oszczędna pamięciowo i zgodna z klasycznym stosem nawigacyjnym. Natomiast mapa 3D jest konieczna, gdy pojawiają się schody, rampy, progi, nierówności, nawisy albo przeszkody nad stopą. Humanoid porusza się w przestrzeni, w której wysokość naprawdę ma znaczenie, więc w wielu zastosowaniach 2D okazuje się niewystarczające.

Koszt przejścia w środowisku humanoida

- **Teren łatwy:** niska wartość kosztu – szeroki, płaski, bez przeszkód w pobliżu
- **Teren wąski:** podwyższony koszt – ograniczone miejsce na krok i manewr tułowia
- **Teren pochyły:** koszt zależny od kąta nachylenia i kierunku ruchu (pod górę = więcej energii)
- **Teren śliski:** wysoki koszt lub blokada – niestabilny kontakt stóp, ryzyko poślizgu
- **Inflacja wokół przeszkód:** sąsiedztwo obiektu ma podwyższony koszt (gradient odpychający)
- **Funkcja kosztu:** $C(x,y) = w_dist * d_obstacle + w_slope * slope + w_narrow * clearance_penalty$
- **Cel:** preferowanie trasy bezpiecznej, nie tylko najkrótszej geometrycznie

Koszt przejścia nie może zależeć wyłącznie od długości trasy. Teren szeroki, płaski i daleki od przeszkód powinien mieć niski koszt. Teren wąski, pochyły albo śliski powinien mieć koszt wyższy lub wręcz być zablokowany. Dodatkowo stosujemy inflację wokół przeszkód, czyli gradient kosztu, który odpycha trajektorię od obiektów. W praktyce funkcja kosztu uwzględnia odległość od przeszkód, nachylenie terenu i karę za mały prześwit. Celem jest znalezienie trasy nie najkrótszej geometrycznie, lecz najbezpieczniejszej i najbardziej stabilnej.

Budowa prostej mapy zajętości (*occupancy grid*)

```
import numpy as np

def build_occupancy_grid(obstacle_points: list,
                        grid_size: int = 100,
                        resolution: float = 0.05) -> np.ndarray:
    """Tworzy siatkę zajętości z listy punktów przeszkód [m]."""
    grid = np.zeros((grid_size, grid_size), dtype=np.uint8)
    origin_x = origin_y = grid_size // 2
    for (px, py) in obstacle_points:
        cell_x = int(origin_x + px / resolution)
        cell_y = int(origin_y + py / resolution)
        if 0 <= cell_x < grid_size and 0 <= cell_y < grid_size:
            grid[cell_y, cell_x] = 100 # zajęte
    return grid
```

Opis: Siatka 100x100 komórek, rozdzielczość 5 cm/komórkę; wartość 100 = zajęte, 0 = wolne

Obliczanie mapy kosztów z inflacją

```
from scipy.ndimage import distance_transform_edt
import numpy as np

def compute_cost_map(grid: np.ndarray,
                    inflation_radius: float = 3.0,
                    max_cost: float = 100.0) -> np.ndarray:
    """Dodaje gradient kosztu wokół przeszkód (inflacja)."""
    obstacle_mask = (grid >= 100).astype(np.uint8)
    # Odleglosc euklidesowa kazdej komorki od najblizszej przeszkody
    dist = distance_transform_edt(1 - obstacle_mask)
    # Koszt: max dla przeszkody, maleje z odlegloscia
    cost = np.clip(max_cost * (1 - dist / inflation_radius), 0, max_cost)
    cost[obstacle_mask == 1] = max_cost
    return cost.astype(np.float32)
```

Opis: Inflacja tworzy gradient kosztu wokół przeszkód. Planista preferuje trasy z dala od obiektów

Mapowanie lokalne i globalne

Mapa lokalna (*robocza*)

- Budowana z **bieżących pomiarów sensorów**
- **Ograniczony zasięg**: ~5-10 m od robota
- **Wysoka aktualność**: uwzględnia obiekty dynamiczne
- Podstawa decyzji **planisty lokalnego**
- **Regularnie czyszczona** i przebudowywana

Mapa globalna (*trwała*)

- Budowana w **długim horyzoncie** (SLAM)
- **Zasięg**: cały budynek, piętro
- Wolniej aktualizowana, mniej dynamiczna
- Podstawa planowania **globalnej trasy A***
- Może zawierać **informacje semantyczne**

W praktyce prawie zawsze rozdzielamy mapowanie lokalne i globalne. Mapa lokalna ma mały zasięg, jest bardzo aktualna, uwzględnia obiekty dynamiczne i służy planowaniu lokalnemu. Mapa globalna obejmuje całe piętro albo budynek, jest aktualizowana wolniej i stanowi podstawę planowania trasy długodystansowej. Często zawiera również informacje semantyczne. To rozdzielenie jest kompromisem między aktualnością a skalą środowiska.

Ograniczenia mapowania w czasie rzeczywistym

1. **Pamięć:** mapa 3D OctoMap z rozdzielczością 5 cm dla biura 50x50 m to setki MB RAM
2. **Rozdzielczość, a wydajność:** wyższa rozdzielczość = dokładność wzrasta, czas przetwarzania rośnie
3. **Opóźnienia (*latencja*):** SLAM nie może działać wolniej niż cykl sterowania (*~100 ms*)
4. **Zmienność środowiska:** dynamiczne przeszkody nie mogą na stałe zaznaczać komórek jako zajętych
5. **Kompromis:** optymalny dobór rozdzielczości i zasięgu mapy do konkretnego zadania
6. **Strategie:** rozdzielona mapa lokalna (*wyższa rozdzielczość*) i globalna (*niższa rozdzielczość*)

Mapowanie w czasie rzeczywistym ma poważne ograniczenia. Duże mapy 3D zużywają setki megabajtów pamięci, a zwiększanie rozdzielczości poprawia dokładność kosztem czasu obliczeń. Do tego SLAM nie może być wolniejszy niż pętla sterowania, a środowisko dynamiczne nie może być na stałe zapisywane jako zajęte. W praktyce trzeba dobrać kompromis między zasięgiem, rozdzielczością i aktualnością mapy, często stosując wyższą rozdzielczość lokalnie i niższą globalnie.

PODSUMOWANIE

- 01 Nawigacja humanoida wymaga integracji percepcji, lokalizacji, planowania i sterowania chodem.
- 02 Sensory (IMU, LiDAR, RGB-D) dostarczają komplementarnych danych o otoczeniu i stanie robota.
- 03 Filtracja i synchronizacja danych sensorycznych to warunek niezawodnej percepcji.
- 04 Reprezentacja mapy (siatka zajętości, mapa wokselowa) warunkuje skuteczne planowanie.
- 05 Funkcja kosztu przejścia musi uwzględniać stabilność, a nie tylko geometrie trasy.

Pytania problemowe i zadanie do analizy

1. Jakie sensory wybrałbyś dla humanoida patrolującego biuro w nocy (słabe oświetlenie)?
2. Dlaczego synchronizacja IMU z kamerą jest krytyczna dla odometrii wizyjno-inercyjnej?
3. Czym różni się occupancy grid od height map i kiedy potrzebujesz obu jednocześnie?
4. Jak zmodyfikować funkcję kosztu, aby robot unikał zawilgoconej posadzki w pobliżu toalety?

Zadanie

Zaprojektuj strukturę modułów nawigacji dla humanoida poruszającego się po schodach.

Jakich sensorów i jakich map potrzebujesz, których nie potrzebujesz na płaskiej podłodze?

NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 2

Lokalizacja, odometria i fuzja sensoryczna

dr inż. Mateusz Pomianek



Estymacja stanu

Filtry EKF/UKF

Fuzja sensoryczna

SLAM

Plan wykładu

Sekcja 1

Estymacja stanu: odometria humanoida, dryft, dead reckoning, układy transformacji

Sekcja 2

Filtry estymacyjne: komplementarny, EKF, UKF, cząsteczkowy, fuzja IMU+enkodery+wzrok

Sekcja 3

Lokalizacja globalna, SLAM, pose graph, domykanie pętli, dobór metody

Kody Python

Całkowanie IMU, odometria kroku, filtr komplementarny, EKF 2D, korekta z landmarkem

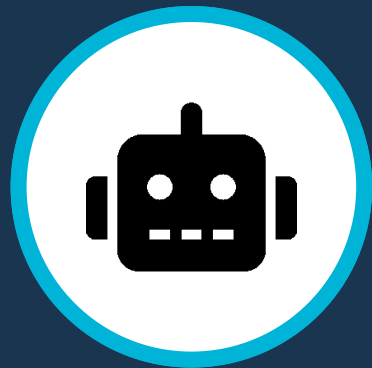
Plan tego wykładu obejmuje trzy główne bloki. Najpierw omówimy estymację stanu, odometrię humanoida, dryft, dead reckoning i układy transformacji. Następnie przejdziemy do filtrów estymacyjnych, takich jak filtr komplementarny, EKF, UKF i filtr cząsteczkowy, oraz do fuzji IMU, enkoderów i wzroku. Na końcu zajmiemy się lokalizacją globalną i SLAM-em, w tym pose graphem, domykaniem pętli i dobozem metody do zadania.

01

SEKCJA

Estymacja stanu robota

Odometria, dryft, dead reckoning i układy transformacji



Czym jest stan robota humanoidalnego?

Pozycja i orientacja

x, y, z [m] oraz kąty roll, pitch, yaw [rad] w przestrzeni 3D

Prędkości

liniowa v [m/s] i kątowa ω [rad/s]:
wejście do modelu predykcji

Konfiguracja stawów

Kąty $q(t)$ [rad]: kinematyka prosta, Jacobian, zakres ruchu nóg

Stan kontaktu stóp

Lewa/prawa stopa: podparta, czy w locie? Kluczowe dla odometrii

Środek masy (CoM)

Pozycja CoM w przestrzeni; związana z warunkiem stabilności ZMP

Niepewność estymacji

Macierz kowariancji P , czyli miary zaufania do szacowanego stanu

Stan robota humanoidalnego jest wielowymiarowy. Obejmuje pozycję i orientację w przestrzeni 3D, prędkości liniowe i kątowe, konfigurację stawów, stan kontaktu obu stóp, położenie środka masy oraz niepewność estymacji opisaną macierzą kowariancji. To bardzo ważne: w praktyce nie estymujemy tylko samej pozycji, ale cały pakiet zmiennych potrzebnych do stabilnego sterowania i wiarygodnego planowania.

Odometria humanoida vs odometria kołowa

Odometria kołowa (AGV)

- Zliczanie impulsów enkoderów napędowych
- Proste równania kinematyczne (v , ω)
- Dryf głównie z poślizgu i nierówności kół
- Przewidywalny, mały błąd na krótkich dystansach
- Dojrzałe algorytmy korekcji (AMCL, EKF)

Odometria humanoida

- Integracja kroków + IMU + kontakty stóp
- Stan kontaktu: stopa oparta = referencyjna
- Dryf ze śliskich podłóg, wibracji tułowia
- Niesymetria kroków i ugięcie struktury
- Wymagana fuzja z wizją lub LiDAR-em

Odometria humanoida jest trudniejsza niż odometria platformy kołowej. W robocie kołowym liczymy impulsy enkoderów i stosujemy proste równania kinematyczne. W humanoidzie musimy integrować kroki, IMU i kontakty stóp. Dryft wynika nie tylko z poślizgu, ale też z asymetrii kroków, wibracji tułowia i ugięć struktury. Dlatego odometria humanoida niemal zawsze wymaga korekcji zewnętrznej, na przykład przez wizję albo LiDAR.

Dryft i błędy estymacji

- **Kumulacja błędów:** każde całkowanie dodaje szum; po 100 krokach błąd pozycji może wynosić $>0,5\text{m}$
- **Poślizg stopy:** nagła zmiana kontaktu stopy powoduje skok w estymowanej pozycji bazowej
- **Błędy kalibracji:** nieprecyzyjne IMU (niezrównoważenie osi) lub enkodery (luz w przekładni)
- **Szum żyroskopu:** dryft yaw przy całkowaniu prędkości kątowej – typowo $0,1\text{--}1^\circ/\text{s}$
- **Opóźnienia transmisji:** przesunięcie fazowe między sensorami zakłóca fuzję
- **Model kinematyczny:** uproszczenia (sztywne ogniwa, brak ugięcia) powodują systematyczny błąd
- **Zmiana środowiska:** ruchome podłoże (dywan, rampa) nie jest uwzględnione w modelu

Główne źródła błędów estymacji to kumulacja szumu przy całkowaniu, poślizg stopy, błędy kalibracji, szum żyroskopu, opóźnienia transmisji i uproszczenia modelu kinematycznego. Dochodzi też wpływ środowiska, na przykład miękkiej nawierzchni albo rampy, których model nie uwzględnia. Wszystkie te zjawiska powodują, że błąd lokalizacji narasta z czasem i bez korekty zewnętrznej nie da się go zatrzymać.

Idea i ograniczenia Dead reckoning

Na czym polega

- Estymacja pozycji wyłącznie z modelu i pomiarów wewnętrznych (*IMU, enkodery*)
- Brak korekty z zewnętrznych punktów odniesienia
- Całkowanie prędkości: $x(t) = x_0 + \int v(\tau) d\tau$
- Wystarczające w krótkim horyzoncie czasowym
- Zastosowanie: awaryjne przejście przez strefę bez GPS/LiDAR

Dlaczego nie wystarczy

- Błąd rośnie z kwadratem czasu (*całkowanie szumu*)
- Bez korekty dryft yaw na poziomie kilku stopni/min
- Po 30 s marszu błąd pozycji może przekroczyć 1m
- Nie wykrywa zakłóceń środowiskowych (*poślizg*)
- Konieczna fuzja z SLAM, GPS lub znacznikami

Dead reckoning to estymacja pozycji wyłącznie na podstawie modelu ruchu i pomiarów wewnętrznych, takich jak IMU i enkodery. W krótkim horyzoncie działa zaskakująco dobrze i bywa użyteczne jako rozwiązanie awaryjne. Problem polega na tym, że błąd rośnie wraz z czasem, szczególnie przez dryft orientacji. Po kilkudziesięciu sekundach marszu odchylenie pozycji może być już bardzo duże. Dlatego dead reckoning samo w sobie nie wystarcza i musi być wspierane przez zewnętrzne punkty odniesienia, SLAM albo znaczniki.

Referencyjne układy: map, odom, base

- **world** – absolutna ramka inercjalna; stała; punkt odniesienia misji
- **map** – ramka mapy SLAM; skokowe korekty przy domknięciu pętli; world $\approx$ map w małych środowiskach
- **odom** – ramka odometrii; ciągła, ale dryfuje; baza dla planisty lokalnego
- **base** – miednica / CoM robota; centrum drzewa kinematyki
- **Relacja kluczowa:** odom→map to korekta SLAM; base→odom to bieżąca odometria
- **ROS 2 tf2:** każda transformacja niesie timestamp; stare transformacje są buforowane
- **Niespójne drzewo TF** jest najczęstszą przyczyną błędów nawigacji w ROS-ie

W systemie lokalizacji kluczowe są trzy ramki: map, odom i base. Ramka map jest związana z mapą SLAM i może być korygowana skokowo przy domknięciu pętli. Ramka odom jest ciągła, ale dryfuje. Ramka base opisuje aktualne położenie bazy robota. Relacja odom→map to korekta globalna, a relacja base→odom to bieżąca odometria. W ROS-ie każda transformacja ma znacznik czasu i jest buforowana. Niespójne drzewo TF to jeden z najczęstszych praktycznych powodów awarii lokalizacji.

Kontakt stopy jako źródło informacji

- **Faza podparcia:** stopa oparta na podłożu (może służyć jako punkt odniesienia odometrii)
- **Faza przenoszenia:** stopa w locie (brak danych pozycyjnych z tej nogi)
- **Czujniki F/T w stopie mierzą 6-DoF:** $F_x, F_y, F_z, M_x, M_y, M_z$
- **Detekcja kontaktu:** $F_z > \text{próg}$ (np. 30 N) $\rightarrow$ stopa oparta; przydatne przy miękkich nawierzchniach
- **Kinematyczna odometria kroków:** pozycja bazy = pozycja stopy + FK^{-1}
- **Brak kontaktu obu stóp jednocześnie:** skok lub upadek – należy zresetować estymator
- **Fuzja z IMU:** stopa jako warunek brzegowy do korekcji dryftu yaw i translacji

Kontakt stopy jest bardzo cennym źródłem informacji. Gdy stopa jest w fazie podparcia, może pełnić funkcję punktu odniesienia dla odometrii. Gdy jest w fazie przenoszenia, tej informacji nie mamy. Czujniki sił i momentów w stopach pozwalają wykryć kontakt przez próg siły pionowej. Na tej podstawie możemy budować kinematyczną odometrię kroków i korygować estymację bazy. W połączeniu z IMU daje to istotne ograniczenie dryftu translacji i orientacji.

Metryki jakości lokalizacji

RMSE pozycji

Pierwiastek średniokwadratowego błędu xy [m] względem ścieżki referencyjnej

Błąd orientacji

Średni błąd kąta yaw [°];
krytyczny dla planowania kroków

Dryft na minutę

Narastanie błędu pozycji bez korekty zewnętrznej [m/min]

Spójność estymatora

Czy kowariancja P odpowiada rzeczywistemu błędowi? (test NIS/NEES)

Jakość lokalizacji oceniamy przez RMSE pozycji, błąd orientacji, dryft na minutę oraz spójność estymatora. Ta ostatnia cecha jest bardzo ważna, bo nie wystarczy mieć mały błąd. Estymator musi też poprawnie oceniać własną niepewność, a to sprawdzamy testami takimi jak NIS czy NEES.

Całkowanie prędkości kątowej z IMU

```
import numpy as np

class ImuIntegrator:
    def __init__(self):
        self.yaw = 0.0 # [rad]
        self.pitch = 0.0 # [rad]
        self.roll = 0.0 # [rad]

    def update(self, gyro_xyz: np.ndarray, dt: float) -> tuple:
        """Aktualizacja orientacji przez całkowanie żyroskopu"""
        # gyro_xyz = [omega_x, omega_y, omega_z] w rad/s
        self.roll += gyro_xyz[0] * dt
        self.pitch += gyro_xyz[1] * dt
        self.yaw += gyro_xyz[2] * dt
        return self.roll, self.pitch, self.yaw

# Przykład użycia:
integrator = ImuIntegrator()
roll, pitch, yaw = integrator.update(np.array([0.01, -0.02, 0.15]), dt=0.005)
```

Opis: Proste całkowanie żyroskopu → szybkie, lecz podatne na dryft bez korekty zewnętrznej

Odometria kroku i aktualizacja pozycji robota

```
import numpy as np

class StepOdometry:
    def __init__(self):
        self.x = 0.0 # [m]
        self.y = 0.0 # [m]
        self.yaw = 0.0 # [rad]

    def step(self, stride_len: float, delta_yaw: float) -> tuple:
        """Aktualizacja pozycji po jednym kroku (stride)."""
        self.yaw += delta_yaw
        self.x += stride_len * np.cos(self.yaw)
        self.y += stride_len * np.sin(self.yaw)
        return self.x, self.y, self.yaw

odo = StepOdometry()
for _ in range(10):
    x, y, yaw = odo.step(stride_len=0.35, delta_yaw=0.02)
```

Opis: Każdy krok aktualizuje pozycję bazy przez kinematykę planarną; **delta_yaw** pochodzi z IMU

02

SEKCJA

Filtry estymacyjne i fuzja sensoryczna

*EKF, UKF, filtr cząsteczkowy, fuzja
IMU+enkodery+wzrok*



Po co łączyć sensory?

Komplementarność

Żyroskop: szybki, dryfuje. Akcelerometr: wolny, stabilny kąt. Razem: lepiej.

Redundancja

Awaria jednego sensora nie blokuje estymacji, jeśli mamy zapas informacji.

Wzajemna korekta

Kamera koryguje skumulowany dryft IMU; IMU stabilizuje estymację przy rozmazaniu

Pewność statystyczna

Kowariancja złożona < kowariancja pojedynczego sensora (*reguła filtracji*)

Sensory łączymy z czterech głównych powodów. Po pierwsze komplementarność, bo jeden sensor jest szybki, a drugi stabilny. Po drugie redundancja, bo awaria pojedynczego źródła nie powinna zatrzymać całej lokalizacji. Po trzecie wzajemna korekta, na przykład kamera koryguje dryft IMU, a IMU stabilizuje estymację obrazu. Po czwarte pewność statystyczna, bo połączona estymata może mieć mniejszą niepewność niż każdy sensor osobno.

Filtr komplementarny - intuicja

- **Idea:** połącz żyroskop (szybki, dryfuje) z akcelerometrem (wolny, stabilny w długim czasie)
- **Wzór:** $\theta(t) = \alpha \cdot (\theta_{\text{prev}} + \omega_{\text{gyro}} \cdot dt) + (1-\alpha) \cdot \theta_{\text{accel}}$
- **Parametr α :** blisko 1 (np. 0,98) – większe zaufanie do żyroskopu na krótkich interwałach
- **Działanie:** żyroskop śledzi szybkie zmiany; akcelerometr koryguje powolny dryft
- **Zalety:** niska złożoność obliczeniowa, brak macierzy – działa na mikrokontrolerze IMU
- **Wada:** nie uwzględnia niepewności pomiarowych (brak kowariancji) ani dynamiki zewnętrznej
- **Zastosowanie:** pochylenie tułowia robota w czasie rzeczywistym (roll i pitch)

Filtr komplementarny jest intuicyjny i bardzo użyteczny. Łączy szybki, ale dryfujący żyroskop z wolniejszym, lecz stabilnym akcelerometrem. Parametr alfa ustala proporcję zaufania do obu źródeł. Dzięki temu żyroskop śledzi szybkie zmiany, a akcelerometr powoli koryguje dryft. Zaletą jest prostota i mały koszt obliczeniowy. Wadą jest brak jawnej obsługi kowariancji i ograniczona możliwość modelowania pełnej dynamiki układu.

Filtr komplementarny dla kąta pochylenia

```
import numpy as np

class ComplementaryFilter:
    def __init__(self, alpha: float = 0.98):
        self.alpha = alpha
        self.pitch = 0.0 # estymowany kąt pochylenia [rad]

    def update(self, gyro_y: float, accel_xz: np.ndarray, dt: float) -> float:
        """Estymacja pitch z gyro i akcelerometru."""
        # Predykcja z żyroskopu
        pitch_gyro = self.pitch + gyro_y * dt
        # Pomiar kąta z akcelerometru (g_x i g_z)
        pitch_accel = np.arctan2(accel_xz[0], accel_xz[1])
        # Fuzja
        self.pitch = self.alpha * pitch_gyro + (1 - self.alpha) * pitch_accel
        return self.pitch
```

Opis: $\alpha \approx 0,98$: 98% z żyroskopu (szybkość) i 2% z akcelerometru (długoterminowa stabilność)

Porównanie filtrów: EKF, UKF, cząsteczkowy

EKF - Extended Kalman Filter

- **Linearyzacja Jacobianem** wokół aktualnego stanu
- **Niski koszt $O(n^2)$** : działa w czasie rzeczywistym
- Zakłada **Gaussowski rozkład szumu**
- **Zawodzi** przy silnej **nieliniowości modelu**
- **Złoty standard w robotyce mobilnej**

UKF - Unscented Kalman Filter

- **Sigma-points** zamiast **linearyzacji**
- **Lepsza** aproksymacja **nieliniowości**
- **Bez** konieczności wyliczania **Jacobiana**
- **Większy koszt** niż EKF ($2n+1$ punktów)
- **Zalecany** przy mocno **nieliniowych dynamikach**

Filtr cząsteczkowy (PF)

- **Wielomodalny** rozkład prawdopodobieństwa
- **N cząsteczek** jako hipotezy stanu
- **Resampling**: eliminacja słabych hipotez
- **Koszt $O(N)$** : wymaga wielu cząsteczek
- **Idealny** do lokalizacji globalnej (AMCL)

Jeżeli porównamy EKF, UKF i filtr cząsteczkowy, to każdy z nich ma inny profil zastosowań. EKF jest szybki i od lat stanowi standard w robotyce mobilnej, ale opiera się na linearyzacji. UKF lepiej radzi sobie z nieliniowością, bo używa punktów sigma zamiast Jacobianów, lecz jest droższy obliczeniowo. Filtr cząsteczkowy reprezentuje rozkład przez wiele hipotez stanu i świetnie nadaje się do lokalizacji globalnej, jednak wymaga znacznie większych zasobów. Wybór filtru zawsze jest kompromisem między kosztem a jakością modelowania.

Filtr Kalmana: cykl predykcja-korekcja

1

Predykcja

Model ruchu propaguje stan $\hat{x}$ i kowariancję P do przodu w czasie.

$$\hat{x}^- = F \cdot \hat{x} + B \cdot u$$

2

Pomiar

Sensor dostarcza nową obserwację z – zawierającą szum R .
Innowacja: $v = z - H \cdot \hat{x}^-$

3

Wzmocnienie K

Wagi Kalmana wyznaczają optymalny kompromis między modelem, a pomiarem.
 $K = P^- \cdot H^T \cdot (H \cdot P^- \cdot H^T + R)^{-1}$

4

Korekcja

Stan i kowariancja zaktualizowane o innowację.

$$\hat{x} = \hat{x}^- + K \cdot v$$
$$P = (I - K \cdot H) \cdot P^-$$

Filtr Kalmana pracuje w cyklu predykcja–korekcja. Najpierw model ruchu propaguje stan i kowariancję w przyszłość. Następnie przychodzi nowy pomiar obarczony szumem. Z różnicy między pomiarem a przewidywaniem powstaje innowacja. Potem liczymy wzmocnienie Kalmana, które określa, jak bardzo zaufać modelowi, a jak bardzo sensorowi. Na końcu aktualizujemy stan i kowariancję. To jest centralny mechanizm nowoczesnej fuzji sensorycznej.

Uproszczony EKF dla pozycji 2D

```
import numpy as np

class EKF2D:
    def __init__(self):
        self.x = np.zeros(3)           # [x, y, yaw]
        self.P = np.eye(3) * 0.5       # kowariancja początkowa
        self.Q = np.diag([0.01, 0.01, 0.005]) # szum procesu
        self.R = np.diag([0.1, 0.1])   # szum pomiaru (x, y)

    def predict(self, v: float, omega: float, dt: float):
        yaw = self.x[2]
        self.x += np.array([v*np.cos(yaw)*dt, v*np.sin(yaw)*dt, omega*dt])
        F = np.eye(3); F[0,2]=-v*np.sin(yaw)*dt; F[1,2]=v*np.cos(yaw)*dt
        self.P = F @ self.P @ F.T + self.Q

    def correct(self, z_xy: np.ndarray):
        H = np.array([[1,0,0],[0,1,0]])
        S = H @ self.P @ H.T + self.R
        K = self.P @ H.T @ np.linalg.inv(S)
        self.x += K @ (z_xy - self.x[:2])
        self.P = (np.eye(3) - K @ H) @ self.P
```

Opis: predict() używa modelu jednośladowego (unicycle); correct() koryguje pomiar GPS lub znacznika

Fuzja IMU, enkoderów i wizji

- **IMU (200 Hz):** predykcja stanu – szybka propagacja orientacji i przyspieszenia
- **Enkodery + kontakty stóp (100–500 Hz):** kinematyczna odometria kroków – korekta translacji
- **Kamera (30 Hz) / LiDAR (10 Hz):** obserwacje pozycyjne – usuwanie skumulowanego dryfu
- **Architektura loosely coupled:** każde źródło dostarcza niezależnej estymatę pozycji do EKF
- **Architektura tightly coupled:** surowe pomiary sensorów trafiają bezpośrednio do jednego filtra
- **Opóźnienia:** pomiar z kamery z 33 ms opóźnieniem wymaga cofnięcia stanu (state rewinding)
- **Stan złożony:** $x = [\text{pos}, \text{vel}, \text{orient}, \text{biasy IMU}]$ – typowo 15–21 zmiennych stanu

W pełnym systemie humanoida typowa architektura wygląda tak: IMU pracuje bardzo szybko i służy do predykcji stanu, enkodery wraz z kontaktami stóp korygują translację przez odometrię kroków, a kamera albo LiDAR dostarczają rzadszych, ale absolutnie kluczowych obserwacji pozycyjnych, które kasują skumulowany dryft. Można to spiąć architekturą loosely coupled, gdzie źródła dają osobne estymaty, albo tightly coupled, gdzie do jednego filtra trafiają surowe pomiary. Przy tym trzeba jeszcze uwzględnić opóźnienia i biasy IMU.

Wykrywanie awarii sensora

Test innowacji (chi-squared)

- Innowacja $v = z - H \cdot \hat{x}^-$ powinna być zbliżona do zera
- Statystyka χ^2 : $v^{-1} \cdot S^{-1} \cdot v > \text{próg} \rightarrow \text{sensor awaryjny}$
- Zaimplementowane w bibliotekach iSAM2, GTSAM
- Działa online, bez zatrzymywania robota

Tryb degradacji (graceful degradation)

- Awaria kamery $\rightarrow$ nawigacja tylko na IMU + enkodery
- Awaria LiDAR $\rightarrow$ zmniejszony zasięg planowania
- Awaria jednej stopy F/T $\rightarrow$ wyłącz kinematyczną odo dla tej nogi
- Zwiększenie macierzy Q i P $\rightarrow$ większy obszar niepewności

Dobry system lokalizacji musi również wykrywać awarie sensorów. Robi się to między innymi przez analizę innowacji. Jeśli różnica między pomiarem a przewidywaniem staje się statystycznie zbyt duża, uznajemy sensor za niewiarygodny. Wtedy system przechodzi w tryb degradacji, na przykład jedzie bez kamery albo wyłącza odometrię jednej stopy. Jednocześnie rośnie niepewność estymacji, co powinno znaleźć odbicie w macierzach Q i P.

03

SEKCJA

Lokalizacja globalna i SLAM

ICP, visual odometry, pose graph, domykanie pętli



Lokalizacja względem znanych map

1. **Cel:** wyznaczenie pozycji robota w już zbudowanej mapie środowiska
2. **Monte Carlo Localization (*MCL / AMCL*):** filtr cząsteczkowy z obserwacjami LiDAR
3. **Matching 2D:** porównanie aktualnego skanu z mapą zajętości – scoring hipotez
4. **Matching 3D (*ICP*):** dopasowanie chmury punktów do globalnej mapy PCL
5. **Histogram Filter (*HMM*):** dyskretne komórki mapy jako hipotezy lokalizacji
6. **Warunek dobrej lokalizacji:** mapa aktualna i środowisko o charakterystycznych elementach
7. **Kidnapped robot problem:** wymagana inicjalizacja globalna, gdy pozycja startowa jest nieznana

Lokalizacja względem znanej mapy polega na znalezieniu pozycji robota w już istniejącym modelu środowiska. Można to robić filtrem cząsteczkowym AMCL z obserwacjami LiDAR, dopasowaniem skanu 2D do mapy zajętości, ICP dla chmur 3D albo filtrem histogramowym. Warunkiem sukcesu jest aktualna mapa i środowisko z wystarczająco charakterystycznymi elementami. Jeśli robot zostanie przeniesiony w nieznane miejsce, pojawia się problem kidnapped robot i wtedy potrzebna jest inicjalizacja globalna.

ICP – Iterative Closest Point

- **Cel:** znalezienie transformacji (R, t) minimalizującej odległości między dwoma chmurami punktów
- **Krok 1 – Korespondencja:** dla każdego punktu źródłowego znajdź najbliższy punkt w chmurze docelowej
- **Krok 2 – Estymacja:** oblicz optymalną transformację (metoda SVD lub minimalizacja MLS)
- **Krok 3 – Aktualizacja:** zastosuj transformację i oblicz błąd residualny
- **Iteruj do zbieżności** (błąd $<$ próg) lub **osiągnięcia max. iteracji**
- **Wady:** wolna zbieżność, wrażliwość na punkt startowy (koniecznie dobre wstępne wyrównanie)
- **Warianty:** Point-to-Plane ICP, GICP – znacznie szybsze i bardziej niezawodne

ICP, czyli Iterative Closest Point, służy do znajdowania transformacji między dwiema chmurami punktów. Najpierw wyznaczamy korespondencje, potem obliczamy transformację minimalizującą błąd, aktualizujemy położenie i iterujemy do zbieżności. Metoda jest bardzo ważna, ale ma też swoje ograniczenia: bywa wolna i jest czuła na punkt startowy. Dlatego w praktyce stosuje się też warianty takie jak point-to-plane ICP albo GICP.

Odometria wizyjna (Visual Odometry)

Monokularna VO

- Śledzenie cech ORB/SIFT między klatkami
- Essential Matrix → ruch kamery R, t
- Skala ruchu nieznana (do 5-punktowego)
- Skumulowany dryft bez domknięcia pętli
- Niska cena – jedna kamera wystarczy

Stereofoniczna / RGB-D VO

- Bezpośrednia metryczna skala ruchu
- Głębia z paralaksy lub strukturyzowanego światła
- Mniejszy dryft niż mono VO
- Wymagana synchronizacja i kalibracja stereo
- Podstawa Visual-Inertial Odometry (VIO)

Odometria wizyjna może być monokularna albo stereofoniczna, ewentualnie RGB-D. W wersji monokularnej śledzimy cechy między klatkami i odzyskujemy ruch kamery, ale tracimy metryczną skalę. W stereo albo RGB-D skala jest bezpośrednio dostępna, więc dryft jest zwykle mniejszy. To właśnie dlatego systemy visual-inertial odometry tak często opierają się na stereo lub głębi, szczególnie wtedy, gdy zależy nam na stabilnej estymacji metrycznej.

SLAM – Simultaneous Localization and Mapping

- **Problem sprzężony:** mapa zależy od dobrej lokalizacji; lokalizacja zależy od dobrej mapy
- **Online SLAM:** estymacja bieżącej pozycji i mapy w czasie rzeczywistym
- **Full SLAM:** pełna trajektoria i mapa (post-processing w np. Google Cartographer)
- **Dane obserwacyjne:** LiDAR (2D/3D SLAM), kamera (Visual SLAM – ORB-SLAM3, RTAB-Map)
- **Reprezentacja mapy:** chmura punktów, mapa cech (deskryptory), submapa siatek
- **Współdziałanie z planistą:** gotowa mapa przekazywana do globalnego planisty ścieżki
- **Błąd rośnie liniowo z odległością** bez domknięcia pętli; po 100 m może sięgnąć kilku metrów

SLAM to jednoczesna lokalizacja i mapowanie. Jest to problem sprzężony, bo dobra mapa wymaga dobrej lokalizacji, a dobra lokalizacja wymaga dobrej mapy. W wersji online system estymuje bieżącą pozycję i buduje mapę w czasie rzeczywistym. W wersji full SLAM optymalizujemy całą trajektorię i mapę również po zakończeniu przejazdu. Możemy robić to na danych z LiDAR-u albo kamery, a wynik trafia potem do planisty globalnego. Bez domknięcia pętli błąd rośnie wraz z odległością i może stać się bardzo duży.

Pose Graph SLAM, czyli idea grafowa SLAM-u

1

Węzły (*poses*)

Każda pozycja robota $x_1, x_2, \dots, x_n$ to wierzchołek w grafie; zazwyczaj SE(2) lub SE(3)

2

Krawędzie (*constraints*)

Relacja między dwoma pozycjami: odometria, ICP, wizja. Niesie kowariancję błędu.

3

Optymalizacja

Minimalizacja sumy błędów kwadratowych (least squares): g2o, GTSAM, iSAM2

4

Domknięcie pętli

Detekcja powrotu do miejsca; dodanie krawędzi pogrubionej; jednorazowa korekta dryfu

Pose Graph SLAM ujmuje problem w formie grafu. Węzły reprezentują kolejne pozycje robota, a krawędzie relacje między nimi: odometrię, dopasowanie ICP albo ograniczenia z wizji. Każda krawędź niesie też informację o niepewności. Optymalizacja grafu polega na minimalizacji sumy błędów wszystkich ograniczeń. Gdy wykryjemy powrót do wcześniej odwiedzonego miejsca, dodajemy nową krawędź i globalnie korygujemy cały przebieg trajektorii.

Domykanie pętli (*loop closure*)

Detekcja powrotu do miejsca

- **Bag-of-Words** (DBoW2): histogram deskryptorów wizualnych
- **Scan matching**: ICP między bieżącym, a historycznym skanem
- **Mapa cech**: dopasowanie deskryptorów ORB/SIFT
- **Próg podobieństwa**: zapobiega fałszywym detekcjom
- **Weryfikacja geometryczna**: RANSAC na korespondencjach

Korekta dryfu po domknięciu

- **Nowa krawędź** w pose graph z małą kowariancją
- **Optymalizator** (g2o) przelicza cały graf
- **Błąd dryfu** rozłożony wzdłuż zamkniętej pętli
- **Efekt**: mapa globalna spójna bez artefaktów
- **Częstość**: co kilka sekund dla środowisk cyklicznych

Domykanie pętli składa się z dwóch etapów. Najpierw trzeba wykryć, że wróciliśmy do znanego miejsca, na przykład przez bag of words, scan matching albo dopasowanie deskryptorów cech. Potem trzeba to jeszcze zweryfikować geometrycznie, często przez RANSAC. Jeśli wynik jest wiarygodny, dodajemy mocne ograniczenie do grafu i uruchamiamy optymalizację. Efekt jest taki, że skumulowany dryft rozkłada się po całej pętli, a mapa staje się globalnie spójna.

Prosta korekta pozycji po detekcji znacznika

```
import numpy as np

def correct_with_landmark(state: np.ndarray,
                          P: np.ndarray,
                          landmark_pos: np.ndarray,
                          measured_pos: np.ndarray,
                          R_meas: np.ndarray) -> tuple:
    """Korekta EKF po pomiarze pozycji znajomego znacznika."""
    H = np.eye(2, 3) # obserwujemy tylko x, y
    predicted_z = landmark_pos[:2]
    innovation = measured_pos - predicted_z
    S = H @ P @ H.T + R_meas
    K = P @ H.T @ np.linalg.inv(S) # wzmocnienie Kalmana
    state_new = state + K @ innovation
    P_new = (np.eye(3) - K @ H) @ P
    return state_new, P_new
```

Opis: Korekta EKF: innowacja z detekcji ArUco/AprilTag eliminuje skumulowany dryft odometrii

Dobór metody lokalizacji do zadania

Laboratorium

Znana, statyczna mapa; AMCL z LiDAR-em 2D wystarczy; opcjonalnie znaczniki ArUco

Biuro

Zmienna mapa (meble); SLAM online (Cartographer) + MCL; UWB jako wsparcie

Magazyn

Pojazdy AGV i regały; Visual SLAM z kamerami panoramicznymi; QR na półkach

Schody

3D LiDAR SLAM (LeGO-LOAM) lub RGB-D SLAM (RTAB-Map); height map aktywna

Dynamiczne otoczenie

EKF + filtr cząsteczkowy; odrzucanie ruchomych obiektów ze skanów mapowania

Zewnętrzne warunki

GPS + IMU (EKF) jako baza; LiDAR do refinement; odporność na warunki pogodowe

Dobór metody lokalizacji zależy od zadania. W laboratorium zwykle wystarcza AMCL z LiDAR-em 2D. W biurze, gdzie środowisko się zmienia, lepiej sprawdza się SLAM online z dodatkowym wsparciem. W magazynie przydaje się visual SLAM i znaczniki na regałach. Na schodach potrzebujemy już 3D LiDAR SLAM albo RGB-D SLAM wraz z aktywną height mapą. W dynamicznym otoczeniu trzeba odrzucać ruchome obiekty, a na zewnątrz połączyć GPS z IMU i LiDAR-em. Nie istnieje jedna uniwersalna metoda dobra do wszystkiego.

PODSUMOWANIE

- 01 **Dryft odometrii jest nieunikniony:** konieczna jest korekta z zewnętrznych sensorów lub SLAM
- 02 **Filtr Kalmana (EKF/UKF)** to złoty standard fuzji IMU, enkoderów i danych wizyjnych
- 03 **Filtr cząsteczkowy** radzi sobie z wielomodalnością, lecz wymaga większych zasobów obliczeniowych
- 04 **SLAM** (jednoczesne mapowanie i lokalizacja): pozwala robotowi budować i korygować mapę w locie
- 05 **Domykanie pętli** eliminuje skumulowany dryft; awarię sensora wykrywa się przez test innowacji

Pytania i mini-studium przypadku

1. Dlaczego filtr komplementarny nie może zastąpić EKF w pełnym systemie lokalizacji?
2. Jak wzmacnienie Kalmana K zmienia się, gdy macierz szumu pomiaru R jest bardzo duża?
3. Co to jest NIS (Normalized Innovation Squared) i jak wykrywa awarię sensora?
4. Opisz, jakie sensory i jakie filtry wybrać, dla humanoida patrolującego budynek przez 30 min, aby błąd pozycji nie przekroczył 0,3m. Jakie jest minimalne wymaganie na częstotliwość domknięcia pętli?

Zadanie

Zaimplementuj StepOdometry z Sekcji 1 i porównaj dryft z i bez korekty z EKF przy symulowanym szumie $yaw = 0,02$ rad/krok.

NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 3

Planowanie ruchu i unikanie przeszkód

dr inż. Mateusz Pomianek



Planowanie globalne

Planowanie lokalne

Footstep planning

Kody Python

Plan wykładu

Sekcja 1

Planowanie globalne: przestrzeń stanów, Dijkstra, A*, heurystyki, cost map, wygładzanie

Sekcja 2

Planowanie lokalne: DWA, potential fields, MPC, przeszkody dynamiczne, replanowanie

Sekcja 3

Footstep planning: sekwencja kroków, ograniczenia kinematyczne, schody, trajektoria CoM

Kody Python

A* na grid map, A* z cost map, pole potencjału, ocena trajektorii, generacja kroków

01

SEKCJA

Planowanie globalne

Reprezentacje środowiska, Dijkstra, A, heurystyki i cost map*



Miejsce planowania w architekturze systemu

Wejście: mapa + lokalizacja

Planista dostaje mapę kosztów i bieżącą pozycję robota z estymatora stanu

Wejście: cel misji

Punkt docelowy lub sekwencja celów pośrednich z modułu zarządzania misją

Wyjście: ścieżka globalna

Ciąg punktów w przestrzeni (x,y) lub (x,y,z) przekazywany do planisty lokalnego

Wyjście: cost map

Mapa kosztów przesyłana do lokalnego planisty jako mapa naviagacyjna

Problem planowania ścieżki

- **Przestrzeń stanów:** zbiór wszystkich dopuszczalnych konfiguracji robota $C_{\text{free}} \subseteq C$
 - **Stan początkowy** q_{start} : aktualna pozycja i orientacja robota w mapie
 - **Stan docelowy** q_{goal} : żądana pozycja końcowa misji
- **Ograniczenia:** przeszkody, granice mapy, kinematyka, stabilność robota
- **Funkcja kosztu:** $c(q_{\text{start}}, q_{\text{goal}})$ minimalizowana wzdłuż ścieżki (czas, energia, ryzyko)
- **Ścieżka optymalna:** ciąg $\gamma : [0,1] \rightarrow C_{\text{free}}$ taki, że $\gamma(0)=q_{\text{start}}, \gamma(1)=q_{\text{goal}}$
- **Zupełność:** algorytm znajdzie ścieżkę, jeśli istnieje; optymalność: znajdzie najlepszą

Reprezentacje grafowe środowiska

Siatka komórek (Grid)

- Regularna siatka $N \times M$ komórek
- Komórka: wolna / zajęta / koszt
- Łatwa implementacja A^*
- Duże mapy = wysoka pamięć
- Rozdzielczość ogranicza precyzję

Graf topologiczny

- Węzły = charakterystyczne miejsca
- Krawędzie = przejazdy między nimi
- Szybkie planowanie wysokopoziomowe
- Niska rozdzielczość geometryczna
- Wymaga mapowania semantycznego

Visibility/Roadmap

- Graf widzialności krawędzi przeszkód
- Trasy wzdłuż wklętości przeszkód
- Optymalne w 2D dla znanych map
- Trudne do uogólnienia na 3D
- Rzadko stosowane dla humanoida

Algorytm Dijkstry

Pseudokod Dijkstry

```
1. dist[start] = 0; dist[v] = ∞ dla pozostałych
2. priority_queue Q: (dist, vertex)
3. Q.push((0, start))
4. while Q nie jest pusta:
5.     (d, u) = Q.pop_min()
6.     if d > dist[u]: continue
7.     for (v, w) in sąsiedzi(u):
8.         nowy_koszt = dist[u] + w
9.         if nowy_koszt < dist[v]:
10.            dist[v] = nowy_koszt
11.            poprzednik[v] = u
12.            Q.push((nowy_koszt, v))
13. odtwórz ścieżkę przez poprzednik[]
```

Właściwości

- Gwarantuje optymalność
- Brak heurystyki, eksploruje we wszystkich kierunkach
- Złożoność: $O((V+E) \log V)$
- Wolny dla dużych map
- Dobry jako punkt odniesienia
- Podstawa A* i Bellmana-Forda

Algorytm A* - planowanie z heurystyką

Pseudokod A*

$f(n) = g(n) + h(n)$

$g(n)$ - koszt dotarcia od startu do n

$h(n)$ - heurystyka: szacunek do celu

```
1. open_set = {start}
2. g[start] = 0; f[start] = h(start)
3. while open_set nie jest pusta:
4.     u = węzeł z min f w open_set
5.     if u == goal: odtwórz ścieżkę
6.     open_set.remove(u)
7.     for v in sąsiedzi(u):
8.         tentative_g = g[u] + cost(u,v)
9.         if tentative_g < g[v]:
10.            g[v] = tentative_g
11.            f[v] = g[v] + h(v)
12.            parent[v] = u
```

Właściwości

- Optymalny gdy h admissible
- Szybszy od Dijkstry dzięki h
- h euklidesowa: $\sqrt{dx^2+dy^2}$
- h Manhattan: $|dx|+|dy|$
- Im lepsza h , tym mniej węzłów
- Złożoność: $O(b^d)$, b =branching

A* na mapie zajętości (grid map)

```
import heapq, numpy as np

def astar(grid, start, goal):
    """A* na siatce 2D. grid[r][c]=0: wolne, 1: zajete."""
    rows, cols = grid.shape
    h = lambda r, c: abs(r-goal[0]) + abs(c-goal[1])
    g = {start: 0}
    parent = {start: None}
    pq = [(h(*start), start)]
    while pq:
        _, u = heapq.heappop(pq)
        if u == goal:
            path = []
            while u:
                path.append(u)
                u = parent[u]
            return path[::-1]
        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            v = (u[0]+dr, u[1]+dc)
            if 0<=v[0]<rows and 0<=v[1]<cols and grid[v]==0:
                ng = g[u] + 1
                if ng < g.get(v, 1e9):
                    g[v]=ng; parent[v]=u
                    heapq.heappush(pq, (ng+h(*v), v))
```

Opis: Manhattan h – dopuszczalna i spójna; 4-kierunkowa sąsiedztwo; zwraca listę komórek ścieżki

Właściwości i zastosowanie Heurystyki w A*

Heurystyka Manhattan (L1)

- $h(n) = |dx| + |dy|$
- Idealna dla siatek z ruchem 4-kierunkowym
- Dopuszczalna: nigdy nie przeszacowuje kosztu
- **Bardzo szybka:** prosta operacja arytmetyczna
- **Dla ruchu 8-kierunkowego:** lekko nieprecyzyjna

Heurystyka euklidesowa (L2)

- $h(n) = \sqrt{dx^2 + dy^2}$
- Odpowiednia dla ciągłego ruchu w 2D/3D
- Dopuszczalna dla humanoida (metryczny koszt)
- **Droższa obliczeniowo** (sqrt), lecz precyzyjniejsza
- Zalecana gdy ścieżka jest wygładzana

Planowanie z uwzględnieniem kosztów terenu

- **Cost map** zastępuje binarną mapę wolne/zajęte gradientem kosztów
- **Cel**: preferowanie bezpiecznych, szerokich przejść zamiast najkrótszej trasy geometrycznej
- **Warstwy kosztu**: inflacja wokół przeszkód, nachylenie terenu, wskaźnik stabilności, gęstość ruchu
- **Implementacja**: komórki siatki mają wartość $c \in [0, 255]$; A* używa $\text{cost}(u,v) = 1 + c(v)/50$
- **Efekt**: trasa omija obrzeża mebli i wąskie przejścia nawet gdy są geometrycznie wolne
- **Dynamiczna aktualizacja**: nowe obserwacje LiDAR/kamery zmieniają cost map online
- **Parametryzacja**: wagi warstw dobierane eksperymentalnie lub przez uczenie maszynowe

A* z mapą kosztów terenu

```
import heapq, numpy as np

def astar_cost(grid, cost_map, start, goal):
    """A* z kosztem komórki z cost_map [0-100]."""
    rows, cols = grid.shape
    h = lambda r, c: np.hypot(r-goal[0], c-goal[1]) # euklidesowa
    g = {start: 0.0}
    parent = {start: None}
    pq = [(h(*start), start)]
    while pq:
        _, u = heapq.heappop(pq)
        if u == goal:
            path, node = [], u
            while node: path.append(node); node = parent[node]
            return path[::-1]
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(1,1),(-1,1),(1,-1)]:
            v = (u[0]+dr, u[1]+dc)
            if 0<=v[0]<rows and 0<=v[1]<cols and grid[v]==0:
                step = np.hypot(dr,dc) * (1 + cost_map[v]/50.0)
                if (ng := g[u]+step) < g.get(v,1e9):
                    g[v]=ng; parent[v]=u
                    heapq.heappush(pq, (ng+h(*v),v))
```

Opis: 8-kierunkowe sąsiedztwo; koszt przejścia = odległość * (1 + wartość_kosztu/50); ruch po skosie droższy

Wygładzanie ścieżki globalnej

- A^* na siatce zwraca ścieżkę z ostrymi załamaniami przy każdej zmianie kierunku
- **Wygładzanie:** konieczne przed przekazaniem ścieżki do generatora kroków lub MPC
- **Metoda 1, interpolacja B-spline:** krzywa przechodzi blisko punktów kontrolnych, zachowuje ciągłość krzywizny
- **Metoda 2, skracanie (shortcutting):** iteracyjne usuwanie pośrednich punktów, jeśli prosta jest wolna od przeszkód
- **Metoda 3, filtr Gaussa 1D:** ważona średnia po sąsiednich punktach wzdłuż ścieżki
- **Metryka:** kąt ugięcia $< 15^\circ$ między kolejnymi odcinkami jako warunek akceptacji
- **Ważne:** wygładzona ścieżka musi pozostawać w wolnej przestrzeni – weryfikacja kolizji po wygładzeniu

02

SEKCJA

Planowanie lokalne i unikanie przeszkód

DWA, potential fields, MPC, przeszkody dynamiczne



Po co potrzebny planista lokalny?

Świat dynamiczny

Mapa globalna nie uwzględnia ludzi, wózków, otwieranych drzwi. Planista lokalny reaguje na bieżąco

Niedoskonała mapa

Sensory mają szum, mapa jest zbudowana ze starego skanu – lokalna mapa jest zawsze świeższa

Ograniczenia dynamiki

Robot nie może skręcić dowolnie ostro ani zatrzymać się natychmiast – kinematyka musi być uwzględniona

Horyzont predykcji

Decyzje na 1–3 s do przodu wystarczają; pełny replan jest zbyt kosztowny w każdym cyklu (10–50 Hz)

Pola potencjału (Potential Fields)

Wzory sił

$U_{\text{attr}}(q) = \frac{1}{2} \cdot k_{\text{att}} \cdot d^2(q, q_{\text{goal}})$
- atrakcja do celu; rośnie z odległością

$U_{\text{rep}}(q) = \frac{1}{2} \cdot k_{\text{rep}} \cdot (1/d - 1/d_0)^2$
gdy $d < d_0$, inaczej 0 - odpychanie od przeszkód
 d_0 - promień wpływu przeszkody

$F(q) = -\nabla U(q)$
 $= F_{\text{attr}} + F_{\text{rep}}$

Ruch: $q_{\text{new}} = q + \alpha \cdot F(q)$
 α - krok iteracji

Właściwości

- Prosta implementacja
- Działa w czasie rzeczywistym
- Problem: minima lokalne
- Oscylacje w wąskich przejściach
- Trudna parametryzacja k_{att} , k_{rep}
- Rozwiązanie: potencjały harmoniczne lub RRT

Idea Dynamic Window Approach (DWA)

- **Idea:** w każdym kroku czasowym szukaj najlepszej pary prędkości (v, ω) z okna osiągalnego
- **Okno dynamiczne:** $\{(v, \omega) \mid v \in [v_{-a_max} \cdot dt, v_{+a_max} \cdot dt], \omega \text{ analogicznie}\}$
- **Ocena trajektorii:** dla każdego kandydata (v, ω) symuluj ruch przez T sekund (np. $T=2$ s)
- **Funkcja celu:** $G(v, \omega) = \alpha \cdot \text{heading} + \beta \cdot \text{clearance} + \gamma \cdot \text{velocity}$
- **heading:** wyrównanie kierunku ruchu z celem globalnym
- **clearance:** odległość od najbliższej przeszkody wzdłuż trajektorii
- **velocity:** nagroda za wyższą prędkość do przodu
- **wybierz $(v^*, \omega^*) = \operatorname{argmax} G$** – wykonaj jeden krok, przelicz w kolejnym cyklu

Model Predictive Control, czyli taka „intuicja”

- **MPC** optymalizuje sekwencję sterowań $u_0, u_1, \dots, u_{N-1}$ na horyzoncie N kroków
- **Model predykcji**: $x_{t+1} = f(x_t, u_t)$ – uproszczony model kinematyczny/dynamiczny robota
- **Funkcja celu**: minimalizacja odchylenia od ścieżki + kosztu sterowania + penalizacja za przeszkody
- **Ograniczenia**: $|v| \leq v_{\max}$, $|\omega| \leq \omega_{\max}$, brak kolizji, stabilność ZMP
- **Receding horizon**: po wyznaczeniu sekwencji wykonuje się tylko u_0 , a plan jest aktualizowany
- **Zalety**: jawne ograniczenia dynamiczne, optymalne zarządzanie kolizjami w horyzoncie
- **Wada**: wysoki koszt obliczeniowy (solver QP/NLP); wymaga dedykowanego CPU lub GPU

Pole potencjału - lokalny wektor ruchu

```
import numpy as np

def potential_field_move(robot_pos, goal_pos, obstacles,
                        k_att=1.0, k_rep=100.0, d0=1.5):
    """Wyznacza wektor ruchu z pol potencjalnych."""
    robot = np.array(robot_pos, dtype=float)
    goal = np.array(goal_pos, dtype=float)
    # Siła atrakcji do celu
    f_att = k_att * (goal - robot)
    # Suma sil odpychania od przeszkód
    f_rep = np.zeros(2)
    for obs in obstacles:
        diff = robot - np.array(obs)
        d = np.linalg.norm(diff) + 1e-6
        if d < d0:
            f_rep += k_rep * (1/d - 1/d0) * diff / d**3
    total = f_att + f_rep
    return total / (np.linalg.norm(total) + 1e-6) # wektor jednostkowy
```

Opis: Zwraca znormalizowany wektor ruchu; mnożymy przez Δt i prędkość robota, by uzyskać przemieszczenie

Ocena kandydatów trajektorii (uproszczone DWA)

```
import numpy as np

def score_trajectory(v, omega, robot_state, goal, obstacles,
                    T=2.0, dt=0.1,
                    alpha=0.4, beta=0.4, gamma=0.2):
    """Ocenia trajektorie (v, omega) w horyzoncie T sekund."""
    x, y, yaw = robot_state
    min_clearance = 1e9
    for _ in range(int(T / dt)):
        x += v * np.cos(yaw) * dt
        y += v * np.sin(yaw) * dt
        yaw += omega * dt
        for obs in obstacles:
            min_clearance = min(min_clearance, np.hypot(x-obs[0], y-obs[1]))
    heading = -abs(np.arctan2(goal[1]-y, goal[0]-x) - yaw)
    clearance = min_clearance
    return alpha*heading + beta*clearance + gamma*v
```

Opis: Wyższy score = lepsza trajektoria; wywoływane dla N par (v, ω) , wybierana para z max score

Unikanie ludzi i przeszkód dynamicznych

Detekcja i predykcja

- **Detekcja ludzi:** YOLO, PointNet na chmurze LiDAR
- **Śledzenie** (tracking): filtr Kalmana per obiekt
- **Predykcja:** stałe velocity, Social Force Model
- **Horyzont predykcji:** 1–3 s wystarczy
- **Niepewność:** elipsa kowariancji wokół ścieżki

Planowanie z uwzględnieniem ruchu

- **Strefa bezpieczeństwa:** okrąg lub elipsa wokół człowieka
- **Prędkość** przeszkody skaluje rozmiar strefy
- **ORCA** (Optimal Reciprocal Collision Avoidance)
- **Social navigation:** min. 1,2 m od człowieka
- Zatrzymanie gdy człowiek zbliży się $< 0,5$ m

Kiedy i jak przeplanowywać online

Replan globalny (rzadki)

- Gdy cel misji zostaje zmieniony
- Gdy mapa zasadniczo się zmienia (*zamknięte przejście*)
- Co 5–30 sekund lub na żądanie operatora
- Kosztowny obliczeniowo – całkowite A*
- Alternatywa: D* Lite (*przyrostowy A**)

Replan lokalny (częsty)

- Co 50–100 ms w każdym cyklu planisty
- Okno lokalne wokół bieżącej pozycji robota
- Reaguje na nowe przeszkody dynamiczne
- Możliwy na CPU embarked 1–2 ms
- Brak zmiany ścieżki globalnej

Konflikt między planem, a stabilnością

- **Problem:** optymalny manewr unikania może wymagać szybkiego skrętu lub dużej prędkości bocznej
- **Konsekwencja:** środek masy wychodzi poza wielobok podparcia → utrata równowagi
- **Reguła nadrzędna:** stabilność > unikanie kolizji > optymalizacja trasy
- **Implementacja:** każda kandydatka trajektoria lokalnej jest filtrowana przez weryfikator ZMP
- **Weryfikator ZMP:** odrzuca trajektorie, gdzie przewidywany ZMP wychodzi poza margines bezpieczeństwa
- **Efekt:** planista lokalny preferuje wolniejsze, bardziej konserwatywne manewry w trudnym terenie
- **Parametr:** `safety_margin_zmp` = 2–4 cm; dla schodów i pochyłości zwiększony do 5–8 cm

03

SEKCJA

Planowanie kroków i trajektorii humanoida

Footstep planning, ograniczenia, schody, trajektoria CoM



Od ścieżki geometrycznej do sekwencji kroków

1

Ścieżka bazy

Globalna trasa (x,y) lub (x,y,z) wyznaczona przez A^* ; reprezentuje ruch CoM lub miednicy

2

Wzorzec chodu

Parametry: długość kroku l , szerokość w , czas fazy T_{step} , T_{swing} ; regularne lub adaptacyjne

3

Wyznaczenie stóp

Dla każdego punktu ścieżki wyznacz cele stopy: offset lewo/prawo $\pm w/2$, orientacja = yaw trasy

4

Weryfikacja

Sprawdź długość kroku, osiągalność kinematyczną, kontakt z terenem, margines ZMP i stabilność

Parametry i ograniczenia planowania kroków

- **Długość kroku:** typowo 0,2–0,5 m; zależy od prędkości i stabilności (przy niskiej prędkości krócej)
- **Szerokość kroku:** 0,1–0,25 m; za mała = utrata stabilności, za duża = kolizja nóg
- **Orientacja stopy:** dostosowana do krzywizny ścieżki i pochylenia terenu (normal surface)
- **Strefy dozwolonego kontaktu** (foothold regions): płaskie, niegrzybowe obszary $\geq$ stopa robota
- **Kolejność kroków:** lewa–prawa naprzemiennie; podwójne podparcie $\geq$ 20% cyklu
- **Wykluczenie:** krawędzie, śliskie powierzchnie, zbyt strome pochylenie ($>20^\circ$), wąskie progi
- **Planowanie w 3D:** wymagane gdy teren ma zmienną wysokość (schody, kamienie, nierówności)

Ograniczenia kinematyczne i stabilnościowe

Ograniczenia kinematyczne

- **Zasięg nogi:** max odległość stopa–biodro $\leq l_{\max}$
- **Min. odstęp:** stopy nie mogą nachodzić na siebie
- **Zakres stawów:** każdy staw w przedziale $[q_{\min}, q_{\max}]$
- **Prędkość stawów:** $\omega_{\text{joint}} \leq \omega_{\max}$ (przepustowość aktuatora)
- **Konfiguracja singularna:** unikanie pełnego wyprostu kolana

Ograniczenia stabilnościowe

- **ZMP** wewnątrz wieloboku podparcia z marginesem
- **Capture Point** w dopuszczalnej strefie landing
- **Odchylenie tułowia** $\leq \pm 15^\circ$ od pionu
- **Prędkość CoM** ograniczona przez marginesy LIPM
- **Przy schodach:** dodatkowy margines boczny +3 cm

Planowanie na schodach i nierównościach

- **Detekcja schodów:** segmentacja chmury punktów; klasyfikacja płaskich poziomych płaszczyzn
- **Wysokość stopnia:** robot musi unieść stopę o $(h_{\text{step}} + \text{clearance})$ ponad krawędź
- **Miejsca kontaktu:** wyznaczane na środku głębokości stopnia z marginesem ≥ 2 cm od krawędzi
- **Kolejność działań:** wejście przodem – lewa noga wyżej $\rightarrow$ prawa noga dołącza, powtórz
- **Trajektoria stopy w powietrzu:** krzywa beziera nad stopniem; max wysokość = $h_{\text{step}} + 5$ cm
- **Kontrola ciała:** tułów pochyla się nieznacznie do przodu przy wchodzeniu, do tyłu przy schodzeniu
- **Zabezpieczenie:** jeśli wysokość stopnia > 25 cm lub brak miejsca kontaktu – zatrzymaj misję

Generacja sekwencji kroków (footstep planner)

```
import numpy as np

def generate_footsteps(path_xy, step_length=0.35,
                      step_width=0.18) -> list:
    """Generuje naprzemienne kroki L/P wzdłuż ścieżki."""
    footsteps = []
    left_foot = True # zaczyna lewa noga
    dist_accum = 0.0
    for i in range(1, len(path_xy)):
        dx = path_xy[i][0] - path_xy[i-1][0]
        dy = path_xy[i][1] - path_xy[i-1][1]
        dist_accum += np.hypot(dx, dy)
        if dist_accum >= step_length:
            yaw = np.arctan2(dy, dx)
            sign = 1.0 if left_foot else -1.0
            foot_x = path_xy[i][0] - np.sin(yaw)*sign*step_width/2
            foot_y = path_xy[i][1] + np.cos(yaw)*sign*step_width/2
            footsteps.append({'x':foot_x,'y':foot_y,'yaw':yaw,'left':left_foot})
            left_foot = not left_foot
            dist_accum = 0.0
    return footsteps
```

Opis: Offset $\pm \text{step_width}/2$ prostopadłe do kierunku ruchu; 'left' flaga steruje naprzemiennością L/P

Weryfikacja wykonalności kroku

```
import numpy as np

def generate_footsteps(path_xy, step_length=0.35,
                      step_width=0.18) -> list:
    """Generuje naprzemienne kroki L/P wzdłuż ścieżki."""
    footsteps = []
    left_foot = True # zaczyna lewa noga
    dist_accum = 0.0
    for i in range(1, len(path_xy)):
        dx = path_xy[i][0] - path_xy[i-1][0]
        dy = path_xy[i][1] - path_xy[i-1][1]
        dist_accum += np.hypot(dx, dy)
        if dist_accum >= step_length:
            yaw = np.arctan2(dy, dx)
            sign = 1.0 if left_foot else -1.0
            foot_x = path_xy[i][0] - np.sin(yaw)*sign*step_width/2
            foot_y = path_xy[i][1] + np.cos(yaw)*sign*step_width/2
            footsteps.append({'x':foot_x,'y':foot_y,'yaw':yaw,'left':left_foot})
            left_foot = not left_foot
            dist_accum = 0.0
    return footsteps
```

Opis: Trzy sprawdzenia: długość wzdłużna, odchylenie boczne i skręt; zwraca dict z diagnostyką

Trajektoria środka masy, a sekwencja kroków

- **CoM** (Centre of Mass) musi śledzić zaplanowane kroki przez model LIPM (Linear Inverted Pendulum)
- **Przy podwójnym podparciu**: CoM przesuwa się od poprzedniej stopy do następnej
- **Przy pojedynczym podparciu** (faza swing): CoM podąża parabolą wyznaczoną przez ZMP
- **Sprzężenie**: sekwencja kroków $\rightarrow$ trajektoria ZMP $\rightarrow$ trajektoria CoM (generator QP)
- **Weryfikacja online**: $ZMP(t) \in \text{convex hull}(\text{stopa_oparta})$ przez cały cykl kroku
- **Narzędzie**: biblioteki Choreonoid, OpenHRP, ROS2 Whole Body Controller
- **Kompromis**: krótsze, częstsze kroki = lepsza stabilność CoM kosztem prędkości ruchu

Dobór planisty do konkretnej aplikacji

Laboratorium badawcze

A* + DWA; prosta cost map; footstep planer 2D;
iteracyjna weryfikacja ZMP; MuJoCo do testów

Robot usługowy (biuro)

A* z cost map + Social DWA; detekcja YOLO;
adaptacyjne kroki; zatrzymanie przy ludziach < 1 m

Ratownictwo (gruz)

3D A* lub RRT*; footstep w 3D z height map;
MPC lokalny; konserwatywne marginesy ZMP

Środowisko zatłoczone

ORCA lub S-MPC; krótki horyzont predykcji (0,5 s);
priority queue stref bezpiecznych; niskie prędkości

Schody i poziomy

Hierarchiczny planer: globalny graf pięter +
lokalny 3D footstep; detekcja stopni z point cloud

Sport/zawody

MPC z agresywnym horyzontem; dynamiczny
chód biegowy; uproszczony footstep dla
szybkości kosztem marginesu ZMP

PODSUMOWANIE

- 01 **Planowanie globalne** (A^* , Dijkstra) wyznacza optymalną trasę; heurystyki i cost map sterują wyborem ścieżki
- 02 **Planista lokalny** (DWA, MPC, potential fields) reaguje na zmiany środowiska i przeszkody dynamiczne
- 03 **Stabilność** ma wyższy priorytet niż optymalność trasy – planista musi uwzględniać margines równowagi
- 04 **Footstep planning** tłumaczy geometryczną ścieżkę na fizyczną sekwencję kroków z weryfikacją kinematyczną
- 05 **Trzy poziomy decyzji** – globalny, lokalny, footstep – są sprzężone i muszą działać w spójnej pętli

Zadanie projektowe i pytania problemowe

- Dlaczego heurystyka Manhattan nie jest dopuszczalna dla siatek z ruchem 8-kierunkowym?
- Co się dzieje z polem potencjału, gdy robot wpadnie w lokalne minimum? Zaproponuj wyjście.
- Jaką wartość parametru α i β wybrałbyś dla DWA w zatłoczonym korytarzu? Uzasadnij.

Zadanie projektowe: zaprojektuj planistę kroków dla humanoida poruszającego się w korytarzu 1,5 m szerokości z napotykanymi ludźmi. Opisz: jakie dane wejściowe są potrzebne, jak wyznaczasz miejsca kontaktu stóp, jak weryfikujesz każdy krok, jak reaguje planista gdy droga jest zablokowana przez >3 s.

Bonus: zaimplementuj wygładzanie B-spline dla ścieżki zwróconej przez A* i porównaj liczbę kroków koniecznych do pokonania tej samej trasy przed i po wygładzeniu.

NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 4

Chód, stabilność, kontrola
i integracja pełnego systemu

dr inż. Mateusz Pomianek



Stabilność i lokomocja

Kontrola wykonania

Architektura ROS 2

Bezpieczeństwo

Plan wykładu

Sekcja 1

Stabilność i lokomocja: ZMP, Capture Point, LIPM, chód statyczny i dynamiczny, zakłócenia

Sekcja 2

Kontrola wykonania ruchu: warstwy sterowania, WBC, PID, śledzenie trajektorii, symulacja

Sekcja 3

Architektura ROS 2, bezpieczeństwo, walidacja eksperymentalna, studium przypadku

Kody Python

ZMP, LIPM, PID, saturacja sygnału, pętla symulacji, komunikacja ROS 2

01

SEKCJA

Stabilność i lokomocja

*Chód, ZMP, Capture Point, LIPM i reakcja na
zakłócenia*



Pełny pipeline: od percepcji do postawienia kroku

1

Percepcja

IMU, LiDAR, RGB-D,
enkodery: fuzja
sensoryczna
i estymacja stanu

2

Planowanie

A^* + DWA: ścieżka
globalna i lokalna
przekazana do planisty
footstep

3

Generacja kroków

Sekwencja stóp,
trajektoria CoM
z modelu LIPM,
weryfikacja ZMP

4

Wykonanie

WBC + PID: komendy
momentów do
aktuatorów, odczyt
stanu i zamknięcie pętli

Chód statyczny, a chód dynamiczny

Chód statyczny

- CoM zawsze nad wielobokiem podparcia
- Możliwy przy bardzo niskich prędkościach
- Margines stabilności statycznej zawsze >0
- Prosta implementacja – brak dynamiki LIPM
- Zastosowanie: schody, nierówny teren, precyzja
- Wada: wolny, nienaturalny, energochłonny

Chód dynamiczny

- CoM może chwilowo wyjść za wielobok podparcia
- Wymagana kontrola ZMP w czasie rzeczywistym
- Naturalny, energooszczędny marsz człowieka
- Modele: LIPM, 3D-LIPM, Spring-Mass
- Zastosowanie: szybkie przemieszczanie, biuro
- Wada: wymaga pełnego WBC i szybkiego planisty

Środek masy i wielobok podparcia

- **Wielobok podparcia** (support polygon): wypukłe otoczek punktów kontaktu stóp z podłożem
- **Stabilność statyczna**: rzut CoM pionowo w dół musi leżeć wewnątrz wieloboku podparcia
- **Faza jednostronnego podparcia**: wielobok = tylko jeden prostokąt stopy ($\sim 20 \times 9$ cm)
- **Faza podwójnego podparcia**: wielobok = suma obu stóp + przestrzeń między nimi
- **Margines stabilności** (stability margin): minimalna odległość rzutu CoM od krawędzi wieloboku
- **Cel**: utrzymać margines $\geq d_{\text{safe}} = 2\text{--}3$ cm przez cały cykl kroku
- **Monitorowanie**: obliczane w każdym cyklu sterowania (1 kHz) na podstawie stanu stawów

Definicja i znaczenie Zero Moment Point

- **ZMP** to punkt na podłożu, w którym suma momentów sił kontaktu jest zerowa
- **Warunek stabilności dynamicznej:** ZMP musi leżeć wewnątrz wieloboku podparcia
- **Obliczanie ZMP z trajektorii CoM (LIPM):** $ZMP = x_CoM - (z_CoM/g) \cdot \ddot{x}_CoM$
- **Praktyczna interpretacja:** ZMP dąży do stopy podporowej podczas przenoszenia ciężaru
- **Margines ZMP:** typowo 2–4 cm od krawędzi; w trudnym terenie 5–8 cm
- **Planowanie oparte na ZMP:** wyznacz dozwolony ZMP → oblicz trajektorię CoM wstecznie
- **Biblioteki:** Choreonoid, OpenHRP3, DART – wbudowane generatory trajektorii ZMP/CoM

Capture Point i Linear Inverted Pendulum Model

Capture Point (CP)

- CP = punkt na podłożu, na który należy postawić stopę, by zatrzymać upadek
- $CP = x_{CoM} + (1/\omega_0) \cdot \dot{x}_{CoM}$
- $\omega_0 = \sqrt{g / z_{CoM}}$ – częstość LIPM
- Jeśli CP poza zasięgiem kroku → robot upadnie
- Zastosowanie: reakcja na pchnięcia, korekty równowagi
- Narzędzie: push-recovery reflex

LIPM - uproszczona dynamika chodu

- Ciało traktowane jako punkt masowy (CoM) na sztywnym, bezmasowym wahadle
- Ruch CoM w 2D: $\ddot{x} = \omega_0^2 \cdot (x - z_{ZMP})$
- Rozwiązanie: $x(t) = C_1 \cdot \cosh(\omega_0 t) + C_2 \cdot \sinh(\omega_0 t) + x_{ZMP}$
- Generowanie trajektorii CoM przez optymalizację QP
- Podstawa większości komercyjnych humanoida (ASIMO, Atlas, NAO)
- Ograniczenie: zakłada stałą wysokość CoM

Obliczanie ZMP z trajektorii CoM (model LIPM)

```
import numpy as np

def compute_zmp(com_pos: np.ndarray,
                com_acc: np.ndarray,
                com_height: float,
                g: float = 9.81) -> np.ndarray:
    """Wyznacza ZMP z przyspieszenia CoM (model LIPM, 2D: x i y)."""
    # com_pos [m], com_acc [m/s^2], com_height [m]
    zmp_x = com_pos[0] - (com_height / g) * com_acc[0]
    zmp_y = com_pos[1] - (com_height / g) * com_acc[1]
    return np.array([zmp_x, zmp_y])

# Przykład:
pos = np.array([0.05, 0.0]) # CoM 5 cm przed srodkiem stopy
acc = np.array([0.3, 0.0]) # przyspieszenie [m/s^2]
zmp = compute_zmp(pos, acc, com_height=0.85)
print(f'ZMP = {zmp}')      # np. [0.024, 0.0] m
```

Opis: Wynik ZMP [m] należy porównać z wielobokiem podparcia i sprawdzić margines bezpieczeństwa

Prosty model LIPM - symulacja ruchu CoM

```
import numpy as np

class LIPM:
    def __init__(self, com_height: float = 0.85, g: float = 9.81):
        self.omega = np.sqrt(g / com_height) # częstotliwość LIPM [rad/s]
        self.x = 0.0 # pozycja CoM [m]
        self.vx = 0.0 # prędkość CoM [m/s]

    def step(self, zmp_x: float, dt: float) -> tuple:
        """Jeden krok całkowania równania LIPM:  $x_{ddot} = \omega^2(x - zmp)$ ."""
        acc = self.omega**2 * (self.x - zmp_x)
        self.vx += acc * dt
        self.x += self.vx * dt
        return self.x, self.vx

lipm = LIPM(com_height=0.85)
for t in np.arange(0, 1.0, 0.005):
    x, vx = lipm.step(zmp_x=0.02, dt=0.005) # ZMP 2 cm przed stopą
```

Opis: $\omega = \sqrt{g/h} \approx 3,4$ rad/s dla $h=0,85$ m; numeryczne całkowanie Euler – wystarczające dla demonstracji.

Reakcja na zakłócenia zewnętrzne

- **Pchnięcie (push):** nagła zmiana prędkości CoM → CP wychodzi z dozwolonej strefy → korekta kroku
- **Mechanizm ankle strategy:** regulacja momentu w stawie skokowym (małe zakłócenia $\leq \sim 5 \text{ N}\cdot\text{s}$)
- **Mechanizm hip strategy:** ruch tułowia kompensuje przesunięcie CP (średnie zakłócenia)
- **Mechanizm stepping strategy:** postawienie dodatkowego kroku w kierunku CP (duże zakłócenia)
- **Poślizg stopy:** detekcja z czujnika F/T – nagły skok siły bocznej → reset estymatora i replan
- **Utrata kontaktu:** siła normalna $F_z < 10 \text{ N}$ → przejście do trybu push-recovery
- **Opóźniona detekcja przeszkody:** aktywacja hamowania awaryjnego i replan lokalny (10 ms)

Stabilność, a decyzje nawigacyjne

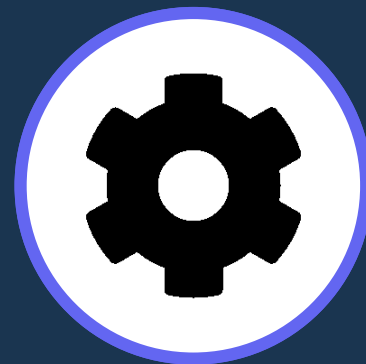
- **Reguła nadrzędna:** decyzja o ruchu jest akceptowana tylko wtedy, gdy predykcja ZMP pozostaje w wieloboku
- **Sprzężenie planista-kontroler:** planista kroków przekazuje cele, kontroler zgłasza aktualny margines
- **Prędkość chodu, a stabilność:** wyższa prędkość = mniejszy czas podwójnego podparcia = wyższe ryzyko
- **Teren pochyły lub śliski:** kontroler redukuje prędkość i wydłuża fazę podwójnego podparcia
- **Sygnal margines stabilności:** gdy $SM < \text{threshold}$ → planista obniża prędkość lub zatrzymuje robota
- **Priorytet:** stabilność > plan ruchu > optymalizacja energii > czas misji
- **Architektura:** stability monitor działa w pętli 1 kHz i może vetować komendy planisty lokalnego

02

SEKCJA

Kontrola wykonania ruchu

Warstwy sterowania, WBC, PID, symulacja, akwatory



Warstwy sterowania w humanoidzie

Warstwa wysoka (High-level)

- Planista globalny i lokalny
- Generator sekwencji kroków
- Trajektoria CoM (LIPM / QP)
- Częstość: 10–50 Hz
- Wyjście: ref. trajektoria CoM, stóp

Warstwa środkowa (WBC)

- Whole-Body Controller
- Optymalizacja całego ciała
- Przestrzeganie kontaktów i ZMP
- Częstość: 200–500 Hz
- Wyjście: ref. kąty stawów lub τ_{ref}

Warstwa niska (aktuator)

- Regulatory PID / impedancji
- Pętle momentowe / prędkościowe
- Enkodery i czujniki F/T
- Częstość: 1000–5000 Hz
- Wyjście: napięcie / prąd silnika

Priorytety zadań w sterowaniu (WBC)

P1 Stabilność

ZMP w wieloboku podparcia; Capture Point w zasięgu; stabilność LIPM

P2 Kontakt stóp

Siły kontaktu ≥ 0 ; przyczepność (tarcie); poprawna orientacja stopy

P3 Realizacja ruchu

Śledzenie trajektorii CoM i stóp zgodnie z planem footstep

P4 Pozycja tułowia

Orientacja tułowia do pionu;
pozycja głowy do percepcji

P5 Minimalizacja

Minimalizacja zużycia energii
i momentów stawowych
(*regularyzacja*)

Zasada działania Whole-Body Control (WBC)

- **WBC** wyznacza optymalny wektor momentów τ dla wszystkich stawów jednocześnie
- **Formułowane jako QP** (Quadratic Programming): $\min \frac{1}{2} \cdot \tau^T Q \tau \text{ s.t. } A\tau \leq b$
- **Ograniczenia**: równania dynamiki, siły kontaktu, zasięgi stawów, ograniczenia ZMP
- **Hierarchia zadań**: każde zadanie ma macierz wagową; zadania niższe nie zakłócają wyższych
- **Null-space projection**: niższe priorytety realizowane w przestrzeni zerowej wyższych
- **Czas obliczeń**: solver OSQP lub qpOASES wyznacza rozwiązanie w ~ 1 ms na CPU embedded
- **Integracja z LIPM**: trajektoria CoM z modelu LIPM $\rightarrow$ WBC $\rightarrow$ momenty stawów

Regulator PID dla kierunku yaw robota

```
class PIDController:
    def __init__(self, kp, ki, kd, output_limit=1.0):
        self.kp, self.ki, self.kd = kp, ki, kd
        self.limit = output_limit
        self.integral = 0.0
        self.prev_error = 0.0

    def update(self, setpoint: float, measured: float, dt: float) -> float:
        """Krok regulatora PID; zwraca sygnał sterujący [rad/s]."""
        error = setpoint - measured
        # Normalizacja błędu kąta do [-pi, pi]
        error = (error + 3.14159) % (2*3.14159) - 3.14159
        self.integral += error * dt
        derivative = (error - self.prev_error) / max(dt, 1e-6)
        self.prev_error = error
        output = self.kp*error + self.ki*self.integral + self.kd*derivative
        return max(-self.limit, min(self.limit, output)) # saturacja
```

Opis: $K_p \approx 2,0$; $K_i \approx 0,1$; $K_d \approx 0,05$ – typowe wartości dla regulacji kierunku humanoida z IMU 200 Hz

Śledzenie trajektorii - regulator nadążny

- **Cel:** robot ma podążać za zadaną trajektorią ($x_{ref}(t)$, $y_{ref}(t)$, $yaw_{ref}(t)$) z minimalnym błędem
- **Błąd boczny** (cross-track error): odległość prostopadła od aktualnej pozycji do trajektorii
- **Błąd podłużny** (along-track error): opóźnienie lub wyprzedzenie względem parametru t
- **Metoda Pure Pursuit:** cel = punkt na trajektorii w odległości look-ahead L_d od robota
- **Metoda Stanley Controller:** kombinacja błędu bocznego i kąta nagłówka, stosowana w autonomicznych pojazdach
- **Dla humanoida:** oddzielne regulatory dla CoM (translacja) i tułowia (orientacja yaw)
- **Anti-windup:** ograniczenie całkowania w PID, gdy robot jest poza pętlą (np. stoi w miejscu)

Ograniczenia aktuatorów i ich wpływ na planowanie

- **Moment maksymalny $\tau_{\max}$** : przekroczenie powoduje przegrzanie silnika lub uszkodzenie przekładni
- **Prędkość kątowna $\omega_{\max}$** : zbyt szybki ruch może wywołać efekt kinetyczny zaburzający stabilność
- **Saturacja sygnału**: gdy komendy WBC przekraczają możliwości, planista musi zwolnić
- **Przegrzewanie termiczne**: ciągła praca przy $\tau_{\max}$ skraca żywotność; monitoring temperatury uzwojeń
- **Pasmo przenoszenia**: aktuator serwo działa poprawnie do ~ 50 Hz; powyżej – opóźnienie fazowe
- **Wpływ na planowanie**: planista globalny może obniżyć prędkość tras o 20% przy dużym obciążeniu
- **Narzędzia diagnostyczne**: logowanie momentów na osi czasu pozwala identyfikować przeciążone stawy

Saturacja i monitorowanie sygnału sterującego

```
import numpy as np

def saturate_torques(tau_desired: np.ndarray,
                    tau_max: np.ndarray,
                    warn_ratio: float = 0.85) -> tuple:
    """Nasycenie momentow stawowych z ostrzezeniem o przeciazeniu."""
    tau_sat = np.clip(tau_desired, -tau_max, tau_max)
    sat_ratio = np.abs(tau_desired) / (tau_max + 1e-6)
    overloaded = np.where(sat_ratio > warn_ratio)[0]
    warnings = []
    for idx in overloaded:
        warnings.append(f'Staw {idx}: {sat_ratio[idx]*100:.1f}% tau_max')
    return tau_sat, warnings

tau_ref = np.array([120.0, -85.0, 200.0]) # zadane momenty [Nm]
tau_lim = np.array([100.0, 100.0, 100.0]) # limity
tau_out, warns = saturate_torques(tau_ref, tau_lim)
```

Opis: Funkcja clipuje momenty i generuje ostrzeżenia gdy staw > 85% limitu – sygnał do zwolnienia planisty.

Symulacja przed testami na robocie

- **Cyfrowy bliźniak (digital twin):** model robota z dokładnymi parametrami inercji, tarcia i aktuatorów
- **Gazebo (ROS 2):** popularny w akademii; silniki fizyki: ODE, Bullet, DART; integracja z ROS 2 control
- **MuJoCo:** lider w lokomocji RL; dokładna symulacja kontaktu; używany przez Boston Dynamics, DeepMind
- **PyBullet:** szybki, pythonowy; prostszy w konfiguracji; stosowany w badaniach akademickich
- **Zasada bezpiecznego przejścia:** algorytm działa poprawnie w symulacji → testy na robocie w ograniczonej przestrzeni
- **Metryki sym2real gap:** różnica prędkości kroku, błędu ZMP, zużycia energii między symulacją a robotem
- **Domain randomization:** losowanie tarcia, masy i opóźnień w symulacji wzmacnia odporność kontrolera

Pętla sterowania w symulacji (*PyBullet* / *abstrakcja*)

```
import numpy as np

class SimulationLoop:
    def __init__(self, robot, planner, controller, dt=0.005):
        self.robot      = robot      # interfejs do symulatora
        self.planner     = planner    # planista (ścieżka + footstep)
        self.controller  = controller # WBC lub PID
        self.dt          = dt

    def run(self, goal_pos, max_steps=10_000):
        for step in range(max_steps):
            state      = self.robot.get_state()      # odczyt stanu
            footsteps  = self.planner.plan(state, goal_pos)
            tau        = self.controller.compute(state, footsteps)
            tau, _     = saturate_torques(tau, self.robot.tau_max)
            self.robot.apply_torques(tau)            # wyslij komendy
            self.robot.step_simulation(self.dt)
            if np.linalg.norm(state['pos'] - goal_pos) < 0.05: break
```

Opis: Pętla odczyt→plan→kontrola→aktuacja w kroku $dt=5$ ms (200 Hz); kryterium stopu: odległość < 5 cm

03

SEKCJA

Architektura systemowa, bezpieczeństwo i walidacja

ROS 2, fail-safe, monitoring jakości i studium przypadku



Integracja modułów w ROS 2

- **Topiki (*topics*):** asynchroniczna wymiana danych; sensor_msgs/Imu, nav_msgs/Odometry, geometry_msgs/Twist
- **Serwisy (*services*):** synchroniczne wywołania; np. /set_goal, /reset_localization, /emergency_stop
- **Akcje (*actions*):** długotrwałe zadania z feedbackiem; nav2 NavigateToPose – standard dla nawigacji
- **TF2:** drzewo transformacji; każdy node publikuje tf; lookupTransform() z buforowaniem przez ~5 s
- **ros2_control:** ustandaryzowany interfejs dla aktuatorów – hardware_interface + controller_manager
- **QoS (*Quality of Service*):** RELIABLE dla danych krytycznych (stan robota), BEST_EFFORT dla sensorów
- **Modularność:** każdy moduł (SLAM, planner, WBC) jako oddzielny node lub lifecycle node

Przykładowy pipeline uruchomieniowy (ROS 2)

1. Sensory

lidar_driver, imu_driver,
depth_camera_node – odczyt
i preprocessing danych sensorycznych

2. Lokalizacja

slam_toolbox lub Nav2 AMCL – estymacja
pozycji w mapie; publikacja /odom
i /map→base tf

3. Mapa kosztów

Nav2 costmap_2d – global_costmap
i local_costmap aktualizowane z LiDAR
i RGB-D

4. Planista globalny

Nav2 NavFn lub Smac Planner – A* lub
Hybrid-A* z cost map; ścieżka do /plan
topic

5. Planista lokalny

Nav2 DWB Controller – DWA z krytykami
trajektorii; komendy do /cmd_vel lub
footstep

6. WBC + wykonanie

ros2_control + whole_body_controller –
śledzenie footstep, moment stawowy,
ZMP monitoring

Publikacja celu i odbiór stanu w ROS 2 (Python)

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from nav_msgs.msg import Odometry

class NavClient(Node):
    def __init__(self):
        super().__init__('nav_client')
        self.goal_pub = self.create_publisher(
            PoseStamped, '/goal_pose', 10)
        self.odom_sub = self.create_subscription(
            Odometry, '/odom',
            self.odom_callback, 10)
    def odom_callback(self, msg: Odometry):
        x = msg.pose.pose.position.x
        y = msg.pose.pose.position.y
        self.get_logger().info(f'Pos: ({x:.2f}, {y:.2f})')
    def send_goal(self, x, y):
        msg = PoseStamped()
        msg.header.frame_id = 'map'
        msg.pose.position.x = x; msg.pose.position.y = y
        self.goal_pub.publish(msg)
```

Opis: Minimalna integracja ROS 2: subscriber /odom + publisher /goal_pose; Nav2 obsługuje resztę pipeline'u.

Bezpieczeństwo i mechanizmy fail-safe

Detekcja sytuacji awaryjnych

- **Detekcja upadku:** akcelerometr $|a| > 3g$ lub roll/pitch $> 45^\circ$
- **Brak kontaktu obu stóp:** $F_{z_L} + F_{z_R} < 20 \text{ N}$ przez $> 200 \text{ ms}$
- **Przekroczenie limitu momentu:** $\tau > 1,2 \cdot \tau_{\max}$ przez $> 50 \text{ ms}$
- **Utrata sensorów:** brak wiadomości IMU przez $> 50 \text{ ms} \rightarrow$ alarm
- **Kolizja:** siła na strukturze $> F_{\text{collision_thresh}}$
- **Błąd lokalizacji:** kowariancja $P > P_{\max} \rightarrow$ stop i relokalizacja

Reakcje bezpieczne (fail-safe)

- **Zatrzymanie awaryjne:** momenty $\rightarrow 0$ w ciągu 50 ms
- **Bezpieczna postura:** zejście do pozycji kucającej lub siadania
- **Tryb degradacji:** nawigacja tylko na IMU + enkodery
- **Powiadomienie operatora:** alert przez ROS 2 diagnostics
- **Automatyczny restart** po relokalizacji (autorecovery)
- **Hardware e-stop:** przycisk fizyczny odcina zasilanie aktuatorów

Metryki online do monitorowania jakości nawigacji

- **Błąd lokalizacji:** bieżące RMSE pozycji względem trajektorii referencyjnej [m]
- **Liczba korekt kroku:** ile razy planista footstep przeplanował z powodu zmiany otoczenia
- **Margines ZMP:** $\min(\text{distance}(\text{ZMP}, \text{edge}))$ w bieżącym cyklu [cm]
- **Temperatura aktuatorów:** topik /joint_temperatures; próg ostrzegawczy 70°C, krytyczny 85°C
- **Straty kontaktu:** ile razy $F_z < \text{próg}$ podczas fazy podparcia – wskaźnik poślizgu lub złej nawierzchni
- **Near-miss:** liczba sytuacji gdy ZMP < 1 cm od krawędzi wieloboku w ostatnich 60 s
- **Narzędzie:** ROS 2 diagnostics + rqt_runtime_monitor; zapis do MCAP/rosbag2 do późniejszej analizy

Walidacja eksperymentalna - scenariusze testowe

Nawigacja prosta

Przejsie A→B 10 m po równej powierzchni; metryka: RMSE pozycji, czas, zużycie energii

Omijanie przeszkód

5 statycznych + 2 dynamiczne przeszkody; metryka: liczba kolizji, liczba replanu

Schody

3 stopnie $h=15$ cm; metryka: powodzenie, czas, liczba korekt ZMP, szczyt momentów

Pchnięcie boczne

Impuls 20 Ns; metryka: amplituda CP, czas odzyskania równowagi, krok korekcyjny

Drzwi i progi

Otwieranie klamki, przekroczenie progu 3 cm; metryka: sukces, czas interakcji

Długa misja

45 min patrol budynku; metryka: dryft SLAM, zużycie energii, liczba incydentów

Najczęstsze błędy projektowe

- **Zbyt optymistyczny model dynamiki:** symulator nie uwzględnia ugięcia struktury ani tarcia statycznego stawów
- **Brak synchronizacji czasowej:** IMU i kamera nie zsynchronizowane → duchy w mapie i błędne fusje
- **Ignorowanie opóźnień:** kamera ma 33 ms opóźnienie; bez kompensacji EKF daje błędną pozycję
- **Niestabilność kontaktu:** zbyt twarde regulatory powodują drgania w fazie kontaktu stopy z podłożem
- **Brak anti-windup w PID:** przy saturacji moment integrujący rośnie bez ograniczeń → overshooting
- **Monolityczna architektura:** cały system w jednym procesie – awaria jednego modułu blokuje całość
- **Zbyt rzadkie testy na robocie:** przeniesienie działającego symulatora na fizyczny robot bez etapów pośrednich

Studium przypadku: humanoid w zadaniu usługowym

- **Scenariusz:** robot dostarcza dokument z recepcji (parter) do sali konferencyjnej (2. piętro)
- **Etap 1 – Inicjalizacja:** wczytanie mapy budynku, lokalizacja AMCL, plan globalny z windą lub schodami
- **Etap 2 – Korytarz parter:** DWA z detekcją YOLO, 3 osoby omijane, ORCA dla dynamicznych obiektów
- **Etap 3 – Windy:** detekcja drzwi, wciśnięcie guzika (manipulacja), oczekiwanie na otwarciu, wejście
- **Etap 4 – Korytarz 2. piętro:** reinicjalizacja AMCL po wyjściu z windy, dojście do celu
- **Etap 5 – Interakcja:** podanie dokumentu, potwierdzenie odbioru, powrót trasy odwrotnej
- **Kluczowe wyzwania:** zachowanie w windzie (mała przestrzeń), interakcja z człowiekiem, niezawodność 45 min

PODSUMOWANIE

- 01 **ZMP i Capture Point** to fundamenty równowagi dynamicznej; planista CoM musi przestrzegać ich ograniczeń
- 02 **Whole-body control** hierarchizuje zadania: stabilność > kontakt stóp > realizacja trajektorii
- 03 **PID, śledzenie trajektorii i saturacja aktuatorów** to warunki bezpiecznego wykonania ruchu
- 04 **ROS 2 + TF2 integruje wszystkie moduły**; synchronizacja i modularność są warunkiem niezawodności
- 05 **Walidacja w symulacji** (MuJoCo, Gazebo) poprzedza testy na robocie. Cyfrowy bliźniak oszczędza sprzęt

PODSUMOWANIE

Nawigacja Robotów Humanoidalnych

W1

Percepcja
i mapowanie

W2

Lokalizacja
i fuzja sensoryczna

W3

Planowanie ruchu
i unikanie przeszkód

W4

Chód, stabilność
i integracja systemu

Pełna pętla nawigacyjna: Sensory → Estymacja stanu → Planowanie → Sterowanie → Aktuatory → Sensory

Reinforcement
learning
dla lokomocji

Nawigacja
semantyczna

Social navigation
i HRI

Multimodalna
percepcja

Adaptacyjny
chód

Tematy dalszych badań i kierunki rozwoju

1. **Reinforcement learning dla lokomocji:** uczenie chodu end-to-end bez modelu (MuJoCo + PPO/SAC)
2. **Nawigacja semantyczna:** reprezentacje CLIP/ViLD; robot rozumie polecenia językowe i mapę semantyczną
3. **Social navigation:** modele ruchu tłumu, normy kulturowe, personalizacja interakcji człowiek–robot
4. **Multimodalna percepcja:** fuzja kamery, LiDAR, taktylna + duże modele wizyjno-językowe (VLM)
5. **Adaptacyjny chód:** zmiana wzorca chodu w zależności od terenu bez ręcznego programowania
6. **Whole-body manipulation:** jednoczesna nawigacja i manipulacja, otwieranie drzwi w ruchu
7. **Cloud robotics + edge AI:** ciężkie obliczenia SLAM i planowania na serwerze edge z niskim opóźnieniem

NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 5

VLM i VLA w nawigacji
robota humanoidalnego

dr inż. Mateusz Pomianek



VLM

VLA

Sem. nav.

ROS 2 + AI

Plan wykładu

Sekcja 1

VLM - architektura, modele CLIP/LLaVA/GPT-4V, grounding, zero-shot reasoning w robotyce

Sekcja 2

VLA - modele RT-2, OpenVLA, $\pi 0$, Octo; architektura i generowanie akcji z języka i wizji

Sekcja 3

Praktyczne zastosowania: nawigacja semantyczna, instruction following, integracja z ROS 2

Kody Python

Zapytanie VLM o scenę, grounding w mapie, router VLM→planner, pętla VLA w symulacji

01

SEKCJA

VLM - modele wizyjno-językowe

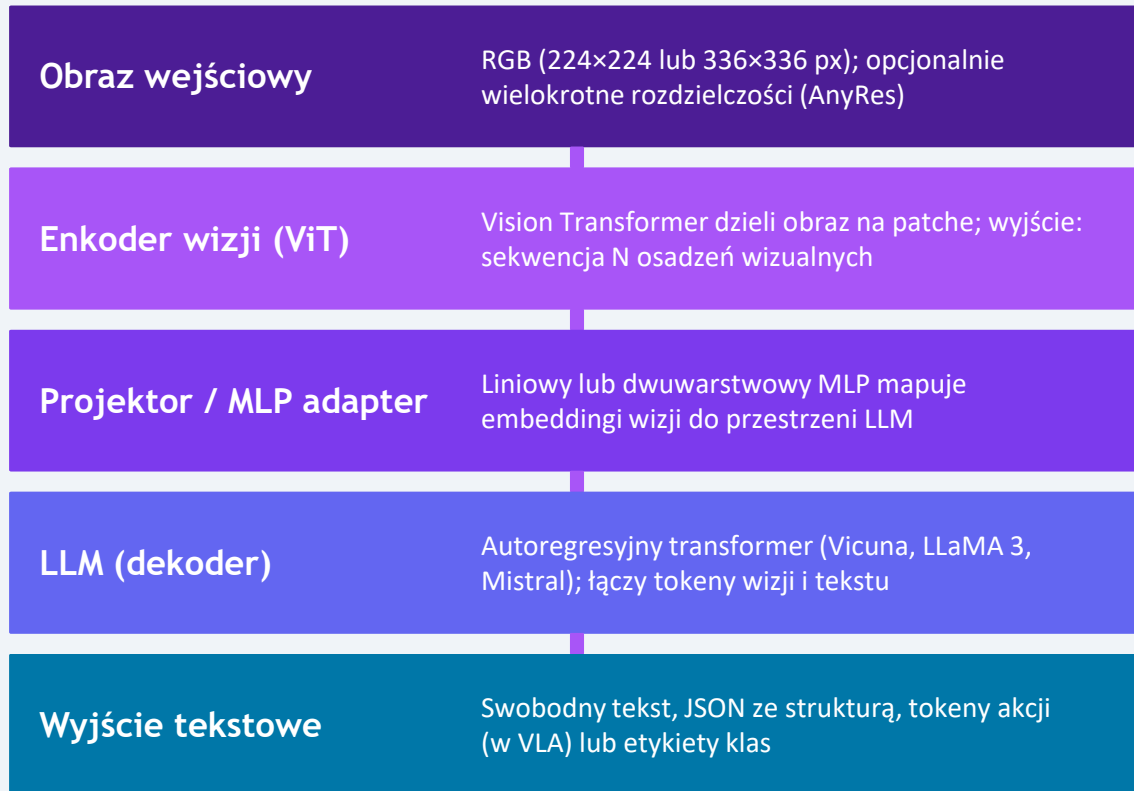
*Architektura, kluczowe modele, grounding
i zastosowania w robotyce*



Czym jest Vision-Language Model (VLM)?

- **VLM** to model uczenia głębokiego łączący kodowanie obrazu i tekstu we wspólnej przestrzeni reprezentacji
- **Zadania:** opis obrazu (captioning), VQA (visual question answering), wykrywanie obiektów z opisu, rozumowanie przestrzenne
- **Architektura typowa:** enkoder wizji (ViT lub CNN) + projektor + duży model językowy (LLM)
- **Trening:** pretraining na miliardach par obraz-tekst z internetu (LAION, CC12M, WebLI...)
- **Fine-tuning:** dostrojenie na zadaniach robotycznych z parami (obs, opis, akcja)
- **Przykłady:** CLIP (OpenAI), LLaVA-1.6, InstructBLIP, GPT-4o Vision, Gemini 1.5 Pro, Qwen-VL
- **Kluczowa własność:** zero-shot generalization - rozumienie scen niewidzianych podczas treningu

Architektura VLM: od pikseli do tokenów



Kluczowe cechy

- Freeze ViT → trenuj tylko projektor + LLM (LLaVA)
- Jeden forward pass przez cały stos
- Kontekst: typowo 4 096-128 000 tokenów
- Parametry: 7B-72B (edge: 3B-7B)
- Latencja GPU: 100-800 ms / zapytanie
- Kwantyzacja 4-bit na embedded GPU

CLIP: fundament grounded perception

Idea CLIP (OpenAI, 2021)

- **Trening kontrastowy:** 400M par (obraz, opis) z internetu
- **Dwa enkodery:** Image Encoder (ViT-L) + Text Encoder (transformer)
- **Cel:** maksymalizacja podobieństwa $\cos.$ dla pasujących par
- **Wynik:** wspólna przestrzeń embeddingów o wymiarze 512-1024
- **Możliwość zero-shot klasyfikacji:** obraz vs. opisy klas
- Baza dla LLaVA, DALL-E, Flamingo, CogVLM

CLIP w nawigacji humanoida

- **Lokalizacja semantyczna:** podobieństwo embedding (klatka) vs. embedding (opis celu)
- Np. "znajdź niebieskie drzwi" → szukaj w mapie klatek z max similarity
- **Object grounding:** ranking regionów ROI vs. zapytanie językowe
- **Działanie w czasie rzeczywistym:** CLIP ViT-B/16 ~10 ms na GPU
- **Ograniczenie:** brak rozumowania wieloetapowego; tylko matching

LLaVA i Visual Question Answering w robotyce

- **LLaVA (Large Language and Vision Assistant):** CLIP ViT + 2-layer MLP projektor + LLaMA/Vicuna
- **Trening dwuetapowy:** (1) freeze ViT, ucz projektor; (2) fine-tune projektor + LLM na danych instrukcji
- **VQA dla robota:** robot zadaje pytanie o obraz z kamery; VLM odpowiada w języku naturalnym
- **Przykłady pytań VQA:** "Czy drzwi są otwarte?", "Ile osób stoi przy windzie?", "Który korytarz jest wolny?"
- **Odpowiedź strukturalna:** prompt wymuszający JSON: {"door_open": true, "people_count": 2, "clear_path": "left"}
- **LLaVA-1.6 (34B):** rozumowanie przestrzenne, zliczanie, rozróżnianie lewej/prawej strony
- **Wersje on-device:** LLaVA-Phi-3-mini (3,8B) - możliwa na Jetson Orin AGX (~200 ms / klatka)

Grounding semantyczny: od słów do lokalizacji

1

Opis celu

Użytkownik lub system misji podaje opis: "idź do stolika z kawą przy oknie"

2

Feature extraction

VLM (LLaVA lub CLIP) koduje klatki z kamery; VLM opisuje każdy kandydujący region mapy

3

Matching

Cosinus similarity lub LLM scoring między embeddingiem opisu, a embeddingami regionów

4

Lokalizacja na mapie

Region z najwyższym score → jego współrzędne (x,y) na mapie → cel dla A*

Zero-shot vs. few-shot reasoning w nawigacji

Zero-shot

- Model nigdy nie widział danej sceny ani zadania podczas treningu
- Polecenie: "idź do drukarki" - VLM rozumie bez przykładów
- Działa gdy obiekt jest w słowniku treningu VLM (>1B tokenów tekstu)
- Słabszy dla specyficznego sprzętu laboratoryjnego lub lokalnych obiektów
- Praktyka: GPT-4o opisuje scenę i wskazuje cel nawigacyjny

Few-shot (in-context learning)

- 2-5 przykładów par (obraz, opis, akcja) w prompcie kontekstowym
- Nie wymaga fine-tuningu - szybkie dostosowanie do nowego środowiska
- Np. 3 zdjęcia korytarza ze strzałkami + pytanie → kierunek ruchu
- Kluczowe dla robotów przemysłowych z niestandardowymi obiektami
- Ograniczenie: długi kontekst = wyższy koszt inferowania i latencja

Zapytanie VLM o stan sceny (Anthropic / OpenAI API)

```
import base64, httpx, json

def query_vlm_scene(image_bgr, question: str,
                    model='claude-sonnet-4-6') -> dict:
    """Wysyla klatke kamery do VLM i zwraca odpowiedz JSON."""
    import cv2
    _, buf = cv2.imencode('.jpg', image_bgr, [cv2.IMWRITE_JPEG_QUALITY, 85])
    b64 = base64.b64encode(buf).decode()
    payload = {
        "model": model, "max_tokens": 256,
        "messages": [{ "role": "user", "content": [
            { "type": "image", "source": { "type": "base64",
                "media_type": "image/jpeg", "data": b64 },
            { "type": "text", "text": f'Odpowiedz TYLKO JSON. {question}" }
        ] } ] }
    }
    r = httpx.post('https://api.anthropic.com/v1/messages',
                  json=payload, headers={'x-api-key': API_KEY,
                  'anthropic-version': '2023-06-01'}, timeout=5.0)
    return json.loads(r.json()['content'])[0]['text'])
```

Pytanie np.: "Czy droga do przodu jest wolna? **Odpowiedz:** {"clear": true/false, "obstacle": string}."

02

SEKCJA

VLA - modele wizyjno-językowe-akcyjne

*RT-2, OpenVLA, $\pi 0$, Octo, architektura i
generowanie akcji*



Czym jest Vision-Language-Action Model (VLA)?

- **VLA rozszerza VLM o głowicę akcji** (action head): model generuje nie tylko tekst, lecz tokeny ruchu
- **Wejście:** obraz (lub sekwencja klatek) + polecenie językowe; **wyjście:** wektor akcji lub tokeny dyskretne
- **RT-2 (Google DeepMind, 2023):** fine-tuning PaLI-X (55B) i PaLM-E na danych robotycznych
- **OpenVLA (Stanford, 2024):** open-source; Prismatic VLM + 7B LLaMA; tokeny akcji jako tekst
- **$\pi 0$ (Physical Intelligence, 2024):** diffusion policy head dla ciągłych akcji; duże słownictwo gestów
- **Octo (Berkeley, 2024):** transformer-based; multi-task; przyjmuje różne modalności wejściowe
- **Kluczowa różnica od klasycznej polityki RL:** VLA używa wiedzy językowej do generalizacji zero-shot

Architektura VLA, od obserwacji do akcji



Kluczowe cechy

- **Częstość wyjść:** 3-10 Hz (ograniczona przez LLM)
- **Buforowanie akcji:** temporal ensemble lub action chunking
- **Fine-tuning:** ~100-10 000 demonstracji
- Imitation learning + RL fine-tuning
- **Deployment:** Jetson AGX Orin (7B q4) $\approx$ 3 Hz
- **Bezpieczeństwo:** clip akcji + velocity guard

Przegląd kluczowych modeli VLA

RT-2 (Google, 2023)

- **Base:** PaLI-X 55B lub PaLM-E 562B
- **Tokeny akcji:** dyskretne (256 bin/oś)
- Co-fine-tuning na Web + Robot data
- Generalizacja zero-shot do nowych obiektów
- Proprietary; działa na TPU v4
- Wykazuje chain-of-thought reasoning

OpenVLA (Stanford, 2024)

- **Base:** Prismatic (*SigLIP* + *DINOv2*) + LLaMA 7B
- **Open source:** wagi na HuggingFace
- Fine-tuning na Open-X-Embodiment (~970 k ep.)
- Akcje jako tekst: "move_left 0.3"
- LoRA fine-tuning w <24 h na 1× A100
- Dobrze udokumentowane API

$\pi 0$ / Octo (2024)

- **$\pi 0$:** diffusion action head (*ciągłe akcje*)
- Wspiera dexterious manipulation
- **Octo:** multi-task transformer policy
- **Modular:** różne enkodery wizji
- Open-X-Embodiment benchmark SoTA
- Integracja z ROS 2 przez action server

Tokenizacja akcji: jak VLA generuje ruch

- **Dyskretna tokenizacja:** zakres każdej osi ruchu (np. $\pm 0,05$ m/krok) dzielony na 256 równych binów
- **Każda oś akcji** $\rightarrow$ jeden token z alfabetu $[0 \dots 255]$; cały wektor akcji = K tokenów po sobie
- **Dekodowanie:** token_id $\rightarrow$ wartość ciągła przez de-binning + ewentualne odszumowanie
- **Action chunking:** VLA generuje N kolejnych kroków akcji naraz (np. $N=10$); redukuje latencję
- **Temporal ensemble:** ważona suma K ostatnich prognoz dla bieżącego kroku - wygładzenie ruchu
- **Diffusion policy (π_0):** akcja jako wynik odwróconego procesu dyfuzji; ciągła, wielomodalna
- **Hybrid:** tokeny dyskretnie dla nawigacji (v, ω) + ciągłe diffusion dla manipulacji dłońmi

Fine-tuning VLA na danych robotycznych

Zbieranie danych demonstracji

- **Teleoperation:** operator steruje robotem; zapis (obs, action) w ~ 10 Hz
- **Symulacja:** generowanie danych w MuJoCo / Isaac Sim (szybko, bez ryzyka)
- **Play data:** swobodne zbieranie bez konkretnego zadania; labels posta hoc
- **Liczba demonstracji:** 100-10 000 epizodów zależy od złożoności zadania
- **Format:** HDF5 lub RLDS; zgodność z Open-X-Embodiment dataset

Strategie fine-tuningu

- **Full fine-tuning:** wszystkie wagi; potrzebny duży GPU (≥ 80 GB VRAM)
- **LoRA (Low-Rank Adaptation):** trainable rank-r matrices; 16-32 GB GPU
- **Adapter modules:** dodatkowe warstwy między blokami transformera
- **Co-training:** robot data + Web data $\rightarrow$ zapobiega catastrophic forgetting
- **Ewaluacja:** success rate na held-out zadaniach w symulacji i na robocie

Integracja OpenVLA w pętli sterowania (HuggingFace)

```
from transformers import AutoModelForVision2Seq, AutoProcessor
import torch, numpy as np

processor = AutoProcessor.from_pretrained('openvla/openvla-7b',
                                         trust_remote_code=True)

model = AutoModelForVision2Seq.from_pretrained(
    'openvla/openvla-7b', torch_dtype=torch.bfloat16,
    low_cpu_mem_usage=True, trust_remote_code=True).cuda()

def get_action(image_rgb: np.ndarray, instruction: str) -> np.ndarray:
    """Zwraca wektor akcji [dx, dy, dz, drx, dry, drz, gripper]."""
    prompt = f'In: What action should the robot take to {instruction}?\nOut:'
    inputs = processor(prompt, image_rgb).to('cuda:0', torch.bfloat16)
    with torch.inference_mode():
        action = model.predict_action(**inputs, unnorm_key='bridge_orig',
                                     do_sample=False)
    return action # numpy (7,) [m, m, m, rad, rad, rad, bool]
```

Opis: OpenVLA zwraca 7-DoF akcję w jednostkach metrycznych; unnorm_key określa skalowanie do danego robota.

Latencja i deployment VLA na robocie

Cloud GPU (A100/H100)

Latencja ~80-150 ms; konieczna łączność sieciowa; nieakceptowalne dla dynamicznych zadań

Edge GPU (Jetson Orin)

Latencja 300-600 ms dla 7B q4; wystarczające dla nawigacji; niezależność od chmury

Action chunking

Generuj $N=10$ kroków naraz; wykonuj lokalnie; VLA odpytywana co $N \times dt$ ms $\rightarrow$ efektywnie 10-30 Hz

Distillacja / pruning

Nauczyciel 34B $\rightarrow$ student 3B; zachowanie 85-90% skuteczności przy 5 $\times$ szybszej inferencji

Kwantyzacja 4-bit

AWQ lub GPTQ: 7B model zajmuje ~4 GB VRAM; minimalna utrata jakości akcji ($< 3\%$ success rate)

Speculative decoding

Draft model (0,5B) proponuje tokeny; duży model weryfikuje; 2-3 $\times$ przyspieszenie bez utraty jakości

03

SEKCJA

Praktyczne zastosowania w nawigacji humanoida

*Nawigacja semantyczna, instruction following,
integracja z ROS 2, ograniczenia*



Nawigacja semantyczna z VLM

- **Klasyczna nawigacja:** cel = współrzędne (x, y); robot nie rozumie sensu celu
- **Nawigacja semantyczna:** cel = opis językowy "znajdź salę z projekcją slajdów" lub "idź do kuchni"
- **Etap offline: budowa mapy semantycznej:** robot objeżdża budynek; VLM opisuje każdą pozycję
- **Etap online: lokalizacja celu:** embedding(opis celu) vs. embeddingi pozycji w mapie → argmax sim
- **CLIP-Fields (2022):** neural field z embeddingami CLIP w każdym wokselu mapy 3D
- **SQA3D / EQA (Embodied QA):** robot odpowiada na pytania o przestrzeń, poruszając się po środowisku
- **ConceptFusion (2023):** mapa z deskryptorami CLIP + SAM na chmurze punktów - open-vocab grounding 3D

Pipeline instruction following: od polecenia do akcji

1

**Polecenie
użytkownika**

"Przynieś mi kawę z kuchni
na 2. piętrze" - tekst lub
mowa (ASR → tekst)

2

LLM task planner

Duży LLM (GPT-4o) rozkłada
polecenie na podrzędne
cele: navigate → pick →
navigate → place

3

**VLM scene
grounding**

CLIP lub LLaVA lokalizuje
obiekty ("kawa", "kuchnia")
w mapie semantycznej

4

Nav2 + footstep

Klasyczna nawigacja z A* do
każdego podzadania;
footstep planner dla
precyzyjnych ruchów

Grounding semantyczny w mapie topologicznej

```
import numpy as np
from transformers import CLIPModel, CLIPProcessor
import torch

class SemanticMap:
    def __init__(self, model_name='openai/clip-vit-base-patch16'):
        self.model = CLIPModel.from_pretrained(model_name).eval().cuda()
        self.proc = CLIPProcessor.from_pretrained(model_name)
        self.nodes = [] # lista (x, y, emb) wezlow topologicznych

    def add_node(self, x, y, image_rgb):
        inp = self.proc(images=image_rgb, return_tensors='pt').to('cuda')
        with torch.no_grad():
            emb = self.model.get_image_features(**inp)
            emb = emb / emb.norm(dim=-1, keepdim=True) # normalizacja L2
        self.nodes.append((x, y, emb.cpu().numpy()))

    def find_goal(self, query_text: str) -> tuple:
        inp = self.proc(text=[query_text], return_tensors='pt').to('cuda')
        with torch.no_grad():
            t_emb = self.model.get_text_features(**inp)
            t_emb = (t_emb / t_emb.norm()).cpu().numpy()
            scores = [float(t_emb @ n[2]).T for n in self.nodes]
            best = int(np.argmax(scores))
            return self.nodes[best][0], self.nodes[best][1] # (x, y)
```

Opis: add_node() buduje mapę podczas eksploracji; find_goal() lokalizuje cel z opisu w ~2 ms

Router: VLM ocenia scenę i wywołuje właściwy planistę

```
import json

ROUTING_PROMPT = '''
Analizujesz obraz z kamery robota humanoidalnego.
Zwróć TYLKO JSON z polami:
"situation": "normal"|"obstacle_close"|"stairs"|"person_blocking",
"confidence": 0.0-1.0,
"recommended_action": "continue"|"replan"|"slow_down"|"stop"|"ask_human"
'''

def vlm_route(image_rgb, nav_node) -> str:
    result = query_vlm_scene(image_rgb, ROUTING_PROMPT) # z Kodu 1
    situation = result.get('situation', 'normal')
    confidence = result.get('confidence', 0.0)
    action = result.get('recommended_action', 'continue')
    if confidence < 0.6:
        return 'continue' # niepewny wynik - jedz dalej
    if action == 'replan':
        nav_node.trigger_replan() # sygnał do Nav2
    elif action == 'stop':
        nav_node.emergency_stop()
    return action
```

Opis: Router wywołuje VLM co ~500 ms (asynchronicznie); próg confidence=0,6 filtruje halucynacje modelu

Integracja VLM/VLA z ROS 2: wzorzec asynchroniczny

- **VLM działa zbyt wolno** (>100 ms) dla synchronicznej pętli Nav2 (50 Hz) - konieczna asynchroniczność
- **Wzorzec:** dedykowany node `/vlm_node` subskrybuje `/camera/image_raw`, publikuje `/vlm_observations`
- **Buffer:** `/vlm_observations` to topic z QoS BEST_EFFORT, depth=1 - zawsze najnowsza interpretacja
- **Nav2 BehaviorTree:** VLM jako zewnętrzny condition node - sprawdza buffer i blokuje w razie "stop"
- **Action server:** `/vlm_ground_goal` [text_goal → PoseStamped] - lokalizacja semantyczna na żądanie
- **Lifecycle node:** VLM node uruchamiany/zatrzymywany przez Nav2 state machine (oszczędność GPU)
- **Monitoring:** `/vlm_diagnostics` - latencja, liczba halucynacji wykrytych przez sanity check

Ograniczenia i wyzwania VLM/VLA w robotyce

Problemy techniczne

- **Halucynacje:** VLM "widzi" obiekty których nie ma lub fałszywie ocenia odległości
- **Latencja:** 100-800 ms; nieakceptowalne przy szybkim ruchu ($>0,5$ m/s)
- **Dryft kontekstu:** długie konwersacje tracą koherencję opisu środowiska
- **Sim-to-real gap:** model trenowany na zdjęciach z internetu słabo generalizuje na obraz fisheye robota
- **Koszt zasobów:** 7B model wymaga 16 GB VRAM; 34B - 70+ GB
- **Niespójność czasowa:** opis sceny w klatce t nie zawsze pasuje do pozycji w klatce $t+5$

Problemy bezpieczeństwa i niezawodności

- **Prompt injection:** złośliwy tekst w scenie może manipulować planistą
- **Brak gwarancji:** VLM nie daje formalnych gwarancji bezpieczeństwa (bez dowodów poprawności)
- **Fail-open vs fail-safe:** przy błędzie VLM robot musi zatrzymać się, nie kontynuować
- **Weryfikacja krzyżowa:** wyniki VLM powinny być sanity-checked przez klasyczne sensory (LiDAR)
- **Biasy modeli:** VLM może inaczej reagować na różnych ludzi ze względu na biasy danych treningowych

Przykłady wdrożeń VLM/VLA w humanoidach (2023-2025)

Figure 01 + OpenAI GPT-4o

Humanoid Figure prowadzi rozmowę o środowisku i wykonuje zadania na podstawie poleceń języka naturalnego

Unitree H1 + VLM planner

H1 z LLaVA jako scene understander: autonomiczne nawigowanie w biurze do opisanych celów

Boston Dynamics Spot + GPT-4

Spot opisuje to co "widzi,, i odpowiada na pytania; tryb VQA do inspekcji środowiska przemysłowego

RT-2 (Everyday Robots)

Robot Google wykonuje zadania pick-place z poleceń języka naturalnego; SoTA zero-shot na RT-Benchmark

$\pi 0$ (Physical Intelligence)

Humanoidalny robot kuchenny wykonuje złożone zadania (owijanie naleśników) sterowany VLA $\pi 0$

LEROBOT (HuggingFace 2024)

Open-source platforma RL+IL+VLA; demonstracje na SO-100/LeKiwi; wspólnota ~50k gwiazdek GitHub

Najlepsze praktyki inżynierskie: VLM/VLA w produkcji

- **Separacja pętli:** pętla bezpieczeństwa (1 kHz, deterministyczna) zawsze nadrzędna wobec VLM (2-5 Hz)
- **Sanity check akcji:** każda akcja VLA clampowana do fizycznie osiągalnego zakresu; ZMP sprawdzany przed wykonaniem
- **Confidence threshold:** odrzucaj wyniki VLM z confidence $< 0,7$; fallback do klasycznego planisty
- **Prompt engineering:** jasny, deterministyczny prompt z wymuszeniem formatu JSON + przykładem (few-shot)
- **Logging i replay:** zapis każdego zapytania VLM + odpowiedzi do MCAP/rosbag2 - postmortem po incydencie
- **Graceful degradation:** gdy VLM niedostępny $\rightarrow$ robot przechodzi w tryb klasycznej nawigacji metrycznej
- **Testy regresji:** automatyczny benchmark 50 zapytań VQA na obrazach referencyjnych przy każdej zmianie modelu

Kierunki badań i przyszłość VLM/VLA w nawigacji

World models

VLA z modelem świata (RSSM, JEPA): robot "wyobraża" sobie konsekwencje akcji przed ich wykonaniem

Embodied foundation models

Jeden model dla percepcji, planowania i sterowania; Gato, UniSim, UniPi - unifikacja wszystkich warstw

Continual learning

VLA aktualizuje się online z nowych demonstracji bez catastrophic forgetting; kluczowe dla wdrożeń

Multiagent VLM

Kilka robotów współdzielących mapę semantyczną; VLM jako wspólny language interface dla roju

Certifiable safety

Formalna weryfikacja wyjść VLA (conformal prediction, runtime monitors) dla środowisk krytycznych

Hardware acceleration

Dedykowane NPU dla transformerów (Qualcomm AI 100 Ultra, NVIDIA Thor); latencja <50 ms dla 7B on-device

PODSUMOWANIE

- 01 **VLM** (CLIP, LLaVA, GPT-4V) wnoszą **rozumienie języka naturalnego** i **semantyki sceny** do systemu nawigacji.
- 02 VLA (RT-2, OpenVLA, $\pi 0$) **zamykają pętlę**: obserwacja wizualna + polecenie językowe → akcja robota.
- 03 **Grounding semantyczny** pozwala nawigować do celu opisanego słowami bez uprzedniego uczenia mapy etykiet.
- 04 **Latencja VLM** (>100 ms) wymaga buforowania i asynchronicznego wywoływania względem pętli sterowania.
- 05 **Weryfikacja bezpieczeństwa** i sanityzacja wyjść VLA są obowiązkowe przed przekazaniem do WBC i aktuatorów.

Zadanie projektowe i pytania problemowe

1. Dlaczego pętla sterowania (1 kHz) i pętla VLM (2-5 Hz) muszą być rozdzielone? Narysuj schemat asynchronicznej architektury.
2. Jakie zagrożenia niesie prompt injection dla robota sterowanego VLM w środowisku z ludźmi? Zaproponuj mechanizm obrony.

Zadanie: zaimplementuj klasę SemanticMap (patrz Kod 3), zbierz embeddingi dla 10 pozycji w symulowanym środowisku (dowolny zbiór zdjęć) i sprawdź, które zapytania języka naturalnego są poprawnie rozwiązywane, a które nie. Porównaj OpenVLA i klasyczny sterownik PID dla zadania "idź 2 m do przodu": jakie są różnice w latencji, niezawodności i możliwości generalizacji?

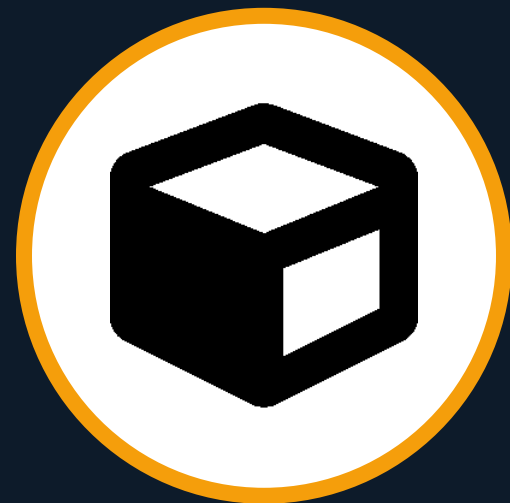
Bonus: skonfiguruj pipeline LLaVA-1.6 (3B) + Nav2 w symulatorze Gazebo i zademonstruj nawigację do celu podanego opisem słownym.

NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 6

World-Model-Action (WMA)
w nawigacji humanoida

dr inż. Mateusz Pomianek



Model świata

Planowanie

DreamerV3/TDMPC

ROS 2 + WMA

Plan wykładu

Sekcja 1

Intuicja i architektura WMA: model świata, stan latentny, planowanie w wyobraźni, RSSM

Sekcja 2

Algorytmy i implementacje: DreamerV3, TDMPC2, JEPA; jak działają krok po kroku

Sekcja 3

WMA w nawigacji humanoida: integracja z ROS 2, planista wielopoziomowy, trening i wdrożenie

Kody Python

RSSM, planowanie w wyobraźni, trening DreamerV3, integracja z Nav2

01

SEKCJA

Czym jest WMA i model świata

*Intuicja, architektura, stan latentny i planowanie w
wyobraźni*



Intuicja: jak człowiek planuje ruch

1. Zanim wstaniesz z krzesła i przejdziesz do kuchni, wyobrażasz sobie tę trasę w głowie
2. Twój mózg posiada wewnętrzny model mieszkania i przewiduje, co zobaczysz za rogiem
3. Nie testujesz każdego kroku fizycznie; symulacja w głowie jest tańsza i bezpieczniejsza
4. Jeśli wyobrażona ścieżka prowadzi do kolizji, wybierasz inną ZANIM zaczniesz iść

To właśnie robi World Model: uczy się kompaktowej reprezentacji (stanu latentnego) i symuluje przyszłość

WMA to robot z wbudowaną wyobraźnią: planuje w przestrzeni latentnej, nie na fizycznym robocie

Korzyść: można przetestować tysiące sekwencji akcji w milisekundach bez ryzyka uszkodzenia sprzętu

Trzy podejścia do sterowania robotem

Reaktywne (Reflex)

- obs $\rightarrow$ action natychmiast
- Szybkie, proste (PID, DWA)
- Brak planowania do przodu
- Nie uczy się środowiska
- Wrażliwe na zmiany otoczenia
- Przykład: regulator P dla yaw

Model-free RL

- Polityka uczona metodą prób
- Miliony rollout-ów na robocie
- Brak jawnego modelu środowiska
- Dobra wydajność końcowa
- Drogie: czas + zużycie sprzętu
- Przykład: PPO, SAC, TD3

WMA (Model-based)

- Model świata: $(s_t, a_t) \rightarrow s_{t+1}$
- Planowanie w wyobraźni (latent)
- 10-100× mniej interakcji z robotem
- Generalizacja z modelu na nowe sceny
- Wymaga dobrego modelu przejścia
- Przykład: DreamerV3, TDMPC2

Definicja i składowe architektury WMA

- **WMA (World-Model-Action)** = system sterowania zbudowany wokół wyuczonego modelu dynamiki środowiska
- **Komponent 1 - Enkoder (encoder):** obserwacja $o_t \rightarrow$ stan latentny z_t (kompresja do niskowymiarowej reprezentacji)
- **Komponent 2 - Model przejścia (transition model):** $(z_t, a_t) \rightarrow z_{t+1}$ (przewiduje następny stan w przestrzeni latentnej)
- **Komponent 3 - Model nagrody/wartości (reward/value head):** $z_t \rightarrow r_t$ lub $V(z_t)$ (ocena jakości stanu)
- **Komponent 4 - Dekoder (decoder, opcjonalny):** $z_t \rightarrow \hat{o}_t$ (rekonstrukcja obserwacji; służy do uczenia enkodera)
- **Komponent 5 - Planista/polityka (policy):** planuje sekwencję akcji maksymalizującą wartość w przestrzeni latentnej
- **Trening:** minimalizacja błędu rekonstrukcji + błędu predykcji przejścia + maksymalizacja nagrody (ELBO / RL)

Architektura WMA - od obserwacji do akcji

Obserwacja o_t

Obraz RGB, chmura LiDAR, propriocepcja (q, dq, τ) - surowe dane z sensorów robota

Enkoder $E(\cdot)$

CNN lub ViT kompresuje obserwację do stanu latentnego $z_t \in \mathbb{R}^d$ ($d = 512-2048$)

Recurrent State h_t

GRU/LSTM przechowuje historię; $h_t = f(h_{t-1}, z_t)$; modeluje zależności czasowe

Model przejścia $T(\cdot)$

MLP/Transformer: $(h_t, a_t) \rightarrow \hat{z}_{t+1}$; predykcja w wyobraźni bez rzeczywistego kroku

Planista + polityka

MPPI / CEM / actor-critic planuje N kroków do przodu w latent space; wybiera a_t^*

Kluczowe cechy

- Planowanie bez GPU-heavy forward pass
- Latent dim 512: 1000 trajektorii w ~ 5 ms
- Model przejścia: $\sim 2M$ parametrów
- Enkoder: ViT-S $\sim 22M$ lub CNN $\sim 5M$
- Trening: 100k-10M klatek z robota
- Inferencja: ~ 1 ms na GPU embedded

Stan latentny - czym jest i dlaczego działa

Czym jest stan latentny z_t ?

- **Kompresja obserwacji** do wektora o wymiarze d (np. 512)
- **Enkoder** uczy się wybrać z 1920×1080 RGB tylko istotne cechy
- **z_t zawiera**: pozycję obiektów, kształt przejść, kierunek ruchu
- **Analogia**: notatka "drzwi otwarte, 2 m do przodu" zamiast całego obrazu
- **Stochastyczny**: $z_t \sim q(z|o_t)$ - rozkład Gaussa; próbkowanie
- **Deterministyczny**: $h_t = \text{GRU}(h_{t-1}, z_t)$ - pamięć sekwencji

Dlaczego planowanie w latent space działa?

- **Model przejścia T**: $(z_t, a_t) \rightarrow z_{t+1}$ jest gładki i ciągły
- **Gradient propaguje się** przez wirtualne kroki $\rightarrow$ uczenie polityki
- 1000 trajektorii w latent space = 1000 wyobrażeń w ~ 2 ms
- **Modele przejścia generalizują**: jeśli korytarz podobny $\rightarrow$ działa
- **Błędy kumulują się** po ~ 10 -20 krokach; stąd krótki horyzont planowania
- **Re-planowanie co krok**: plan 1 kroku, wykonaj, planuj od nowa

RSSM: Recurrent State Space Model (serce WMA)

- **RSSM** to podstawowy blok DreamerV3; łączy deterministyczny i stochastyczny model stanu
- **Deterministyczna część:** $h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1})$ – GRU; zapewnia ciągłość historii
- **Stochastyczna część:** $z_t \sim q_\phi(z|h_t, o_t)$ - enkoder posterior; $z_t \sim p_\theta(z|h_t)$ - prior
- **Predykacja bez obserwacji (wyobrażenia):** zaczynając od (h_t, z_t) propaguj TYLKO prior bez nowych o_t
- **Funkcja straty:** $L = L_{recon} + \beta \cdot KL(posterior || prior) + L_{reward} + L_{continue}$
- **KL-balancing:** gradient przez prior i posterior osobno skalowany ($\beta = 0,1 \rightarrow 0,5$ dla prior)
- **Wynik:** model umie jednocześnie OPISYWAĆ teraźniejszość i WYOBRAŻAĆ sobie przyszłość

Planowanie w wyobraźni - krok po kroku

1

Kodowanie bieżącej obserwacji

Enkoder E pobiera klatkę o_t i tworzy z_t ; GRU aktualizuje h_t ; razem = bieżący stan świata

2

Rozwinięcie wyobraźni

Polityka $\pi(a|h,z)$ proponuje akcje; model $T(h,z,a) \rightarrow (h',z')$ propaguje stan TYLKO w latent space przez H kroków ($H=15$)

3

Ocena trajektorii

Model wartości $V(h,z)$ i nagrody $r(h,z)$ oceniają każdy wirtualny krok; λ -return sumuje zdyskontowane nagrody

4

Aktualizacja polityki

Gradient z V i r propaguje się wstecz przez wirtualne kroki aż do π ; actor-critic update bez fizycznego robota

RSSM: implementacja rdzenia modelu świata (PyTorch)

```
import torch, torch.nn as nn

class RSSM(nn.Module):
    def __init__(self, obs_dim=512, act_dim=7,
                 det_dim=512, stoch_dim=32, stoch_classes=32):
        super().__init__()
        # Deterministyczna czesc (GRU)
        self.gru = nn.GRUCell(stoch_dim*stoch_classes + act_dim, det_dim)
        # Prior p(z | h) - bez obserwacji
        self.prior = nn.Linear(det_dim, stoch_dim * stoch_classes)
        # Posterior q(z | h, obs) - z obserwacja
        self.posterior = nn.Linear(det_dim + obs_dim, stoch_dim * stoch_classes)
        self.stoch_dim, self.stoch_classes = stoch_dim, stoch_classes

    def imagine_step(self, h, z, action):
        """Krok wyobrazni: (h,z,a) -> (h_next,z_next) BEZ obserwacji."""
        z_flat = z.view(z.shape[0], -1) # (B, stoch*classes)
        h_next = self.gru(torch.cat([z_flat, action], -1), h)
        logits = self.prior(h_next) # prior p(z|h)
        z_next = self._gumbel(logits) # Gumbel-softmax
        return h_next, z_next

    def _gumbel(self, l):
        l = l.view(-1, self.stoch_dim, self.stoch_classes)
        return nn.functional.gumbel_softmax(l, tau=1.0, hard=True)
```

Opis: `imagine_step()` propaguje stan w latent space; nie potrzebuje obserwacji; planowanie w wyobraźni

02

SEKCJA

Algorytmy WMA - DreamerV3, TDMPC2, JEPA

*Jak działają krok po kroku; różnice i zastosowania w
lokomocji*



DreamerV3 - architektura i cykl uczenia

Zbieranie danych

Polityka π wykonuje akcje na robocie; (o, a, r, done) zapisywane do replay buffer

Uczenie modelu świata

Mini-batch z replay $\rightarrow$ RSSM; minimalizacja $L_{\text{recon}} + KL + L_{\text{reward}} + L_{\text{continue}}$

Wyobrażenia (Imagine)

Z replay buffer inicjuj stany; rozwiń $H=15$ kroków wyłącznie modelem T; bez robota

Uczenie aktora-krytyka

Actor: max λ -return przez gradienty z wyobraźni; Critic: regresja na λ -return

Wykonanie na robocie

Polityka $\pi(a|h, z)$ w latent space; akcja dekodowana i wysyłana do ros2_control / WBC

Kluczowe cechy

- Symultaniczny trening: model + policy
- Jeden GPU (RTX 3090): ~10h training
- Działa z obrazem RGB lub proprio
- DreamerV3: zunifikowane hiperparametry dla wszystkich środowisk
- Atari, DMControl, HumanoidWalk: SoTA
- Open-source: github.com/danijar/dreamerv3

TDMPC2: Temporal Difference Model Predictive Control

- **TDMPC2** łączy model świata z optymalizacją trajektorii w stylu MPC (Model Predictive Control)
- **Latent world model**: enkoder E + model przejścia T + głowica nagrody R + value function V (wszystkie MLP)
- **Planowanie przez MPPI (Model Predictive Path Integral)**: próbuj N trajektorii w latent space, weź najlepszą
- **Temporal consistency loss**: predykcja H kroków do przodu; gradienty przez cały horyzont
- **Kluczowa zaleta**: bardzo małe modele (~0,5M-5M parametrów) i szybka inferencja (~0,5 ms/krok)
- **Benchmark**: SoTA na Humanoid-9DoF (DMControl); chód, bieg, backflip w ciągłej przestrzeni akcji
- **Open-source**: github.com/nicklashansen/tdmpc2; gotowy interfejs dla robotów z PyTorch

JEPA: Joint Embedding Predictive Architecture

Idea JEPA (LeCun, Meta AI)

- **Predykcja** w przestrzeni reprezentacji, nie pikseli
- **Context encoder**: koduje widoczny fragment sceny
- **Target encoder**: koduje ukryty/przyszły fragment
- **Predictor**: próbuje przewidzieć target z context
- **Brak dekodera**: nie rekonstruuje surowych pikseli
- **Zaleta**: uczy się abstrakcyjnych cech, a nie tekstur
- **Szybszy trening**: bez kosztownej rekonstrukcji RGB

JEPA w nawigacji robota

- **I-JEPA (obraz)**: pre-training enkodera ViT bez etykiet
- **V-JEPA (wideo)**: predykcja przyszłych klatek wideo
- **Używane jako pre-trained backbone** dla RSSM lub TDMPC2
- Robot widzi lewą połowę korytarza → przewiduje prawą
- **Semantyczne reprezentacje** lepsze niż px-rekonstrukcja
- **Integracja**: JEPA encoder → latent z_t → RSSM/TDMPC2

Porównanie algorytmów WMA dla humanoida

DreamerV3

RSSM + actor-critic; imagen. H=15;
SoTA na humanoid locomotion; pełny
kod open-source

TDMPC2

MLP world model + MPPI; 0,5M
param; szybki; idealny na Jetson; SoTA
DMControl Humanoid

IRIS

World model jako autoregressive
transformer (jak GPT); tokeny stanu;
silny w rzadkich nagrodach

DayDreamer (robot)

Adaptacja Dreamer do real robotów;
trening na żywym robocie bez resetów;
prawie zero-shot transfer

HarmonyDream

WMA dla humanoid navigation:
hierarchia Dreamer (wysoki) + TDMPC2
(niski); wyniki ICRA 2024

SWIM (MuJoCo)

Scalable World-model for Imagination;
3D voxel world model; planowanie 3D
dla humanoida na schodach

Uczenie WMA - dane, trening i hiperparametry

- Dane wejściowe: sekwencje $(o_1, a_1, r_1, o_2, a_2, \dots)$ z eksploracyjnej polityki lub teleoperation
- Rozmiar replay buffer: typowo 1M-10M kroków; priorytetowe próbkowanie (PER) poprawia próbkowanie rzadkich zdarzeń
- Batch size: 16 sekwencji $\times$ 64 kroki = 1024 kroków na batch; częstość uczenia: 1 krok uczenia / 1 krok środowiska
- Hiperparametry DreamerV3 zunifikowane: $lr=1e-4$, $\gamma=0,997$, $\lambda=0,95$, $\beta=1,0$; działają bez strojenia między środowiskami
- Curriculum: zacznij od krótkich trajektorii i prostych środowisk; stopniowo zwiększaj horyzont i złożoność
- Augmentacja danych: random crop, color jitter, losowe wycinki czasowe - podwaja efektywność danych z robota
- Metryki treningu: błąd rekonstrukcji, KL divergence, episode return podczas walidacji w symulacji

Planowanie w wyobraźni: horyzont H kroków

```
import torch

def imagine_rollout(rssm, actor, h0, z0, horizon=15, n_traj=512):
    """Rozija N trajektorii w przestrzeni latentnej przez H kroków."""
    B = n_traj
    # Inicjalizacja: powiel stan początkowy dla wszystkich trajektorii
    h = h0.unsqueeze(0).expand(B, -1) # (B, det_dim)
    z = z0.unsqueeze(0).expand(B, -1, -1) # (B, stoch_dim, classes)
    states_h, states_z, actions = [h], [z], []
    for _ in range(horizon):
        # Polityka wybiera akcje z bieżącego stanu latentnego
        z_flat = z.view(B, -1)
        a_dist = actor(torch.cat([h, z_flat], dim=-1))
        a = a_dist.rsample() # różniczkowane próbkowanie
        # Krok wyobraźni (bez robota!)
        h, z = rssm.imagine_step(h, z, a)
        states_h.append(h); states_z.append(z); actions.append(a)
    # Zwróć stosy tensorów: (H+1, B, dim)
    return torch.stack(states_h), torch.stack(states_z), torch.stack(actions)
```

Opis: $n_traj=512$ trajektorii $\times$ $H=15$ kroków: cały rollout zajmuje ~ 2 ms na GPU; gradients flow przez wszystkie kroki

Szkielet treningu DreamerV3 (pętla treningowa)

```
# Zakładamy: rssm, decoder, reward_head, value_fn, actor, critic
# replay_buffer przechowuje sekwencje (o,a,r,done)

for step in range(total_steps):
    # --- 1. Krok na robocie ---
    obs = robot.get_observation()          # RGB + proprio
    z, h = rssm.encode_and_update(obs)     # posterior + GRU
    action = actor(h, z).sample().clamp(-1, 1)
    reward, next_obs, done = robot.step(action)
    replay_buffer.add(obs, action, reward, done)

    # --- 2. Uczenie modelu świata ---
    batch = replay_buffer.sample(batch_size=16, seq_len=64)
    loss_wm = world_model_loss(rssm, decoder, reward_head, batch)
    loss_wm.backward(); opt_wm.step(); opt_wm.zero_grad()

    # --- 3. Uczenie aktora-krytyka w wyobraźni ---
    h0, z0 = rssm.get_current_state()
    hs, zs, acts = imagine_rollout(rssm, actor, h0, z0)
    returns = compute_lambda_returns(reward_head, value_fn, hs, zs)
    loss_ac = actor_critic_loss(actor, critic, hs, zs, acts, returns)
    loss_ac.backward(); opt_ac.step(); opt_ac.zero_grad()
```

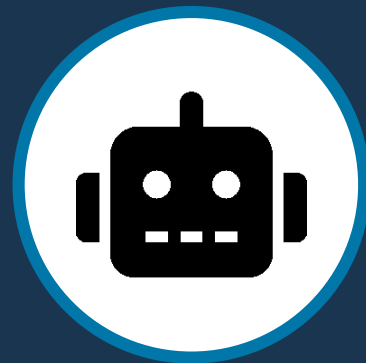
Opis: Trzy fazy w każdym kroku: rollout na robocie → ucz model świata → ucz actor-critic w wyobraźni

03

SEKCJA

WMA w nawigacji humanoida

*Integracja z ROS 2, hierarchia planowania, trening i
wdrożenie*



Hierarchiczna architektura nawigacji z WMA

Poziom misji (LLM/VLM)

Polecenie językowe → sekwencja celów nawigacyjnych; wywołanie co ~2 s

Poziom globalny (A* + SLAM)

Mapa zajętości, ścieżka metryczna do celu; aktualizacja co 0,5-2 s; klasyczny planista

Poziom lokalny: WMA (TDMPC2)

World model przewiduje $H=10$ kroków; MPPI wybiera optymalną (v, ω); 20-50 Hz

Poziom chodu: WBC + RSSM

WMA dla lokomocji: planuje sekwencję kroków w latent space; 200 Hz WBC + 10 Hz WMA

Aktuatory (ros2_control)

Komendy momentów τ [Nm]; pętle PID 1 kHz; enkodery i F/T → sprzężenie zwrotne do WMA

Kluczowe cechy

- Każdy poziom ma własną częstość i horyzont
- Błędy WMA nie blokują WBC (fallback)
- Wspólny stan latentny h między poziomami
- Można zastąpić pojedynczy poziom
- Testowany: DreamerV3 + Nav2 w Gazebo
- **Czas latencji:** WMA ≈ 5 ms / krok

Etapy trenowania WMA dla nawigacji humanoida

1

Pre-training w symulacji

MuJoCo / Isaac Sim; 10M kroków; polityka eksploracyjna; uczenie RSSM i enkodera na danych proprio + RGB

2

Fine-tuning w symulacji

Zadanie nawigacyjne: dotarcie do celu $\pm 0,1$ m; funkcja nagrody: $-d_{\text{goal}}$ + bonus_sukces - kolizja; DreamerV3 / TDMPC2

3

Sim-to-real transfer

Domain randomization: masa, tarcie, opóźnienia, tekstury; tylko enkoder re-fine-tunowany na 1h danych z realnego robota

4

Online adaptation

Na robocie: replay buffer zasilany danymi online; model przejścia update co 100 kroków; polityka adaptuje się w locie

Projektowanie funkcji nagrody dla nawigacji WMA

- Nagroda za zbliżenie do celu: $r_{goal} = -\frac{d(post, goal)}{d_{initial}}$; znormalizowana, gęsta
- Kara za kolizję: $r_{coll} = -10$ gdy $dist(przeszkoda) < 0,3$ m; epizod kończy się
- Nagroda za stabilność ZMP: $r_{zmp} = +0,2$ gdy ZMP w wieloboku; $-0,5$ gdy poza marginesem 2 cm
- Kara za zużycie energii: $r_{energy} = -0,001 \cdot \sum |\tau_i \cdot dq_i|$; preferuje płynny chód
- Nagroda za prędkość: $r_{vel} = +0,1 \cdot v_{forward}$; zachęca do sprawnego ruchu ku celowi
- Nagroda za sukces: $r_{success} = +50$ gdy $|pos - goal| < 0,1$ m; jednorazowy bonus
- Balans: $r_{total} = r_{goal} + r_{zmp} + r_{energy} + r_{vel} + r_{coll} + r_{success}$; wagi dobierane eksperymentalnie

Integracja WMA z ROS 2 - wzorzec implementacji

- **Node /wma_policy:** subskrybuje /camera/image_raw + /odom + /joint_states; publikuje /cmd_vel lub /footstep_goal
- **Cykl działania:** encode(obs) → rsm.imagine(H=10) → mppi_select_action() → publish
- **Latencja:** TDMPC2 ~3 ms, DreamerV3 ~8 ms na Jetson AGX Orin; mieści się w cyklu 50 Hz
- **Interfejs z Nav2:** /wma_policy jako custom local planner plugin (nav2_core::Controller interface)
- **TF2:** node czyta transformacje map→odom→base; stan latentny enkoduje RELATYWNA pozycję celu
- **Watchdog:** jeśli WMA nie odpowie przez >50 ms → Nav2 fallback do DWB controller
- **Diagnostics:** /wma_diagnostics publikuje latencję, wartość funkcji wartości $V(h,z)$, błąd predykcji

WMA jako kontroler lokalny w Nav2 (Python szkielek)

```
import rclpy; from rclpy.node import Node
import torch, numpy as np
from geometry_msgs.msg import Twist

class WMAController(Node):
    def __init__(self):
        super().__init__('wma_controller')
        self.rssm = torch.load('rssm_humanoid.pt').eval().cuda()
        self.actor = torch.load('actor_nav.pt').eval().cuda()
        self.h = torch.zeros(1, 512).cuda() # stan deterministyczny GRU
        self.z = torch.zeros(1, 32, 32).cuda() # stan stochastyczny
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 1)
        self.obs_sub = self.create_subscription(
            CompressedImage, '/camera/compressed', self.obs_cb, 1)
    def obs_cb(self, msg):
        obs = preprocess_image(msg) # (1, 3, 64, 64) tensor
        with torch.no_grad():
            self.h, self.z = self.rssm.observe(obs, self.h, self.z)
            action = self.actor(self.h, self.z.view(1,-1)).mean
        cmd = Twist()
        cmd.linear.x = float(action[0].clamp(-0.5, 0.5))
        cmd.angular.z = float(action[1].clamp(-1.0, 1.0))
        self.cmd_pub.publish(cmd)
```

Opis: observe() aktualizuje (h,z) z nowej klatki; actor wybiera akcję z latent state; cmd_vel do Nav2 costmap

WMA vs. VLM vs. klasyczny planista -> kiedy stosować?

Klasyczny planer (A* + DWA)

- **Deterministyczny** i formalnie poprawny
- **Wymaga gotowej mapy** środowiska
- **Cel = współrzędne** (x, y) w mapie
- **Brak uczenia**: brak adaptacji online
- **Szybki**: A* ~5 ms, DWA ~2 ms
- **Najlepszy**: znane, statyczne środowisko

VLM (CLIP, LLaVA)

- **Rozumie polecenia** językowe i semantykę
- **Zero-shot**: nie wymaga mapy etykiet
- **Wolny**: 100-800 ms / zapytanie
- **Nie planuje fizyki** - tylko co i gdzie
- **Najlepszy**: grounding, instruction follow
- **Łączy się z A*** jako oracle celu

WMA (DreamerV3, TDMPC2)

- **Uczy się fizyki środowiska** z doświadczeń
- **Szybkie planowanie**: ~3-8 ms / krok
- Generalizuje: nowe ułożenia mebli OK
- **Wymaga treningu** (~100k kroków sim.)
- **Najlepszy**: dynamiczna lokomocja, przeszkody
- **Łączy się z VLM** (cel) + A* (trasa globalna)

Ograniczenia i otwarte wyzwania WMA

Problemy techniczne

- **Błąd compounding:** model przejścia po 20+ krokach wyobraźni kumuluje błędy predykcji
- **Trudność z rzadkimi zdarzeniami:** upadek, kolizja; replay buffer rzadko je zawiera
- **Kosztowne uczenie:** 10M kroków simul. $\times$ 5 ms/krok $\approx$ 14h; realny robot = dni
- **Sim-to-real gap:** model przejścia nie uwzględnia elastyczności struktury, tarcia statycznego
- **Rozkład wielomodalny:** kilka równie dobrych ścieżek; RSSM z rozk. Gaussa słabo je modeluje

Kierunki badań i rozwiązania

- **Dłuższy efektywny horyzont:** hierarchiczne WMA (krótki + długi horyzont)
- **Aktywne zbieranie danych:** polityka eksploracyjna celowo szuka rzadkich zdarzeń (curiosity)
- **Real-world fine-tuning:** 1h danych z robota + pre-trained sim model $\rightarrow$ sim-to-real w $\sim$ 30 min
- **Normalizing flows / diffusion:** lepsze modelowanie wielomodalności niż Gauss
- **Foundation World Models:** duży model pre-trained na danych z wielu robotów (Open-X-Embodiment)

Use case: WMA dla humanoida patrolującego budynek

- **Platforma:** Unitree H1 / Agility Digit; symulacja: MuJoCo Menagerie z MJCF modelem robota
- **Architektura:** LLM task planner → A* global path → TDMPC2 local navigation → WBC footstep
- **Trening TDMPC2:** 5M kroków symulacji (≈ 7 h na RTX 4090); nagroda: $r_{\text{goal}} + r_{\text{zmp}} + r_{\text{energy}}$
- **Dane realne:** 45 min teleoperation w budynku → fine-tuning enkodera (tylko E); reszta zamrożona
- **Wyniki walidacji:** success rate 92% (vs. 71% DWA) na trasach z dynamicznymi przeszkodami
- **Zużycie energii:** -18% względem DWA; WMA wybiera płynniejsze trajektorie z mniejszymi momentami
- **Czas odpowiedzi:** TDMPC2 ~ 4 ms/krok na Jetson Orin; A* co 1 s; LLM co 5 s (asynchronicznie)

PODSUMOWANIE

01

Model świata uczy się przewidywać kolejne stany środowiska z obserwacji i akcji; planowanie odbywa się w przestrzeni latentnej.

02

WMA = enkoder + model przejścia + model nagrody/wartości + planista + polityka; każdy komponent jest uczony przez RL lub imitation learning.

03

DreamerV3 i TDMPC2 to referencyjne implementacje gotowe do adaptacji na dane robotyczne z lokomocją humanoidalną.

04

Planowanie w wyobraźni eliminuje kosztowne rollout-y na robocie i przyspiesza uczenie o 10-100×.

05

WMA, VLM i klasyczny planer A* są komplementarne i mogą działać w jednej hierarchicznej architekturze nawigacji.

Zadanie projektowe i pytania problemowe

1. Wyjaśnij własnymi słowami różnicę między posterior $q(z|h,o)$, a prior $p(z|h)$ w RSSM. Dlaczego w fazie "wyobraźni" używamy prioru, a nie posterioru?
2. Dlaczego planowanie w przestrzeni latentnej jest szybsze niż planowanie z surowymi obserwacjami? Podaj konkretne liczby (wymiarowość, liczba operacji).
3. Jakie hiperparametry DreamerV3 należałoby dostosować, gdybyś trenował robot do nawigacji w ciemnym korytarzu z kamerą termowizyjną zamiast RGB?

Zadanie projektowe: zaimplementuj uproszczony RSSM (enkoder 2-warstwy MLP, GRU 256, prior 2-warstwy MLP) i wytrenuj go na danych z symulowanego wahadła odwróconego (CartPole). Sprawdź, czy model potrafi wyobrazić sobie 10-krokową trajektorię startując z losowego stanu.

Bonus: połącz TDMPC2 z Nav2 w Gazebo dla robota kołowego (TurtleBot3) i porównaj success rate z DWB na mapie z 5 dynamicznymi przeszkodami.

NAWIGACJA ROBOTOW HUMANOIDALNYCH

WYKLAD 7

Nawigacja społeczna:

tlumy, normy kulturowe, interakcja z czlowiekiem

dr inż. Mateusz Pomianek



Modele tlumu

Proksemika

Normy kulturowe

HRI / personalizacja

Plan wykładu

Sekcja 1

Modele ruchu tłumu: Social Force Model, ORCA, modele oparte na danych, predykcja trajektorii

Sekcja 2

Proksemika i normy kulturowe: strefy Halla, różnice kultur, kontekst sytuacyjny, sygnały społeczne

Sekcja 3

Personalizacja interakcji człowiek-robot: profil użytkownika, uczenie preferencji, komunikacja, ewaluacja

Kody Python

Social Force Model, ORCA, predykcja Transformer, klasyfikator profilu, mapa społecznych kosztów

01

SEKCJA

Modele ruchu tłumu

Social Force Model, ORCA, modele predykcji trajektorii, detekcja grup



Dlaczego nawigacja społeczna jest potrzebna?

- Robot poruszający się tylko według geometrii ($A^* + DWA$) jest postrzegany jako nieprzewidywalny i nieprzyjemny
- **Badania (Kruse et al., 2013):** 70% użytkowników odczuwa dyskomfort, gdy robot nie respektuje nienawigacyjnych norm społecznych
- **Normy społeczne:** nie wchodzić między rozmawiające osoby, nie scigać pieszych, utrzymywać właściwy dystans
- **Nawigacja społeczna** rozszerza klasyczne planowanie o model agentów: każdy człowiek ma intencje, prędkość i strefę prywatności
- **Standard ISO 13482:2014:** roboty opiekuncze muszą spełniać minimalne wymagania dotyczące bezpiecznej interakcji z człowiekiem
- **Cel:** robot postrzegany jako naturalny uczestnik przestrzeni społecznej, a nie przeszkoda lub zagrożenie

Social Force Model (SFM) - Helbing i Molnar (1995)

- **Idea:** ruch pieszego modelowany jako suma sil: predkosc docelowa + odpychanie od przeszkod + odpychanie od innych ludzi
- **Sila własna:** $F_i^{\text{self}} = (v_i^0 * e_i^0 - v_i) / \tau_i$ (powrot do pozadanej predkosci i kierunku)
- **Sila społeczna** (odpychanie): $F_{ij} = A * \exp((r_{ij} - d_{ij})/B) * n_{ij}$ (im bliżej, tym silniej)
- **Sila od przeszkod:** analogiczna do społecznej; gradient prostopadly do powierzchni sciany
- **Wynikowa:** $m_i * (dv_i/dt) = F_i^{\text{self}} + \sum(F_{ij}) + \sum(F_{iW})$
- **Zalety:** prosta implementacja, przewidywalne zachowanie, dziala w czasie rzeczywistym (~1 ms / agent)
- **Wady:** kalibracja parametrow (A, B, tau) wymaga danych pomiarowych; nie modeluje intencji dlugookresowych

Social Force Model - implementacja dla planisty lokalnego



```
import numpy as np
```

```
def social_force(robot_pos, robot_vel, goal_pos, pedestrians,
                 A=2.0, B=0.3, tau=0.5, v0=1.2):
    """Wyznacza wypadkowa sile spoleczna dla robota."""
    e0 = goal_pos - robot_pos
    e0 /= (np.linalg.norm(e0) + 1e-6)          # kierunek do celu
    # Sila wlasna: robot chce isc do celu z predkoscia v0
    f_self = (v0 * e0 - robot_vel) / tau
    # Sily odpychajace od pieszych
    f_social = np.zeros(2)
    for ped in pedestrians:
        diff = robot_pos - ped['pos']          # wektor od pieszego do robota
        dist = np.linalg.norm(diff)
        r_ij = 0.3 + ped.get('radius', 0.25)   # suma promieni
        n_ij = diff / (dist + 1e-6)             # wektor normalny
        f_social += A * np.exp(-(r_ij - dist) / B) * n_ij
    return f_self + f_social                    # f_w / e^22
```

Opis: Wywolywac co 50 ms; scalkuj sile przez dt aby uzyskac nowa predkosc; clampuj do v_{max} .

ORCA - Optimal Reciprocal Collision Avoidance

Idea ORCA (van den Berg et al., 2011)

- Każdy agent przyjmuje połowę odpowiedzialności za uniknięcie kolizji
- **Dla każdej pary (I, J):** wyznaczana jest half-plane bezpiecznych prędkości
- **Nowa prędkość agenta I:** najbliższy punkt do prędkości preferencji w zbiorze bezpiecznych prędkości
- **Warunek:** przy wzajemnym stosowaniu ORCA przez wszystkich agentów brak kolizji jest gwarantowany
- **Złożoność:** $O(N)$ per agent per krok; praktyczne dla $n < 500$ agentów
- Popularny w symulatorach: CrowdSim, MENGE, PedSim

ORCA dla robota wpośród pieszych

- Robot zna swoją prędkość preferencji V_{PREF} (z DWA lub WMA)
- Dla każdego wykrytego pieszego wyznacza half-plane ORCA
- **Wynikowa prędkość:** projekcja V_{PREF} na przecięcie half-planes
- **Pieszy nie stosuje ORCA:** robot bierze pełną odpowiedzialność
- **Rozszerzenie:** RVO2 library (C++/Python); działa w czasie rzeczywistym
- **Ograniczenie:** nie przewiduje intencji; zakreśla ruch stały

Predykcja trajektorii pieszych

Modele kinematyczne

- **Stałość prędkości (CV):** $x_{t+1} = x_t + v \cdot dt$
- **Stałość przyspieszenia (CA):** uwzględnia inercję
- **Filtr Kalmana:** estymacja + predykcja
- Prosta, szybka, działa krótkoterminowo
- Zawodzi przy zmianie kierunku lub zatrzymaniu
- **Horyzont:** do ~ 1 s

Social Force / ORCA

- Model interakcji między agentami
- **Predykcja**, z uwzględnieniem dynamiki tłumu
- Wymaga pozycji wszystkich pieszych
- Średnio skuteczny na krótkim horyzoncie
- Może propagować błędy siły społecznej
- **Horyzont:** 1–3 s

Modele danych (DL)

- Social LSTM / Social GAN (Alahi 2016)
- Trajectron++ (Salzmann 2021)
- **Transformer:** AgentFormer, Wayformer
- Uwzględniają kontekst i interakcje
- Wymagają treningu na danych (UCY, ETH)
- **Horyzont:** 3–5 s; SoTA na benchmarkach

Detekcja grup i zachowan zbiorowych

- Ludzie czesto poruszaja sie w grupach (dyady, triady, rodziny); robot nie moze wejsc miedzy czonkow grupy
- **Detekcja grupy:** clustering pieszych wedlug wspolnej predkosci i bliskosci; DBSCAN lub hierarchical clustering
- **Formacja grupy:** trojkat, linia, V-ksztalt; kazda ma strefe chroniona miedzy czlonkami (O-space)
- **O-space (Kendon 1990):** przestrzeń miedzy członkami grupy; robot musi ją ominąć, nie przejść na wprost
- **F-formation detection:** detekcja wspolnej orientacji i pola widzenia; robot planuje objazd z zewnatrz
- **Zastosowanie:** robot chcacy podejsc do osoby czeka na rozejscie sie grupy lub sygnalizuje chec dolaczenia
- **Implementacja:** Social NavMesh uwzgledniajacy O-spaces jako dynamiczne przeszkody kosztowe w cost map

Spoleczna mapa kosztow i integracja z Nav2



```
import numpy as np

def build_social_costmap(grid_size, resolution, pedestrians,
                        sigma_pos=0.8, sigma_vel=0.5) -> np.ndarray:
    """Buduje mape kosztow uwzgledniajaca pieszych (Gaussowy potencjal)."""
    cost = np.zeros((grid_size, grid_size), dtype=np.float32)
    cx = cy = grid_size // 2 # srodek mapy = pozycja robota
    for p in pedestrians:
        px = int(cx + p['x'] / resolution)
        py = int(cy + p['y'] / resolution)
        vx, vy = p.get('vx', 0.0), p.get('vy', 0.0)
        for gy in range(grid_size):
            for gx in range(grid_size):
                dx, dy = gx - px, gy - py
                # Wydłużony Gauss w kierunku ruchu (anizotropowy)
                fwd = dx*vx + dy*vy + 1e-6 # składowa w przód
                d2 = dx**2/sigma_pos**2 + dy**2/sigma_pos**2
                d2 = max(fwd, 0)**2 / (sigma_vel**2)
```

Opis: Anizotropowy Gauss: większy koszt z przodu pieszego (w kierunku ruchu) niż z boku lub tyłu.

02

SEKCJA

Proksemika i normy kulturowe

*Strefy osobiste, różnice kulturowe, kontekst
sytuacyjny, sygnały społeczne*



Strefy proksemiczne E. Halla - przestrzen osobista czlowieka

Publiczna > 3,6 m

Spoleczna 1,2 – 3,6 m

Osobista 0,45 – 1,2 m

Intymna < 0,45 m



Implikacje dla robota

- Robot powinien utrzymywać dystans społeczny (1,2 m) przy podejściu do nieznanej osoby
- **Wejście w strefę osobistą** wymaga sygnału zaproszenia lub wyraźnej zgody użytkownika
- **Strefa intymna**: tylko w przypadku asystencji medycznej lub dostępu do dozownika leków
- **Nawigacja**: robot omija ludzi z marginesem 0,5 m = strefa bezpieczna powyżej osobistej
- **Monitoring**: czujnik F/T lub detekcja YOLO jako trigger zatrzymania przy nadmiernym zbliżeniu

Roznice kulturowe w przestrzeni osobistej

- **Kultury kontaktowe** (Ameryka Południowa, Bliski Wschód, Europa Południowa): dystans konwersacyjny 0,5–0,9 m
- **Kultury zdystansowane** (Europa Północna, Japonia, USA): dystans konwersacyjny 0,9–1,5 m
- **Kultury Azji Wschodniej**: wyjątkowo duży dystans w kontekście formalnym; mały dystans w zatłoczonej przestrzeni publicznej
- **Robot współpracujący globalnie** musi adaptować parametry stref do kontekstu kulturowego użytkownika
- **Dane badawcze**: Hall (1966), Beaulieu (2004), meta-analiza Sorokowska et al. (2017) na 42 krajach
- **Implementacja**: baza danych norm kulturowych + modul detekcji języka / regionu z konta użytkownika
- **Ostrożność**: stereotypizacja kulturowa jest ryzykowna; preferencje indywidualne mają priorytet nad normami grupowymi

Kontekst sytuacyjny a normy nawigacyjne

Srodowiska o wysokiej gestosc (public)

- **Dworce, lotniska, centra handlowe:** $\text{tłum} > 1 \text{ os/m}^2$
- **Akceptowalny kontakt fizyczny** (ramie w ramie), norma kulturowa
- **Robot:** tryb zwarty; wąska trasa, mniejszy dystans bezpieczny (0,4 m)
- **Predkosc zredukowana** do max 0,6 m/s; częste zatrzymania
- **Priorytet:** przepuszczanie grup idących w tym samym kierunku
- **Sygnalizacja:** oświetlenie LED i dźwięk przy zbliżaniu

Srodowiska opieki (szpital, dom opieki)

- **Pacjenci, osoby starsze, dzieci:** wzmocniona strefa bezpieczna (+50%)
- **Predkosc** max 0,4 m/s; płynne przyspieszenia i hamowania
- **Brak nagłych ruchow;** robot nie powinien pojawisc sie od tyłu
- **Zachowanie przewidywalne** i komunikowane z wyprzedzeniem
- **Pierwszenstwo** dla personelu medycznego niezależnie od ścieżki
- **Dźwięk sygnalizacyjny:** małe natężenie, przyjazny ton

Sygnały społeczne - co robot musi rozpoznawac

Gesty i mowa ciała

Wyciągnięta reka = stop; kiwniecie głową = zgoda;
odwrócenie się = koniec interakcji.

Detekcja: MediaPipe Pose

Kontakt wzrokowy

Wzrok skierowany na robota = oczekiwanie interakcji;
brak kontaktu = nie przeszkadzać.

Detekcja: gaze estimation

Mowa i prosodia

Komenda głosowa; ton i tempo sygnalizują stan emocjonalny. ASR + klasyfikator sentymentu w czasie rzeczywistym

Orientacja ciała

Zwrot frontem do robota = zaproszenie;
bok lub tył = brak zainteresowania.

Kluczowe dla decyzji o podejściu

Ekspresja twarzy

Dezorientacja, niezadowolenie, panika.

Detekcja: modele AU (Action Units)
lub end-to-end (AffectNet, FER2013)

Czynniki sytuacyjne

Osoba śmieciąca z tornistrem: zajęta.

Osoba stojąca i patrząca: czeka.

Kontekst wnioskowany z położenia i gestów

Kodowanie norm społecznych w mapie kosztów

- **Warstwa geometryczna:** klasyczne przeszkody statyczne (ściany, meble) z inflacją 0,3 m
- **Warstwa pieszych:** anizotropowy Gauss wokół każdej osoby; większy koszt w kierunku ruchu i z przodu twarzy
- **Warstwa grup:** O-space między członkami grupy kodowany jako strefa zakazana (koszt = 100)
- **Warstwa konwersacji:** obszar między rozmawiającymi osobami; robot nie może go przeciąć
- **Warstwa kulturowa:** globalny mnożnik kosztów ($\times 1.0 - \times 1.5$) zależnie od profilu kulturowego środowiska
- **Warstwa kontekstowa:** szpital: +30% kosztu wszędzie; ruchliwy korytarz biurowy: -10% kosztów bocznych
- **Aktualizacja:** wszystkie warstwy odświeżane co 100 ms na podstawie detekcji pieszych z LiDAR + kamery

03

SEKCJA

Personalizacja interakcji człowiek-robot (HRI)

*Profil użytkownika, uczenie preferencji, komunikacja,
ewaluacja*



Czym jest personalizacja w nawigacji społecznej?

Robot generyczny zachowuje się tak samo wobec każdego



Robot spersonalizowany adaptuje zachowanie do konkretnej osoby

Wymiary personalizacji: dystans fizyczny, styl komunikacji, prędkość ruchu, poziom proaktywności

- **Użytkownik 1:** osoba starsza z trudnościami w chodzeniu; oczekuje wolnego robota i wyraźnych sygnałów
- **Użytkownik 2:** pracownik biura; oczekuje robota który nie przeszkadza i nie blokuje drogi
- **Użytkownik 3:** dziecko; robot powinien być bardziej ostrożny i używać prostszych komunikatów

Cel: minimalizacja zakłóceń i maksymalizacja poczucia komfortu konkretnej osoby, nie średniej użytkowników

Metody: profil explicit (preferencje podane przez użytkownika) + profil implicit (wnioskowany z zachowania)

Profil użytkownika - składowe i źródła

Cechy stale (explicit)

- Wiek i mobilność (poruszanie się z łaseczką, na wółku)
- Preferencje kulturowe (kraj pochodzenia, język)
- Znane fobie lub lęki (np. lęki przed maszynami)
- Preferowany styl komunikacji (werbalny / niewerbalny)
- Historia interakcji z robotami (nowicjusz / doświadczony)
- Rola w organizacji (pacjent, personel, gość)

Cechy dynamiczne (implicit)

- Aktualny stan emocjonalny (ekspresja twarzy, tempo mowy)
- Poziom spieszczenia (prędkość chodzenia, gestowość kontaktu)
- Odpowiedzi na poprzednie zachowania robota (wycofanie, kiwnięcie)
- Zajętość (słucha muzyki, rozmawia przez telefon)
- Pozycja i orientacja ciała względem robota
- Historia dzisiejszej sesji (akceptacje, odrzucenia propozycji)

Uczenie preferencji użytkownika w czasie rzeczywistym

- **Cel:** robot aktywnie aktualizuje model preferencji na podstawie sygnału zwrotnego użytkownika
- **Sygnały zwrotne:** wycofanie się (zbyt blisko), kiwnięcie (OK), komenda słowa (szybciej / wolniej), styl chodzenia
- **Regresja Bayesowska:** model dystansu preferowanego $d^* \sim N(\mu, \sigma^2)$; aktualizacja μ i σ po każdej obserwacji
- **Inverse RL (IRL):** z obserwacji ludzkich zachowań wnioskuj funkcję nagrody; zastosuj do polityki robota
- **Gaussian Process Regression:** interpolacja preferencji dystansu dla różnych kontekstów (korytarz, winda, sala)
- **Prywatność:** dane profilowe przechowywane lokalnie; opcja reset profilu przez użytkownika w dowolnej chwili
- **Cold start:** nowy użytkownik startuje z domyślnymi parametrami kulturowymi; personalizacja po ~5 minutach interakcji

Bayesowska aktualizacja preferencji dystansu (online)

```
import numpy as np

class DistancePreferenceModel:
    """Gaussowski prior nad preferowanym dystansem d* [m]."""
    def __init__(self, mu0=1.2, sigma0=0.4):
        self.mu = mu0 # wartosc oczekiwana poczatkowa
        self.sigma = sigma0 # odchylenie standardowe (niepewnosc)
        self.obs_sigma = 0.15 # szum pomiaru (gestosc obserwacji)

    def update(self, observed_distance: float, feedback: str):
        """Aktualizacja po obserwacji; feedback: "ok"/"too_close"/"too_far"."""
        if feedback == 'too_close':
            z = observed_distance + 0.3 # uzytkownik chce wiekszy dystans
        elif feedback == 'too_far':
            z = observed_distance - 0.2 # uzytkownik akceptuje mniejszy
        else:
            z = observed_distance # OK - to jest d*
        # Kalman update: posterior Gaussian
        K = self.sigma**2 / (self.sigma**2 + self.obs_sigma**2)
        self.mu += K * (z - self.mu)
        self.sigma = np.sqrt((1 - K) * self.sigma**2)
        return self.mu, self.sigma
```

Opis: Po 5–10 interakcjach sigma spada do $\sim 0,05$ m; mu zbiezna do prawdziwej preferencji uzytkownika.

Komunikacja robota z człowiekiem - kanały

Mowa (TTS)

Synteza mowy: przyjazny głos, odpowiednia głośność. Robot informuje o zamiarach: 'Przechodzę po lewej stronie'

Światła LED

Kolory sygnalizują stan: zielony = gotowy, żółty = zatrzymanie, czerwony = ostrzeżenie, niebieski = naładowanie

Gestykulacja

Ramię wskazuje kierunek lub zaprasza; kiwnięcie głowy potwierdza. Wymaga platformy z ramionami

Wyswietlacz twarzy

Animowane oczy lub emotikonki: przekazują stan emocjonalny (zdziwienie, zaproszenie, przepraszam)

Ruch anticipacyjny

Robot zaczyna skrecać wcześniej niż wymagane, sygnalizując zamiar. Badania: anticipatory motion zmniejsza konflikty o 40%

Personalizacja głosu

Ton, tempo i język dopasowane do profilu użytkownika. Dziecko: wolniej, prostsze słowa. Senior: większe natężenie

Pipeline personalizacji HRI - od kontaktu do adaptacji

1

Identyfikacja użytkownika

Rozpoznawanie twarzy /
tokenu RFID / konta;
ładowanie lub tworzenie
profilu użytkownika

2

Ocena kontekstu

Środowisko (szpital/biuro),
godzina dnia, tłok; wybór
bazowej polityki nawigacyjnej

3

Inicjacja interakcji

Kontakt wzrokowy, kiwnięc
lub głos; robot sygnalizuje
gotowość do interakcji
i zamiar ruchu

4

Wykonanie z monitoringiem

Nawigacja z aktywna
obserwacja sygnalizacji
zwrotnej; rejestracja
odległości, reakcji ciała, słów

Ewaluacja nawigacji społecznej: metryki i benchmarki

- **Intrusion into Personal Space (IPS):** procent czasu, gdy robot narusza strefe osobista ($< 1,2$ m) bez zaproszenia
- **Path Irregularity (PI):** odchylenie trasy robota od trajektorii bez pieszych; wskaźnik zakłocen w nawigacji
- **Social Space Violation (SSV):** liczba wejść w O-space grupy lub strefe konwersacji
- **Time-to-pass (TTP):** czas potrzebny robotowi do minięcia pieszego; im krótszy tym mniej ograniczeń
- **Perceived Safety Rating (PSR):** subiektywna ocena bezpieczeństwa przez pieszych (kwestionariusz Likerta)
- **Comfort Score (CS):** średnio oceniana wygoda interakcji; zbierany przez aplikacje mobilną lub kiosk
- **Benchmarki:** SocNav1/2 (Manso et al.), SEAN-EP (Tsoi et al.), CrowdNav (Chen et al.); porównywalne wyniki

Klasyfikator profilu użytkownika i adaptacja parametrów

```
import numpy as np

PROFILE_PARAMS = {
    'senior': {'v_max': 0.4, 'dist_safe': 1.5, 'tts_speed': 0.8},
    'child': {'v_max': 0.5, 'dist_safe': 1.8, 'tts_speed': 0.9},
    'adult': {'v_max': 0.8, 'dist_safe': 1.2, 'tts_speed': 1.0},
    'worker': {'v_max': 1.0, 'dist_safe': 1.0, 'tts_speed': 1.1},
}

def classify_user(face_age: float, walking_speed: float,
                 uses_aid: bool) -> str:
    """Prosty klasyfikator profilu użytkownika."""
    if uses_aid or face_age > 65:
        return 'senior'
    if face_age < 14:
        return 'child'
    if walking_speed > 1.2:
        return 'worker'
    return 'adult'

def get_nav_params(profile: str, pref_model) -> dict:
    base = PROFILE_PARAMS[profile].copy()
    # Nadpisz dystans z modelu Bayesowskiego (z Kodu 3)
    base['dist_safe'] = pref_model.mu
    return base
```

Opis: Klasyfikacja wstępna z szybkich cech; model Bayesowski (Kod 3) następnie personalizuje dist_safe online.

Studium przypadku: robot w szpitalu wielokulturowym

- Platforma: robot usługowy na korytarzach szpitalnych; personel z 12 krajów, pacjenci różnych kultur i grup wiekowych
- System detekcji: YOLO dla pieszych + MediaPipe Pose dla orientacji + gaze estimation dla kontaktu wzrokowego
- Profil domyślny: strefa 1,5 m, prędkość max 0,5 m/s, komunikacja głosowa po polsku lub angielsku
- Adaptacja kulturowa: detekcja języka (ASR) uaktywnia moduł proksemiki (+20% dystansu dla kultur nordyckich)
- Adaptacja do seniora: twarz + laska / wózek -> tryb senior: $v_{\text{max}} = 0,4 \text{ m/s}$, dystans 1,8 m, głos wolniejszy
- Adaptacja do dziecka: wiek < 14 z rozpoznania twarzy -> tryb dziecko: jeszcze większy dystans, komunikacja prostsza
- Wyniki: IPS zredukowane o 62% względem klasycznej nawigacji, PSR wzrost o 1,8 pkt (skala 1–7), TTP o 8% dłuższy

Wyzwania i kierunki badan w nawigacji społecznej

Wyzwania techniczne i społeczne

- Niezawodność detekcji emocji i gestów w różnych warunkach oświetlenia i kątach widzenia
- Translacja norm kulturowych na parametry numeryczne bez stereotypizacji
- Zimny start nowego użytkownika: brak danych historycznych utrudnia personalizację
- Konflikty norm: jedna osoba prosi o szybszy ruch, druga w tej samej chwili o zwolnienie
- Prywatność: biometryczna identyfikacja użytkownika wymaga zgody i zasad RODO
- Obliczenia w czasie rzeczywistym: wielowartościowa mapa kosztów społecznych musi być odświeżana < 100 ms

Kierunki badan (2024-2026)

- Foundation models dla HRI: VLM jako zrozumiały interpreter sygnałów społecznych
- Uczenie przez demonstrację: robot uczy się norm z filmów zachowań pieszych bez ręcznie zdefiniowanych reguł
- Teorie umysłu (ToM): robot wnioskuje o intencjach i stanach psychicznych ludzi z obserwacji
- Swoj tłum vs obcy tłum: wielorobotowe systemy społecznej nawigacji z konsensem norm grupowych
- Ewaluacja długookresowa: badania wielomiesięczne w szpitalach i biurach; mierzenie zaufania w czasie

PODSUMOWANIE

01

Social Force Model i ORCA dają deterministyczne podstawy unikania tłumu; modele danych (Transformer) przewidują intencje.

02

Proksemika Halla definiuje strefy, które robot musi szanować; naruszenie strefy osobistej niszczy zaufanie użytkowników.

03

Normy kulturowe istotnie różnią się w skali globalnej; robot działający w wielu krajach wymaga modułu adaptacji kontekstu kulturowego.

04

Personalizacja HRI opiera się na profilu użytkownika, uczeniu online preferencji i wielokanałowej komunikacji niewerbalnej.

05

Ewaluacja nawigacji społecznej wymaga metryk jakości HRI (ITT, PSR, PFC) obok klasycznych wskaźników nawigacji (RMSE, sukces).

Zadanie projektowe i pytania problemowe

- Dlaczego SFM może generować oscylacje w wąskim korytarzu? Zaproponuj modyfikacje parametrów lub dodatkowe ograniczenie, które usunie ten problem.
- Robot wchodzi między dwie osoby rozmawiające (O-space). Jakie sygnały zachowania powinny temu zapobiec i jak je zakodować w mapie kosztów?
- Użytkownik z Japonii oczekuje dystansu 1,5 m, a użytkownik z Brazylii 0,7 m. Jak w jednym systemie obsługiwać obie preferencje bez ręcznie wprowadzonych warunków?
- Zadanie: zaimplementuj klasę SocialCostmap z anizotropowym Gaussem (Kod 2) i przetestuj ją na symulowanej scenie z 5 pieszymi poruszającymi się w różnych kierunkach. Zwizualizuj wynikową mapę kosztów i sprawdź, że planista A* omija ludzi z boku, nie z przodu.
- Bonus: zintegruj DistancePreferenceModel (Kod 3) z Nav2 jako dynamiczny mnożnik strefy inflacji i pokaz zbieżność preferencji dla 3 różnych profili użytkowników w symulacji.

NAWIGACJA ROBOTOW HUMANOIDALNYCH

WYKŁAD 8

Uczenie przez wzmocnienie
w nawigacji i lokomocji humanoidów

dr inż. Mateusz Pomianek



Podstawy RL

PPO / SAC / TD3

Sim-to-real

Lokomocja + Nav

Plan wykładu

Sekcja 1

Podstawy RL: MDP, polityka, nagroda, algorytmy model-free (PPO, SAC, TD3) - od intuicji do wzoru

Sekcja 2

RL w lokomocji humanoidów: trenowanie chodu, biegania, wchodzenia na schody w MuJoCo / Isaac Sim

Sekcja 3

RL w nawigacji społecznej i integracja z ROS 2: reward shaping, curriculum, sim-to-real, wdrożenie

Kody Python

Pętla RL (Gym), PPO sketch, reward humanoid, curriculum, node ROS 2 z polityką RL

01

SEKCJA

Podstawy uczenia przez wzmocnianie

*MDP, polityka, nagroda, Value function, algorytmy
model-free*



Intuicja: uczenie przez doświadczenie

- **Dziecko uczy się chodzić przez tysiące prób i upadków:** nikt nie pisze mu równania kinematyki
- **Każda próba dostarcza sygnału:** upadłeś (kara) lub zrobiłeś krok (nagroda) - mózg aktualizuje strategię
- **Po wystarczająco wielu próbach** dziecko chodzi płynnie, bez świadomej analizy każdego mięśnia
- **Reinforcement Learning (RL) replikuje ten mechanizm:** agent (robot) + środowisko + nagroda r_t
- **Klucz:** robot NIE potrzebuje modelu dynamiki ani ręcznie zakodowanych reguł - uczy się sam z interakcji
- **Przewaga nad klasycznym sterowaniem:** generalizacja do nieprzewidzianych warunków (nierówny teren, pchnięcia)
- **Wyzwanie:** potrzeba milionów kroków interakcji; w robotyce fizycznej = dni; stąd symulacja jako pośrednik

Markowski Proces Decyzyjny (MDP): formalizacja RL

- **MDP definiują:** S - przestrzeń stanów, A - przestrzeń akcji, $P(s'|s, a)$ - model przejścia, $r(s, a)$ - nagroda, γ - współczynnik dyskontowania
- **Stan s_t :** dla humanoida = $[q, dq, orientacja_IMU, kontakt_stop, cel_xyz]$ - typowo 40-80 wymiarów
- **Akcja a_t :** komendy momentów stawowych [Nm] lub docelowe kąty stawów [rad] - 12-30 wymiarów
- **Nagroda r_t :** funkcja zaprojektowana przez inżyniera; klucz do sukcesu całego treningu
- **Cel agenta:** maksymalizacja sumy zdyskontowanych nagród $G_t = \sum_{k=0}^{\infty} \gamma^k * r_{t+k}$
- **Polityka $\pi(a|s)$:** funkcja mapująca stan na rozkład akcji; parametryzowana siecią neuronową θ
- **Value function $V(s) = E[G_t | s_t = s]$:** ocena jak dobry jest dany stan przy optymalnej polityce

Porównanie algorytmów RL model-free

PPO (Proximal Policy Opt.)

- **On-policy**: uczy się z bieżącej polityki
- **Clip loss**: kontroluje wielkość aktualizacji
- **Stabilny**, łatwy w implementacji
- **Popularny w lokomocji** (IsaacGym)
- **Wymaga dużo danych** (on-policy)
- **Ref**: Schulman et al., 2017

SAC (Soft Actor-Critic)

- **Off-policy**: replay buffer, probkowanie
- **Maksymalizuje nagrodę** + entropia
- **Probkowanie efektywniejsze** niż PPO
- **Nadaje się do zadań ciągłych**
- **Dwa krytycy Q1, Q2** - stabilność
- **Ref**: Haarnoja et al., 2018

TD3 (Twin Delayed DDPG)

- **Off-policy**, deterministyczna polityka
- **Dwa Q-networks**: redukuje overestimation
- **Opozniła aktualizacja aktora**
- **Szybki w zadaniach manipulacji**
- **Dobry punkt startowy** dla robotyków
- **Ref**: Fujimoto et al., 2018

Architektura aktor-krytyk (Actor-Critic)

Stan s_t

Propriocepcja + cel: $[q, dq, IMU, contact, goal_dir]$ - wejście do obu sieci

Aktor $\pi(a|s)$

MLP $3 \times [512 \text{ ELU}]$: wyjście μ i $\log_sigma$ dla rozkładu Gaussa nad akcja a_t

Probkowanie akcji

$a_t \sim N(\mu, \sigma^2)$; reparametrization trick:
 $a = \mu + \sigma * \epsilon$, $\epsilon \sim N(0,1)$

Srodowisko / robot

Wykonaj a_t ; odbierz r_t, s_{t+1} ; dane do rollout buffer (PPO) lub replay (SAC)

Krytyk $V(s) / Q(s,a)$

MLP $3 \times [512 \text{ ELU}]$: szacuje wartosc stanu lub pary (stan, akcja); blad TD $\rightarrow$ gradient

Kluczowe cechy

- **Aktor i krytyk** współdzielą warstwy w PPO
- **Typowy MLP**: 3 warstwy po 256-512 neuronów
- **Aktywacja**: ELU lub Tanh (ostatnia warstwa)
- **Batch norm** lub **layer norm**: stabilność
- lr aktor $3e-4$; lr krytyk $1e-3$
- **Gradient clipping**: $max_norm = 1.0$
- **Horyzont rollout PPO**: $T = 24$ kroków

Petla RL: MuJoCo Humanoid z Gymnasium

```
import gymnasium as gym
import numpy as np

env = gym.make("Humanoid-v5", render_mode=None)
obs, _ = env.reset(seed=42) # wektor stanu (376,)
ep_rewards = []

for episode in range(5):
    obs, _ = env.reset()
    ep_ret = 0.0
    done = False
    while not done:
        # Polityka - tu: losowa (zastap siecia PPO/SAC)
        action = env.action_space.sample() # (17,) momenty
        obs, reward, terminated, truncated, info = env.step(action)
        ep_ret += reward
        done = terminated or truncated
    ep_rewards.append(ep_ret)
    print(f"Epizod {episode}: return = {ep_ret:.1f}")
env.close()
print(f"Średni zwrot z 5 epizodów: {np.mean(ep_rewards):.1f}")
```

Opis: Gymnasium Humanoid-v5: 376-wymiarowy stan, 17 momentów stawowych; pełna dynamika MuJoCo 3.x.

02

SEKCJA

RL w lokomocji humanoidów

Trenowanie chodu, biegania, schodów; funkcja nagrody; curriculum learning



Środowiska symulacyjne dla lokomocji RL

MuJoCo / Gymnasium

Klasyczny benchmark; Humanoid-v5, HumanoidStandup; elastyczny; Python-native; open-source od 2022

IsaacGym / IsaacLab

GPU-parallel: 4096+ robotów jednocześnie na jednym GPU; PPO w 10 min; NVIDIA; open-source

IsaacSim (Omniverse)

Fotorealistyczna symulacja; ROS 2 native; RTX rendering; domyślne środowisko dla Franka, H1

PyBullet

Lekki, open-source; wolniejszy od Isaac; dobry do prototypowania; `pip install pybullet`

Genesis (2024)

Python-native; 10-100× szybszy od MuJoCo na CPU; diff physics; nowy standard akademicki

Webots

Open-source; wbudowany ROS 2 driver; dobry dla studentów; słabsza dokładność kontaktu

Projektowanie funkcji nagrody dla chodu humanoida

- **r_velocity**: $+v_x * \cos(\text{yaw_err})$ - nagroda za ruch do przodu we właściwym kierunku; skaluje prędkość liniowo
- **r_alive**: $+0.5$ w każdym kroku w którym robot nie upadł; krytyczne dla uczenia wstawania
- **r_upright**: $+1.0 * (1 - |\text{pitch}| / \pi)$ - nagroda za pionową postawę tułowia (brak pochylenia)
- **r_feet_contact**: $+0.1$ gdy przynajmniej jedna stopa dotyka podłoża; kara za brak kontaktu obu stop
- **r_energy**: $-0.001 * \sum(|\tau_i| * |\dot{q}_i|)$ - kara za zbyt duże zużycie mocy aktuatorów
- **r_action_smooth**: $-0.01 * ||a_t - a_{t-1}||^2$ - kara za gwałtowne zmiany komend (drgania)
- **r_total** = **r_velocity** + **r_alive** + **r_upright** + **r_feet** + **r_energy** + **r_smooth**; suma aż do upadku (done=True)

Curriculum learning - stopniowe trudności treningu

1

Faza 1: Wstawanie

Cel: robot uczy się stać. Nagroda tylko za `r_upright` + `r_alive`. Brak zadania prędkości. ~10M kroków.

2

Faza 2: Chód wolny

Wprowadź `r_velocity` z docelowym `v_ref = 0.5 m/s`. Płaska, gładka nawierzchnia. ~50M kroków.

3

Faza 3: Prędkość

Stopniowo zwiększ `v_ref` do `1.5 m/s`. **Domain rand:** masa +/-10%, tarcie zmienne. ~100M kroków.

4

Faza 4: Teren

Nierówności (heightfield), schody, rampy. Korekta funkcji nagrody o **margins ZMP**. ~200M kroków.

Funkcja nagrody dla chodu humanoida (PyTorch / IsaacLab)

```
import torch

def compute_reward(obs: torch.Tensor,          # (N, obs_dim)
                  action: torch.Tensor,        # (N, act_dim)
                  prev_action: torch.Tensor,
                  alive: torch.Tensor,         # (N,) bool
                  cfg) -> torch.Tensor:

    vx      = obs[:, 0]                        # prędkość do przodu [m/s]
    pitch   = obs[:, 6]                        # pochylenie tułowia [rad]
    tau     = obs[:, 20:37]                    # momenty stawów [Nm]
    dq      = obs[:, 37:54]                    # prędkości stawów [rad/s]
    r_vel    = vx * cfg.w_vel                  # nagroda prędkości
    r_alive  = alive.float() * cfg.w_alive     # przeżycie
    r_uprt   = (1.0 - pitch.abs() / 3.14) * cfg.w_upr # pionowość
    r_ener   = -(tau * dq).abs().sum(-1) * cfg.w_ener # energia
    r_smth   = -(action - prev_action).pow(2).sum(-1) * cfg.w_sm
    return r_vel + r_alive + r_uprt + r_ener + r_smth
```

Opis: `cfg.w_vel=1.0`, `w_alive=0.5`, `w_upr=0.3`, `w_ener=0.001`, `w_sm=0.005` - startowe wagi do strojenia

Domain randomization i transfer sim-to-real

Domain Randomization (DR)

- **Losowanie parametrów modelu podczas treningu:** masa ciała $\pm 15\%$, momenty bezwładności $\pm 10\%$
- **Tarcie stopy:** u w $[0.4, 1.2]$; kluczowe dla różnych nawierzchni
- **Opóźnienie aktuatora:** 0-40 ms; symuluje opóźnienie sieci i kontrolera
- **Szum obserwacji:** $N(0, \sigma)$ do q, dq , IMU - odporność na błędy enkodera
- **Zewnętrzne zakłócenia:** losowe impulsy siły na tułów co N kroków
- **Cel:** pokryć różnice między symulacją a fizycznym robotem

Transfer sim-to-real (best practices)

- **Fine-tuning na robocie:** zamroź aktora, douczaj tylko ostatnią warstwę (1h danych = 30 min treningu)
- **History buffer:** podaj ostatnie $k=3$ obserwacje i akcje; kompensuje opóźnienia
- **Asymetryczny AC:** krytyk ma dostęp do pełnego stanu w sim; aktor uczy się tylko z obserwacji
- **System identification:** dopasuj tarcie i masę przed treningu przez krótkie sesje na robocie
- **Metryki:** v_{forward} , margines ZMP, zużycie energii porównywalne sim vs. real

Wyniki RL w lokomocji humanoidow (2022-2025)

DeepMind - Harness

PPO + MuJoCo; humanoid uczy się wstawać, chodzić, biegać z jednej politykami. Nature 2019.

ETH - ANYmal RL

PPO + IsaacGym; quadruped RL do 3 m/s; transfer sim2real w 15 min kalibracji. RSJ 2022.

UC Berkeley - Cassie

PPO + MuJoCo; bieganie 3 m/s, kucanie, skakanie; odporność na pchnięcia. RSS 2022.

Agility Digit - IROS24

SAC + curriculum; chód po terenie, schody, ramp; transfer na fizycznego Digit ~4h danych.

Unitree H1 - 2024

PPO + IsaacLab; bieg 3,3 m/s z backflip; 2048 środowisk równoległych na jednym GPU.

Figure 01 - 2024

RL + VLA (GPT-4o); humanoid może nauczyć się pieczenia jajek, sprzątania; end to end z LLM

03

SEKCJA

RL w nawigacji i integracja z ROS 2

*Nawigacja społeczna z RL, reward shaping, wdrożenie
na robocie*



RL dla nawigacji społecznej: reward shaping

- **Cel:** robot uczy się omijać pieszych bez ręcznie zaprojektowanego planisty lokalnego
- **r_goal:** $-d(\text{pos}_t, \text{goal}) / d_{\text{initial}}$ - gesta nagroda za zbliżanie się do celu nawigacyjnego
- **r_social:** $-\sum_j [\max(0, d_{\text{safe}} - ||\text{pos} - \text{ped}_j||)]$ - kara proporcjonalna do naruszenia strefy osobistej
- **r_collision:** -10 gdy $||\text{pos} - \text{ped}|| < 0.3$ m lub kolizja ze ścianą; epizod kończy się
- **r_comfort:** $-||a_t - a_{t-1}||^2 * 0.01$ - kara za gwałtowne manewry (komfort pieszego)
- **r_progress:** +0.2 gdy robot przeszedł do nowego punktu kontrolnego; hamuje tendencje do okrążania celu
- **Środowisko:** CrowdNav (Chen et al.) lub SEAN-EP; 5-15 pieszych z Social Force Model lub nagraniami

Hierarchiczne RL dla nawigacji humanoidów

Wysoki poziom - nawigacja

- **Policy nawigacyjna:**

$$\begin{aligned} obs &= [cel_{dir}, dist_{goal}, pedestrians_{relative}] \\ \rightarrow a &= [v, omega] \end{aligned}$$

- **Trening:** CrowdNav / IsaacSim; 8 pieszych, randomizowane cele
- **Częstotliwość:** 10 Hz; wyjście: wektor $(v, omega)$ do planisty kroków
- **Transfer:** fine-tuning na danych z korytarza (30 min sesji)
- **Algorytm:** SAC; replay buffer 1M; batch 256; lr $3e-4$

Niski poziom - lokomocja

- **Policy lokomocyjna:**

$$\begin{aligned} obs &= [v_{cmd}, omega_{cmd}, q, dq, IMU, contact] \\ \rightarrow a &= tau \end{aligned}$$

- **Trening:** IsaacGym; 2048 środowisk; 200M kroków; PPO
- **Częstotliwość:** 200 Hz (WBC loop integration)
- **Odbiera komendy od planisty** nawigacyjnego co 50 ms
- **Algorytm:** PPO; clip = 0.2; T = 24; mini-batches = 4

Uproszczona polityka nawigacyjna RL (SAC)

```
import torch, torch.nn as nn
import numpy as np

class NavPolicy(nn.Module):
    def __init__(self, obs_dim=18, act_dim=2, hidden=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, hidden), nn.ELU(),
            nn.Linear(hidden, hidden), nn.ELU(),
            nn.Linear(hidden, act_dim) # mu [v, omega]
        )
    def forward(self, obs: torch.Tensor) -> torch.Tensor:
        mu = self.net(obs)
        # Ogranicz v in [-0.5, 1.0] m/s; omega in [-1.0, 1.0] rad/s
        v = torch.tanh(mu[:, 0]) * 0.75 + 0.25 # preferencja do przodu
        omega = torch.tanh(mu[:, 1]) * 1.0
        return torch.stack([v, omega], dim=-1)
```

Opis: obs_dim=18: [goal_dx, goal_dy, dist_goal, 5x{ped_dx,dy,vx,vy,dist}]; output: (v, omega) do lokomocji.

Integracja polityki RL z ROS 2

- **Node /rl_nav_policy:** subskrybuje /odom, /goal_pose, /pedestrians; publikuje /rl_cmd_vel (10 Hz)
- **Node /rl_loco_policy:** subskrybuje /joint_states, /imu, /rl_cmd_vel; publikuje /joint_torques (200 Hz)
- **Wczytanie wag:** `torch.load("nav_policy.pt").eval(); torch.jit.trace()` do 2× szybszej inferencji
- **Latencja:** NavPolicy ~0.8 ms; LocoPolicy ~1.2 ms na Jetson AGX Orin - mieści się w cyklu sterowania
- **Safety wrapper:** przed każdą akcją sprawdź clip prędkości i ZMP; watchdog zatrzymuje przy przekroczeniu
- **Fallback:** gdy RL policy nie odpowie przez >30 ms, aktywuj klasyczny DWA kontroler z Nav2
- **Logowanie:** zapisuj (obs, action, reward) do rosbag2 - dane do dalszego fine-tuningu online

ROS 2 node - polityka nawigacyjna RL (Python)

```
import rclpy, torch
from rclpy.node import Node
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry

class RLNavNode(Node):
    def __init__(self):
        super().__init__("rl_nav")
        self.policy = torch.jit.load("nav_policy_traced.pt").eval()
        self.goal = None # PoseStamped goal
        self.peds = [] # lista pieszych z detektora
        self.cmd_pub = self.create_publisher(Twist, "/rl_cmd_vel", 1)
        self.create_subscription(Odometry, "/odom", self.odom_cb, 1)
        self.create_timer(0.1, self.timer_cb) # 10 Hz

    def timer_cb(self):
        if self.goal is None: return
        obs = self._build_obs() # (1, 18) tensor
        with torch.no_grad():
            action = self.policy(obs).squeeze().numpy()
        cmd = Twist()
        cmd.linear.x = float(action[0].clip(-0.5, 1.0))
        cmd.angular.z = float(action[1].clip(-1.0, 1.0))
        self.cmd_pub.publish(cmd)
```

Opis: torch.jit.load() ładuje skompilowany TorchScript - 2x szybszy i niezależny od kodu treningu.

Ograniczenia i otwarte pytania RL w humanoidach

Problemy techniczne i badawcze

- **Kosztowny trening:** 100M kroków PPO to ~ 2 h na $4 \times A100$; realny robot = tygodnie bez DR
- **Reward hacking:** agent nagminnie oszukuje nagrodę nieoczekiwanymi ruchami (np. toczy się zamiast chodzić)
- **Długookresowe planowanie:** RL słabo radzi sobie z horyzontami > 50 kroków bez hierarchii lub modelu świata
- **Generalizacja topologii:** polityka nauczona w jednym budynku może nie działać w nowym
- **Interpretowalność:** sieć neuronowa nie podaje powodu decyzji; trudność certyfikacji bezpieczeństwa

Kierunki (2024-2026)

- **RL + WMA:** DreamerV3 jako efektywny solver MDP z wyobraznia (patrz Wykład 6)
- **RLHF (od człowieka):** ludzkie pary preferencji jako sygnał nagrody zamiast ręcznie projektowanej r
- **Foundation policies:** jeden model pre-trained na 100 robotach (Open-X-Embodiment + PPO)
- **Safe RL:** constrained MDP z gwarancją $P(\text{kolizja}) < \delta$; certifiable przez conformal prediction
- **Multi-agent RL:** kilka humanoidów uczących się współpracować lub omijać się nawzajem

RL vs klasyczne planowanie vs WMA

Klasyczny planista (A*/DWA)

- **Deterministyczny**, formalnie weryfikowalny
- **Wymaga gotowej mapy** i modelu dynamiki
- **Szybki** w czasie wykonania (ms)
- **Nie uczy się** z doświadczenia
- **Słabe przy dużej nieliniowości** i zakłóceniach
- **Najlepszy**: znane środowisko, cert. bezpieczeństwo

Model-based RL / WMA

- **Planowanie w wyobraźni**: 10-100× mniej danych
- **Generalizacja** z modelu przejścia
- **Może być interpretowany** przez model świata
- **Wymaga treningu** RSSM / TDMPC2
- **Średnio skuteczny** w dl. horyzoncie
- **Najlepszy**: ograniczone dane, bezpieczne uczenie

Model-free RL (PPO/SAC)

- **Nie wymaga modelu dynamiki**
- **Skaluje się**: 2048 środowisk w Isaac
- **Najlepsza wydajność** w dl. treningu
- **Wymaga dużo danych**; może hakować nagrodę
- **Trudny do certyfikacji**
- **Najlepszy**: duża zdolność obliczeniowa, sim2real

PODSUMOWANIE WYKLADU 8

01

MDP i algorytmy PPO/SAC/TD3 to fundamenty RL; sieć aktor-krytyk uczy się polityki przez interakcje z symulacją.

02

Funkcja nagrody i curriculum są kluczowe: źle zaprojektowana nagroda prowadzi do reward hackingu zamiast chodzenia.

03

Domain randomization + asymetryczny AC to sprawdzone recepty na udany transfer sim-to-real w mniej niż 2h kalibracji.

04

Hierarchiczne RL rozdziela nawigację (10 Hz) od lokomocji (200 Hz); oba poziomy integrują się z ROS 2 przez torchscript.

05

RL, WMA i klasyczny planista są komplementarne; optymalna architektura łączy je w hierarchii zależnie od horyzontu i ryzyka.

Zadanie projektowe i pytania problemowe

1. Dlaczego polityka RL musi mieć dostęp do historii obserwacji (history buffer) przy transferze sim-to-real? Jaką długość bufora jest wystarczająca i dlaczego?
2. Zaprojektuj funkcje nagrody dla humanoida uczącego się wchodzić po schodach o nieznanej wysokości stopni (10-20 cm). Jakie składniki nagrody są niezbędne, a jakie mogą prowadzić do reward hackingu?
3. Wyjaśnij różnice między on-policy (PPO) a off-policy (SAC) w kontekście ograniczonego czasu treningu na GPU. Kiedy wybrać jeden, a kiedy drugi?

Zadanie: zaimplementuj pętle RL z Gymnasium Humanoid-v5 (Kod 1) i zastąp losową politykę najprostszym możliwym aktorem (MLP 2 warstwy, tanh, opt Adam). Porównaj średni return po 10k i 100k krokach.

Bonus: zintegruj TorchScript policy (Kod 4) z Nav2 w Gazebo i porównaj czas dotarcia do celu z 5 pieszymi względem klasycznego DWB kontrolera.

NAWIGACJA ROBOTOW HUMANOIDALNYCH

WYKŁAD 9

Bezpieczeństwo, certyfikacja
i wdrażanie systemów nawigacji humanoidów

dr inż. Mateusz Pomianek



Normy ISO / IEC

Runtime monitoring

Testing / Validation

Wdrożenie / DevOps

Plan wykładu

Sekcja 1

Normy i standardy: ISO 13482, IEC 61508, EN ISO 10218 - wymagania formalne dla robotów usługowych

Sekcja 2

Bezpieczeństwo w czasie wykonania: runtime monitors, fail-safe, conformal prediction, hardware e-stop

Sekcja 3

Testowanie i walidacja: HIL/SIL/PIL, scenariusze testowe, benchmarki, metryki jakości nawigacji

Sekcja 4

Wdrażanie i utrzymanie: architektura floty, OTA updates, monitoring operacyjny, etyka i perspektywy

01

SEKCJA

Normy i standardy bezpieczeństwa

ISO 13482, IEC 61508, poziomy SIL/PLe i wymagania formalne



Dlaczego normy bezpieczeństwa są obowiązkowe?

- Robot humanoidalny poruszający się wśród ludzi może zadać uraz fizyczny lub psychiczny; odpowiedzialność prawna producenta
- **Normy zmniejszają ryzyko:** definiują systematyczny proces identyfikacji zagrożeń (HAZOP, FMEA) i ich redukcji
- **ISO 13482:2014:** norma dedykowana robotom usługowym (personal care robots); obejmuje wymogi bezpieczeństwa fizycznego i funkcjonalnego
- **IEC 61508:** bezpieczeństwo funkcjonalne systemów elektrycznych i elektronicznych; definiuje poziomy SIL (Safety Integrity Level)
- **EN ISO 10218-1/2:** bezpieczeństwo robotów przemysłowych; stosowany gdy humanoid pracuje wspólnie z operatorem
- **CE Marking (dyrektywa maszynowa 2006/42/WE):** warunek konieczny do sprzedaży robota w UE
- **Brak certyfikatu = zakaz używania** w przestrzeniach publicznych i szpitalach w Polsce i UE

Poziomy nienaruszalności bezpieczeństwa SIL i PLe

SIL: Safety Integrity Level (IEC 61508)

- **SIL 1:** prawdopodobieństwo awarii niebezpiecznej 10^{-5} - $10^{-6}/h$ (np. robot w zamkniętej strefie)
- **SIL 2:** 10^{-6} - $10^{-7}/h$; wymagany dla robotów współpracujących z operatorem
- **SIL 3:** 10^{-7} - $10^{-8}/h$; roboty medyczne i w infrastrukturze krytycznej
- **SIL 4:** $< 10^{-8}/h$; lotnictwo, energetyka jądrowa - praktycznie nieosiągalny w robotyce mobilnej
- **Wyznaczyć SIL** musi jednostka notyfikowana; wymaga analizy FMEA i FTA

PLe: Performance Level (EN ISO 13849)

- **PL a-e:** od najniższego do najwyższego poziomu niezawodności funkcji bezpieczeństwa
- **PLe:** MTTFd > 100 lat; DC $\geq 99\%$; wymagany dla e-stop i watchdog-a robota mobilnego
- **PLd:** MTTFd 30-100 lat; DC $\geq 90\%$; typowy dla ograniczników prędkości i torque limits
- **Walidacja PLe:** testy starzeniowe, analiza FMEA, audyt przez TUV lub notyfikowana jednostkę
- **PLe często wymaga redundancji:** dwa niezależne czujniki kontaktu lub dwa CPU

Analiza ryzyka: HAZOP i FMEA dla humanoida

- **HAZOP (Hazard and Operability Study):** systematyczne przegląd wszystkich węzłów systemu pod kątem odchyień od normalnej pracy
- **Przykład HAZOP:** "robot nie wykryje pieszego" -> zagrożenie: kolizja -> przyczyna: zaślepiiony LiDAR -> bariera: redundantna kamera
- **FMEA (Failure Mode and Effects Analysis):** tabelaryczna analiza trybów awarii każdego komponentu i ich skutków
- **Parametry FMEA:** S - powaga skutku (1-10), O - częstotliwość wystąpienia (1-10), D - wykrywalność (1-10)
- **RPN = S × O × D;** komponenty z RPN > 100 wymagają natychmiastowej modyfikacji lub redundancji
- **Dla humanoida krytyczne komponenty:** IMU, LiDAR, system e-stop, aktuatory bioder i kolan
- **Wymagane dla CE Marking:** pełna dokumentacja analizy ryzyka przed pierwszym wdrożeniem publicznym

Wymagania ISO 13482 dla robota nawigującego wśród ludzi

Ograniczenie prędkości

Max 0,5 m/s w trybie współpracy;
max 1,5 m/s w strefie wydzielonej.
Nadzór sprzętowy.

Sily kontaktowe

Siła nacisku < 50 N na dowolną część ciała; < 35 N dla głowy i szyi. Czujnik F/T obowiązkowy.

Zatrzymanie awaryjne

E-stop: zatrzymanie w < 500 ms od wciśnięcia przycisku; możliwość ręcznego zatrzymania przez operatora.

Detekcja człowieka

Robot musi wykryć osobę w strefie bezpieczeństwa 1 m i zareagować w < 200 ms.

Instrukcje i oznakowanie

Podręcznik użytkownika, etykiety ostrzegawcze, sygnały dźwiękowe i świetlne dla każdego trybu.

Walidacja eksperymentalna

Min. 1000 h testów w środowiskach reprezentatywnych przed certyfikatem; log incydentów.

02

SEKCJA

Bezpieczeństwo w czasie wykonania

Runtime monitors, fail-safe, conformal prediction, e-stop



Architektura bezpieczeństwa runtime: warstwy nadzoru

Warstwa 1: sprzęt

PLc hardware e-stop; wylamywacze momentow;
enkodery redundantne; zasilanie awaryjne

Warstwa 2: RTOS watchdog

Dedykowany MCU 1 kHz; nadzoruje komunikacje z
aktuatorami; resetuje przy braku heartbeat

Warstwa 3: safety monitor

Node ROS 2 lifecycle; sprawdza ZMP, predkosc, strefe
bezpieczna; veto dla planner/controller

Warstwa 4: runtime verif.

Conformal prediction lub formalne monitory STL/MTL;
certyfikuje ze $P(\text{naruszenie}) < \text{delta}$ w trybie online

Warstwa 5: operator HMI

Dashboard z alertami; tele-operacja awaryjna; przycisk
stop zdalny przez WiFi / 4G

Kluczowe cechy

- Każda warstwa działa niezależnie
- Awaria wyższej -> niższe przejma
- Latencja: warstwy 1-2 < 1 ms
- Warstwy 3-4: < 10 ms
- Warstwa 5: akceptowalne > 100 ms
- Testy redundancji co 24h w deploymentcie

Runtime monitor: co sprawdzać i kiedy

- **Prędkość liniowa i kątowa:** $|v| \leq v_max$, $|\omega| \leq \omega_max$ - sprzętowe zabezpieczenie momentów przy przekroczeniu
- **Margines ZMP:** gdy ZMP wychodzi za wielobok podparcia o > 1 cm przez > 3 cykle (15 ms) -> zmniejsz prędkość
- **Detekcja upadku:** $|\text{roll}| > 45^\circ$ lub $|\text{pitch}| > 45^\circ$ lub akcelerometr $|a| > 3g$ przez > 50 ms -> postura awaryjna
- **Strefa bliskości:** wykryty człowiek < 0.5 m -> stop natychmiastowy; < 1.0 m -> zwolnij do 0.3 m/s
- **Temperatura aktuatorów:** $> 80^\circ\text{C}$ -> ogranicz moment do 50%; $> 90^\circ\text{C}$ -> zatrzymaj i wentyluj
- **Utrata połączenia sensora:** brak wiadomości IMU przez > 50 ms -> tryb degradacji (dead reckoning + emergency stop)
- **Innowacja filtra EKF:** $NIS > \chi^2_{.95}$ przez > 5 kroków -> reset lokalizacji i stop w bezpiecznej strefie

Safety Monitor: node ROS 2 (szkielet w Python)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Bool

class SafetyMonitor(Node):
    LIMITS = dict(v_max=0.8, omega_max=1.2, temp_warn=80,
                  zmp_margin=0.01, ped_stop=0.5, ped_slow=1.0)
    def __init__(self):
        super().__init__("safety_monitor")
        self.ok_pub = self.create_publisher(Bool, "/safety_ok", 1)
        self.create_timer(0.01, self.check) # 100 Hz
        self.state = {} # wypełniane przez subskrybentów
    def check(self):
        s = self.state
        ok = (
            abs(s.get("v",0)) <= self.LIMITS["v_max"] and
            s.get("zmp_margin",99) >= self.LIMITS["zmp_margin"] and
            s.get("ped_dist",99) >= self.LIMITS["ped_stop"] and
            s.get("temp_max",0) <= self.LIMITS["temp_warn"]
        )
        self.ok_pub.publish(Bool(data=bool(ok)))
```

Opis: /safety_ok = False -> controller manager wstrzymuje komendy i inicjuje graceful stop

Conformal Prediction dla certyfikacji runtime

- **Conformal Prediction (CP):** formalna metoda statystyczna gwarantująca $P(\text{naruszenie}) \leq \alpha$ na danych kalibracyjnych
- **Idea:** zbierz M kroków kalibracyjnych historii; oblicz „nonconformity scores” $q_1..q_M$ dla wyjść polityki
- **W czasie online:** dla bieżącego kroku oblicz q_t ; jeśli $q_t > \text{kwantyl}_{\{1-\alpha\}}$ z kalibracji $\rightarrow$ ostrzeżenie
- **Gwarancja:** przy prawdziwym rozkładzie danych, fałszywy alarm z częstotliwością $\leq \alpha$ (np. 5%)
- **Zastosowanie:** certyfikacja ze polityka RL NIE narusza ZMP lub stref bezpieczeństwa z $P > 1 - \alpha$
- **Implementacja:** biblioteka MAPIE (Python); integracja jako ROS 2 node z dostępem do historii rollout
- **Odniesienie:** Angelopoulos & Bates (2022) "Conformal Risk Control"; Lindemann et al. (2023) dla robotów

03

SEKCJA

Testowanie i walidacja

HIL, SIL, PIL, scenariusze testowe, benchmarki, metryki



Poziomy testowania: SIL, HIL, PIL

SIL: Software in the Loop

- **Kod sterowania** uruchamiany w symulatorze (Gazebo, Isaac)
- **Brak fizycznego sprzętu**; wszystko wirtualne
- **Szybkie iteracje**: 1000 scenariuszy / godzinę
- Sprawdza logikę, planistę, monitor bezpieczeństwa
- **Ograniczenie**: nie wykrywa błędów sprzętu, ani opóźnień sieci
- **Narzędzia**: pytest + Gazebo, CARLA, AirSim

HIL: Hardware in the Loop

- **Prawdziwy kontroler**, aktuatory lub sterownik wpięty w symulator
- **Symulowane środowisko + realny czas reakcji** mikrokontrolera
- Wykrywa problemy z latencją komunikacji, przerwaniami
- **Koszt**: wymaga symulatora czasu rzeczywistego (dSPACE, NI)
- **Typowy krok**: po SIL, przed testami na pełnym robocie
- Narzędzia: Speedgoat, Opal-RT, NI VeriStand

PIL: Processor in the Loop

- **Docelowy procesor** (Jetson) wpięty w pętlę symulacyjną
- Kod skompilowany na target hardware; sprawdza czas rzeczywisty
- Wykrywa przepełnienia bufora, opóźnienia rzeczywiste
- Pomiedzy HIL, a pełnym testem na robocie
- Kluczowy dla weryfikacji TorchScript / ONNX polityki RL
- **Narzędzia**: ROS 2 bag replay + Jetson + symulator w chmurze

Scenariusze testowe dla nawigacji społecznej

- **TC-01 Prosta trasa 10 m:** brak przeszkod, pomiar RMSE, czasu dotarcia, zużycia energii; 10 powtórzeń
- **TC-02 Statyczna przeszkoda:** skrzynka 0.5×0.5 m w połowie trasy; robot musi ją ominąć bez wchodzenia w strefę 0.3 m
- **TC-03 Dynamiczna przeszkoda:** pieszy przecinający trasę z prędkością 1.2 m/s; metryka: TTC > 1 s w każdej chwili
- **TC-04 Naruszenie strefy osobistej:** aktor zbliży się do robota < 0.5 m; robot musi się zatrzymać w < 300 ms
- **TC-05 Pchnięcie boczne 20 Ns:** robot musi odzyskać równowagę w < 3 s; brak upadku w 95% prób
- **TC-06 Utrata IMU (30 s):** robot musi przejść do trybu degradacji i zatrzymać się bezpiecznie w < 1 s
- **TC-07 Długa misja 30 min:** patrol po budynku; metryki: liczba incydentów, dryft SLAM, zużycie akumulatora

Automatyczny benchmark nawigacji (szkielet pytest)

```
import pytest, time
import numpy as np

# Fikstura: uruchom symulator i robot driver przed testem
@pytest.fixture(scope="session")
def robot_env():
    env = SimEnv(config="corridor_5pedes.yaml") # 5 pieszych
    yield env
    env.close()

def test_tc03_dynamic_obstacle(robot_env):
    """TC-03: robot nie narusza  $TTC < 1$  s z pieszym."""
    env = robot_env
    obs = env.reset(scenario="cross_walker")
    min_ttc = float("inf")
    for _ in range(500): # 500 kroków po 100 ms = 50 s
        action = env.policy(obs)
        obs, _, done, info = env.step(action)
        min_ttc = min(min_ttc, info["ttc"])
        if done: break
    assert min_ttc >= 1.0, f"Naruszenie TTC: {min_ttc:.2f} s"
    assert env.info["success"], "Robot nie dotarł do celu"
```



Opis: TTC = Time-To-Collision; test uruchamiany w CI/CD po każdym merge do branch main

Metryki jakości nawigacji: pełna lista

- **Nawigacja geometryczna:** RMSE pozycji [m], sukces misji [%], czas dotarcia [s], zużycie energii [Wh]
- **Bezpieczeństwo:** liczba kolizji, minimum TTC [s], liczba naruszeń strefy bezpiecznej, max siłą kontaktu [N]
- **Stabilność:** margines ZMP [m], liczba korekt kroku, max odchylenie roll/pitch [deg]
- **Nawigacja społeczna:** Intrusion into Personal Space (IPS) [%], Social Space Violations (SSV) [#], Path Irregularity (PI)
- **Jakość lokalizacji:** RMSE pozycji vs. ground truth (Vicon), błąd yaw [deg], dryft SLAM [m/km]
- **Wydajność obliczeniowa:** latencja planisty [ms], obciążenie CPU/GPU [%], temperatura Jetson [°C]
- **Subiektywne:** Perceived Safety Rating (PSR, 1-7), Comfort Score (CS, 1-7), NASA-TLX dla operatora

04

SEKCJA

Wdrażanie i utrzymanie

Architektura floty, OTA updates, monitoring operacyjny, etyka



Cykl zycia systemu nawigacji humanoidalnej

1

Projektowanie

Analiza wymagań (Safety, Functional), HAZOP/FMEA, wybór stosu technologicznego i architektury systemu

2

Integracja i SIL

Budowa systemu, testy jednostkowe, integracyjne i SIL w Gazebo/Isaac; CI/CD pipeline na GitHub Actions

3

HIL i PIL

Testy na docelowym hardware (Jetson); weryfikacja latencji; testy regresji przy każdym release RC

4

Certyfikacja

Audyt przez jednostkę notyfikowaną; analiza FMEA; dokumentacja CE; testy wytrzymałościowe 1000 h

Architektura floty i zarządzanie flotą robotów

Infrastruktura centralna (cloud)

- **Fleet Manager:** rejestr robotów, przypisanie zadań, priorytetyzacja misji
- **OTA Update Server:** dystrybucja nowych polityk RL i modeli VLM przez HTTPS TLS 1.3
- **Digital Twin:** wirtualna kopia każdego robota; symulacja nowych polityk przed wdrożeniem
- **Centralne logowanie:** ELK Stack / Grafana; zbieranie rosbag2 fragmentów przy incydentach
- **A/B testing polityk:** 10% floty testuje nowszą politykę; rollback automatyczny gdy metryki pogorszone

Architektura lokalna (robot)

- **Modul OTA:** pobiera update poza godzinami pracy; walidacja sumy SHA-256 przed instalacją
- **Dual-bank OS:** aktywny bank + standby; rollback jednym poleceniem przy awarii update
- **Lokalny monitoring:** edge analytics na Jetson; alert jeśli anomalia wykryta bez połączenia z chmurą
- **Security:** mTLS między robotem a serwerem; certyfikaty X.509 rotowane co 90 dni
- **GDPR:** dane biometryczne (twarze) przetwarzane tylko lokalnie; nie wysłane do chmury

OTA Update: weryfikacja i bezpieczna instalacja polityki

```
import hashlib, os, shutil, torch

def verify_and_install_policy(url: str, expected_sha256: str,
                             policy_path="/opt/robot/policy.pt") -> bool:
    """Pobiera, weryfikuje i instaluje nowa polityke RL przez OTA."""
    import urllib.request
    tmp_path = "/tmp/policy_candidate.pt"
    # 1. Pobierz plik z serwera OTA (TLS)
    urllib.request.urlretrieve(url, tmp_path)
    # 2. Weryfikacja sumy SHA-256
    sha = hashlib.sha256(open(tmp_path, "rb").read()).hexdigest()
    if sha != expected_sha256:
        os.remove(tmp_path); return False # kompromitacja pliku
    # 3. Walidacja modelu (forward pass z testowym wejściem)
    model = torch.jit.load(tmp_path).eval()
    test_obs = torch.zeros(1, 18)
    action = model(test_obs) # sprawdz ze nie rzuca wyjątku
    assert action.shape == (1, 2), "Niepoprawny kształt wyjścia!"
    # 4. Atomiczna podmiana (backup starego)
    shutil.copy2(policy_path, policy_path + ".bak")
    shutil.move(tmp_path, policy_path) # atomiczne rename
    return True
```

Opis: Atomiczna zmiana nazwy zapobiega częściowej instalacji; SHA-256 chroni przed podmianą pliku

Monitoring operacyjny: co metryzować w produkcji

- **KPI misji:** sukces [%], średni czas wykonania [s], liczba interwencji operatora dziennie
- **Bezpieczeństwo:** incydenty/miesiąc (kolizje, zatrzymania awaryjne, upadki, skargi użytkowników)
- **Wydajność:** zużycie energii na misje [Wh], czas lądowania, czas dostępności (uptime [%])
- **Zdrowie systemu:** temperatura aktuatorów, błędy enkoderów, zużycie akumulatora (state-of-health)
- **Jakość nawigacji:** RMSE lokalizacji (spot-check z zewnętrznym systemem), dryft mapy na koniec dnia
- **ML drift:** porównanie rozkładu obserwacji online vs. dane treningowe (PSI score); alert przy > 0.2
- **Narzędzia:** Grafana + InfluxDB (metryki); ELK (logi); PagerDuty (alerty 24/7); DataDog (ML monitoring)

Aspekty etyczne i społeczne wdrażania humanoidów

- **Prywatność:** humanoid z kamerami to de facto urządzenie nadzoru; wymaga polityki RODO i zgody użytkowników
- **Transparentność:** użytkownik ma prawo wiedzieć że jest obserwowany przez robota i co system robi z danymi
- **Biasy algorytmów:** modele ML nauczone na niereprezentatywnych danych mogą gorzej rozpoznawać pewne grupy osób
- **Przyszłość pracy:** humanoid wykonujący zadania wcześniej ludzkie - kontekst społeczny i odpowiedzialność pracodawcy
- **Odpowiedzialność:** kto odpowiada za szkody - producent, operator czy użytkownik? Ramy prawne UE AI Act (2024)
- **EU AI Act:** roboty nawigujące w przestrzeni publicznej zakwalifikowane jako systemy wysokiego ryzyka (Annex III)
- **Zasada:** technika może, ale to prawo i etyka decydują czy i jak. To inżynier odpowiada więc za świadome projektowanie

PODSUMOWANIE

Nawigacja Robotów Humanoidalnych

W1

Percepcja i mapowanie

W2

Lokalizacja i fuzja

W3

Planowanie ruchu

W4

Chod i stabilność

W5

VLM / VLA

W6

World-Model Action

W7

Nawigacja społeczna

W8

Reinforcement Learning

W9

Bezpieczeństwo i wdrożenie

***Pełna pętla:** Sensory -> Percepcja -> Lokalizacja -> Planowanie -> WMA/RL/VLA -> Sterowanie -> Aktuatory -> Sensory*

PODSUMOWANIE

01

ISO 13482, SIL/PLe i FMEA to niezbędne ramy prawne i inżynierskie przed wdrożeniem humanoida wśród ludzi.

02

Architektura bezpieczeństwa musi być wielowarstwowa: hardware PLe + RTOS watchdog + runtime monitor + conformal prediction.

03

Testowanie SIL/HIL/PIL i automatyczne benchmarki (pytest + Gazebo) pozwalają wykryć regresje przed wdrożeniem na robocie.

04

OTA updates, dual-bank OS i monitoring ML drift są kluczowe dla długookresowej niezawodności floty humanoidów.

05

Aspekty etyczne (prywatność, biasy, EU AI Act) są równoważne z aspektami technicznymi - inżynier ponosi współodpowiedzialność.

Zadanie projektowe i pytania problemowe

1. Dla jakiego poziomu SIL powinien być certyfikowany czujnik F/T w stopie humanoida pracującego w szpitalu? Uzasadnij opierając się na ISO 13482 i IEC 61508.
2. Zaprojektuj scenariusz testowy TC-08 dla robota wchodzącego do windy: jakie warunki brzegowe należy sprawdzić, jakie metryki zmierzyć i ile powtórzeń jest wymagane dla statystycznej pewności?
3. Czym różni się SIL (IEC 61508) od PLe (EN ISO 13849) i dlaczego system nawigacyjny może wymagać jednocześnie obu certyfikatów?

Zadanie: napisz prosty Safety Monitor (Kod 2) i dodaj test pytest sprawdzający że monitor poprawnie ustawia */safety_ok = False* gdy *predkosc > v_max* lub *temperatura > temp_warn*. Uruchom test z symulowanymi danymi bez prawdziwego robota.

Bonus: przeanalizuj jeden publicznie dostępny raport incydentu z robotem usługowym (np. Knightscope, Softbank Pepper) i wskazać które z omawianych warstw bezpieczeństwa zawiodły lub były nieobecne.